

Intro to Programming with Python

Software

Software is a set of instructions to the hardware.

Programming

Programming means writing the instructions to create a software.

Code

The instructions that we write to create software is called **Code**.

Syntax

Similar to Grammar rules in English, Hindi, each programming language has a unique set of rules. These rules are called the **Syntax** of a Programming Language.

Why Python

Python is an easy to learn, powerful programming language. With Python, it is possible to create programs with minimal amount of code. Look at the code in Java and Python used for printing the message "**Hello World**"

Applications of Python

Python is a versatile language which has applications in almost every field

- Artificial intelligence (AI)
- Machine Learning (ML)
- Big Data
- Smart Devices/Internet of Things (IoT)
- Cyber Security
- Game Development
- Backend Development, etc.

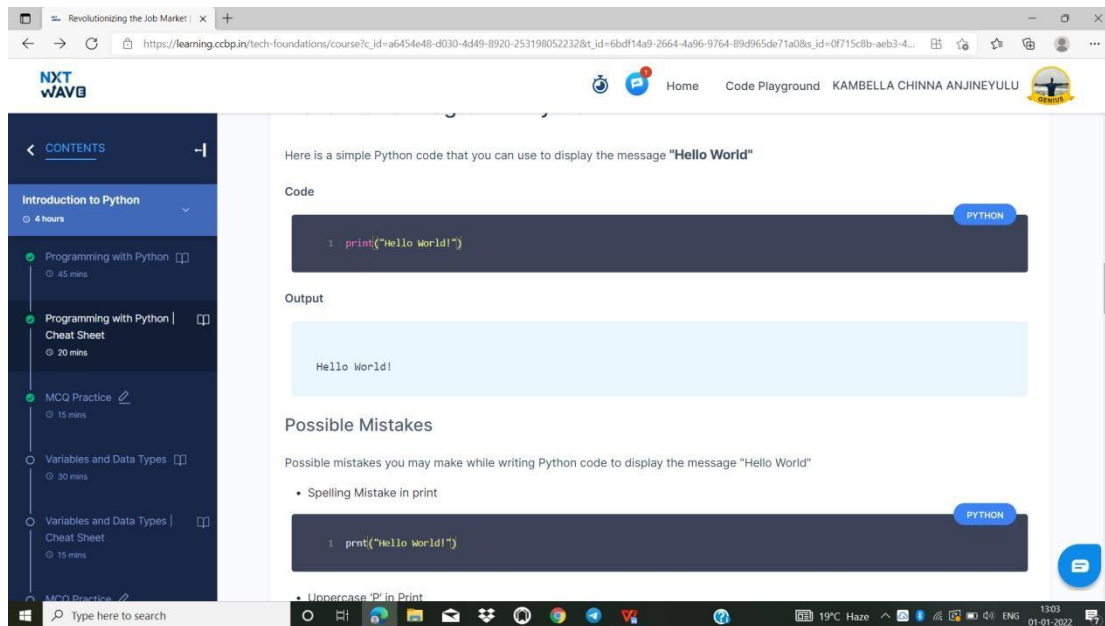
Career Opportunities

Python developers have plenty of opportunities across the world

- DevOps Engineer
- Software Developer
- Data Analyst
- Data Scientist
- Machine Learning (ML) Engineer AI Scientist, etc.

Hello World Program in Python

Here is a simple Python code that you can use to display the message "**Hello World**"



Printing Without Quotes

If we want to print the numerical result of $2 + 5$, we do not add quotes.

If we want to print the exact message " $2 + 5$ ", then we add the quotes.

Variables and Data Types

Variables

Variables are like containers for storing values. Values in the variables can be changed.

Values

Consider that variables are like containers for storing information. In context of programming, this information is often referred to as value.

Data Type

In programming languages, every value or data has an associated type to it known as data type. Some commonly used data types

- String
- Integer
- Float
- Boolean

This data type determines how the value or data can be used in the program. For example, mathematical operations can be done on Integer and Float types of data.

String

A String is a stream of characters enclosed within quotes. Stream of Characters

- Capital Letters (A – Z)
- Small Letters (a – z)
- Digits (0 – 9)
- Special Characters (~ ! @ # \$ % ^ . ? ,)
- Space

Some Examples

```
"Hello World!"  
"some@example.com"  
"1234"
```

Integer

All whole numbers (positive, negative and zero) without any fractional part come under Integers. Examples

```
...-3, -2, -1, 0, 1, 2, 3,...
```

Float

Any number with a decimal point

```
24.3, 345.210, -321.86
```

Boolean

In a general sense, anything that can take one of two possible values is considered a Boolean. Examples include the data that can take values like

```
True or False  
Yes or No  
0 or 1  
On or Off , etc.
```

As per the Python Syntax,

```
True  
and  
False
```

are considered as Boolean values. Notice that both start with a capital letter.

Assigning Value to Variable

The following is the syntax for assigning an integer value

```
10
```

to a variable

```
age
```

Here the equals to

```
=
```

sign is called as **Assignment Operator** as it is used to assign values to variables.

Sequence of Instructions

Program A program is a sequence of instructions given to a computer.

Defining a Variable

A variable gets created when you assign a value to it for the first time.

Code

```
age = 10
```

Printing Value in a Variable

Code

```
age = 10  
print(age)
```

Output

```
.
```

```
10
```

Code

```
age = 10  
print("age")
```

Output

```
.
```

```
age
```

Variable name enclosed in quotes will print variable rather than the value in it. If you intend to print value, do not enclose the variable in quotes.

Order of Instructions

Python executes the code line-by-line.

Code

```
print(age)  
age = 10
```

Output

.

```
NameError: name 'age' is not defined
```

Variable

```
age
```

is not created by the time we tried to print.

Spacing in Python

Having spaces at the beginning of line causes errors.

Code

```
a = 10 * 5  
b = 5 * 0.5  
b = a + b
```

Output

.

```
File "main.py", line 3
```

```
b = a + b
```

```
^
```

```
IndentationError: unexpected indent
```

Variable Assignment

Values in the variables can be changed.

Code

```
a = 1  
print(a)  
a = 2  
print(a)
```

Output

```
1
```

```
2 Examples of Variable Assignment
```

Code

```
a = 2
print(a)
a = a + 1
print(a)
```

Output

.

2

3

Code

```
a = 1
b = 2
a = b + 1
print(a)
print(b)
```

Output

.

3

2

Expression

An expression is a valid combination of values, variables and operators.

Examples $a * b$ $a + 2$ $5 * 2 + 3 * 4$

BODMAS The standard order of evaluating an expression - *Brackets (B) - Orders (O) - Division (D) - Multiplication (M) - Addition (A) - Subtraction (S)*

Step by Step Explanation

.

$(5 * 2) + (3 * 4)$

$(10) + (12)$

22

Code

```
print(10 / 2 + 3)
print(10 / (2 + 3))
```

Output

8.0

2.0

Inputs and Outputs Basics

Take Input From User

```
input()
```

allows flexibility to take the input from the user. Reads a line of input as a string.

Code

```
username = input()  
print(username)
```

Input

.

Ajay

Output

.

Ajay

Working with Strings

String Concatenation

Joining strings together is called string concatenation.

Code

```
a = "Hello" + " " + "World"  
print(a)
```

Output

.

Hello World

Concatenation Errors

String Concatenation is possible only with strings.

Code

```
a = "*" + 10  
print(a)
```

Output

.

File "main.py", line 1 a = "*" + 10

^

TypeError:

can only concatenate str (not "int") to str

String Repetition

*

operator is used for repeating strings any number of times as required.

Code

```
a = "*" * 10  
print(a)
```

Output

.

```
*****
```

Code

```
s = "Python"  
s = (" " * 3) + s + (" " * 3)  
print(s)
```

Output

.

```
* * * Python * * *
```

Length of String

len()

returns the number of characters in a given string.

Code

```
username = input()  
length = len(username)  
print(length)
```

Input

.

Ravi

Output

.

```
4
```

String Indexing

We can access an individual character in a string using their positions (which start from 0) .
These positions are also called as index.

Code

```
username = "Ravi"  
first_letter = username[0]  
print(first_letter)
```

Output

.

R

IndexError

Attempting to use an index that is too large will result in an error:

Code

```
username = "Ravi"  
print(username[4])
```

Output

IndexError: string index out

Type Conversion

String Slicing

Obtaining a part of a string is called string slicing.

Code

```
variable_name[start_index:end_index]
```

Start from the

start_index

and stops at

end_index

end_index

is not included in the slice.

Code

```
message = "Hi Ravi"  
part = message[3:7]  
print(part)
```

Output

.

```
Ravi
```

Slicing to End

If end index is not specified, slicing stops at the end of the string.

Code

```
message = "Hi Ravi"  
part = message[3:]  
print(part)
```

Output

```
.
```

```
Ravi
```

Slicing from Start

If start index is not specified, slicing starts from the index 0.

Code

```
message = "Hi Ravi"  
part = message[:2]  
print(part)
```

Output

```
.
```

```
Hi
```

Checking Data Type

Check the datatype of the variable or value using

```
type()
```

Printing Data Type

Code

```
print(type(10)) print(type(4.2))  
print(type("Hi"))
```

Output

```
.
```

```
<class 'int'>
```

```
<class 'float'>
```

```
<class 'str'>
```

Type Conversion

Converting the value of one data type to another data type is called *Type Conversion* or *Type Casting*. We can convert

- String to Integer
- Integer to Float
- Float to String and so on.

String to Integer

```
int()
```

converts valid data of any type to integer

Code

```
a = "5"  
a = int(a)  
print(type(a)) print(a)
```

Output

```
.
```

```
<class 'int'>
```

```
5
```

Invalid Integer Conversion

Code

```
a = "Five" a  
= int(a)  
print(type(a))
```

Output

```
.
```

```
ValueError:
```

```
invalid literal for int() with base 10: 'Five'
```

Code

```
a = "5.0" a =  
int(a)  
print(type(a))
```

Output

```
.
```

```
invalid literal for int() with base 10: '5.0'
```

Adding Two Numbers

Code

```
a = input()
a = int(a)
b = input()
b = int(b)
result = a + b
print(result)
```

Input

.

2

3

Output

.

5

Integer to String

```
str()
```

converts data of any type to a string.

Code

```
a = input() a
= int(a) b =
input() b =
int(b) result
= a + b
print("Sum: " + str(result))
```

Input

.

2

3

Output

.

Sum: 5

Summary

```
int() -> Converts to integer data type
float() -> Converts to float data type
str() -> Converts to string data type
bool() -> Converts to boolean data type
```

Relational Operators

Relational Operators are used to compare values.

Gives

```
True
```

or

```
False
```

as the result of a comparison. These are different relational operators

Operator	Name
>	Is greater than
<	Is less than
==	Is equal to
<=	Is less than or equal to
>=	Is greater than or equal to
!=	Is not equal to

Code

```
print(5 < 10)
print(2 > 1)
```

Output

```
.
```

```
True
```

```
True
```

Possible Mistakes

Mistake - 1

Code

```
print(3 = 3)
```

Output

.

```
SyntaxError: expression cannot contain assignment, perhaps you meant "=="?
```

Mistake - 2

Code print(2

<= 3)

Output

.

```
SyntaxError: invalid syntax
```

Space between relational operators

```
==
```

,

```
>=
```

,

```
<=
```

,

```
!=
```

is not valid in Python.

Comparing Numbers

Code

```
print(2 <= 3)
print(2.53 >= 2.55)
```

Output

.

```
True
```

```
False
```

Comparing Integers and Floats

Code

```
print(12 == 12.0)
print(12 == 12.1)
```

Output

.

```
True
```

```
False
```

Comparing Strings

Code

```
print("ABC" == "ABC")  
print("CBA" != "ABC")
```

Output

```
.
```

```
True
```

```
True
```

Case Sensitive

Code `print("ABC" ==
"abc")`

Output

```
.
```

```
False
```

Python is case sensitive. It means

```
X
```

(Capital letter) and

```
x
```

(small letter) are not the same in Python.

Strings and Equality Operator

Code

```
print(True == "True")  
print(123 == "123")  
print(1.1 == "1.1")
```

Output

```
.
```

```
False
```

```
False
```

False

Logical Operators

The logical operators are used to perform logical operations on Boolean values.

Gives

True

or

False

as the result.

Following are the logical operators

and
or
not

Logical AND Operator

Gives

True

if both the booleans are true else, it gives

False

Code

```
print(True and True)
```

Output

.

True

Examples of Logical AND

Code

```
print((2 < 3) and (1 < 2))
```

Step by Step Explanation

.

(2 < 3) and (1 < 2)

True and (1 < 2)

True and True

Output

.

True

Logical OR Operator

Gives

True

if any one of the booleans is true else, it gives

False

Code

```
print(False or False)
```

Output

.

False

Examples of Logical OR

Code

```
print((2 < 3) or (2 < 1))
```

Step by Step Explanation

.

(2 < 3) or (2 < 1)

True or (2 < 1)

True or False

Output

.

True

Logical NOT Operator

Gives the opposite value of the given boolean.

Code

```
print(not(False))
```

Output

.

True

Examples of Logical NOT

Code

```
print(not(2 < 3))
```

Step by Step Explanation

```
not(2 < 3) not(True)
```

False

Output

False

Summary

1. Logical *AND* Operator gives

True

2. if all the booleans are true.
3. Logical *OR* Operator gives

True

4. if any of the booleans are true.
5. Logical *NOT* Operator gives the opposite value

Conditional Statements

Block of Code

A sequence of instructions are called block of code.
Python executes code in a sequence.

Condition

An expression that results in either

True

or

False

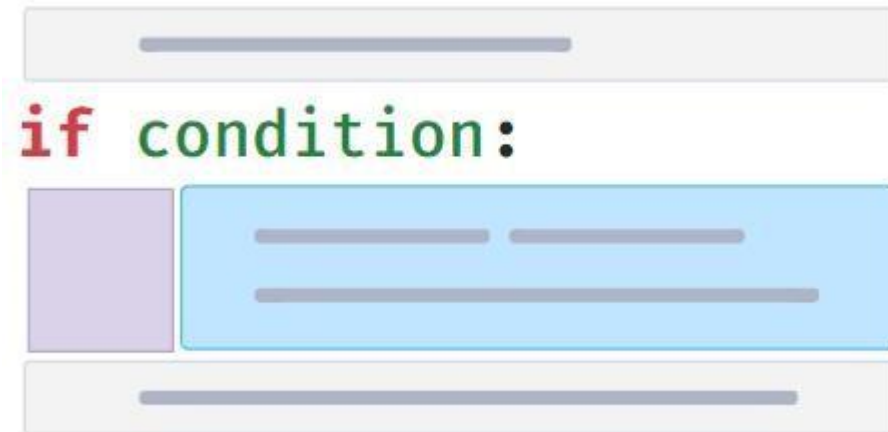
Examples

- i. $2 < 3$
- ii. $a == b$
- iii. True

Conditional Statement

Conditional Statement allows you to execute a block of code only when a specific condition is

True



Conditional Block

Block of code which executes only if a condition is

True

is called **Conditional Block**.



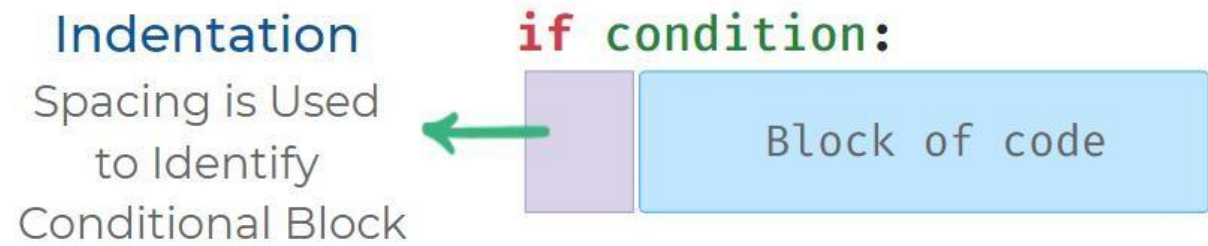
Conditional Block

Executes only if condition is true

Indentation

- Space(s) in front of the conditional block is called *indentation*.
- Indentation(spacing) is used to identify Conditional Block.
- Standard practice is to use *four spaces* for indentation.

Possible Mistakes



Possible Mistakes

Each statement inside a conditional block should have the same indentation (spacing).

Wrong Code

```
if True: print("If Block")
    print("Inside If")
```

Output

.

IndentationError: unexpected indent

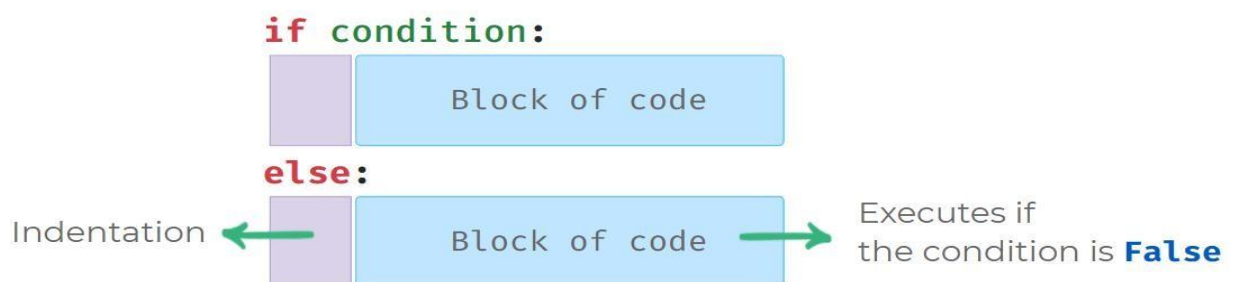
Correct Code

```
if True:  
    print("If Block")  
    print("Inside If")
```

If - Else Syntax

When If - Else conditional statement is used, Else block of code executes if the condition is

False



Using If-Else

Code

```
a = int(input())  
if a > 0:  
    print("Positive")  
else:  
    print("Not Positive")  
print("End")
```

Input

2

Output

Positive

End

Possible Mistakes in If-Else

Else can only be used along with if condition. It is written below if conditional block

Code

```
if False: print("If  
Block")  
print("After If") else:  
    print("Else Block")  
print("After Else")
```

Output

.

SyntaxError: invalid syntax

SingleLineWarning

Warning

Note: No code is allowed in between if conditional block and else statement.

More Arithmetic Operators

Modulus

To find the remainder, we use Modulus operator %

```
a % b
```

Code

```
print(6 % 3)
```

Output

.

0

Exponent

To calculate a power b, we use Exponent Operator **

```
a ** b
```

Code

```
print(2 ** 3)
```

Output

8

You can use the exponent operator to calculate the square root of a number bu keeping the exponent as 0.5

Code

```
print(16 ** 0.5)
```

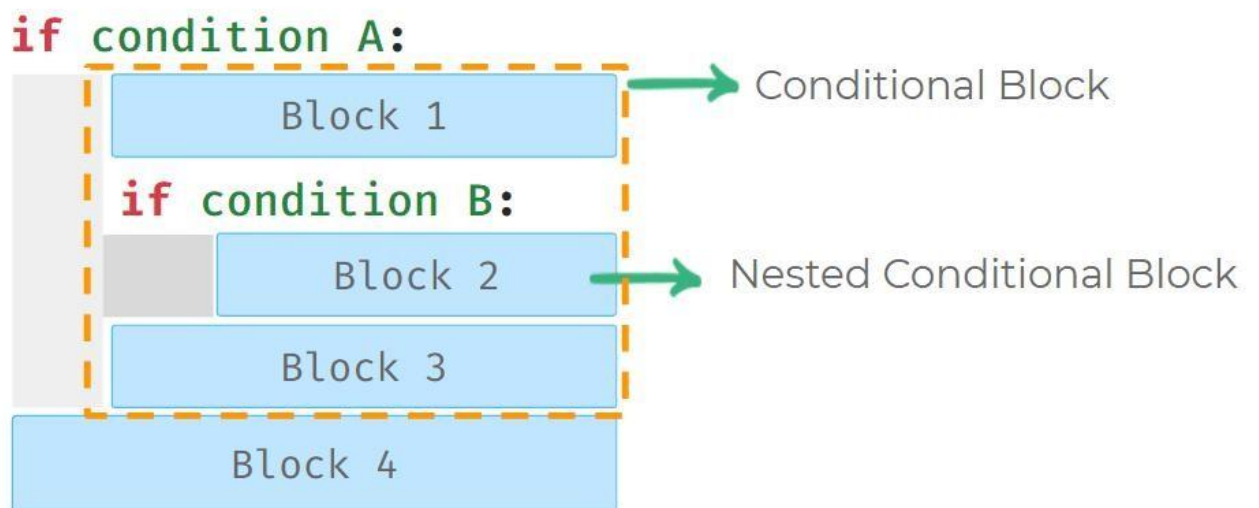
Output

4.0

Nested Conditional Statements

Nested Conditions

The conditional block inside another if/else conditional block is called as *nested conditional block*. In the below example, *Block 2* is nested conditional block and *condition B* is called nested conditional statement.



Code

```
matches_won = int(input())
```

```
goals = int(input())
```

```
if matches_won > 8:
```

```
    if goals > 20:
```

```
        print("Hurray")
```

```
        print("Winner")
```

Input

10
22

Output

Hurray
Winner

Input

10
18

Output

Winner

Nested Condition in Else Block

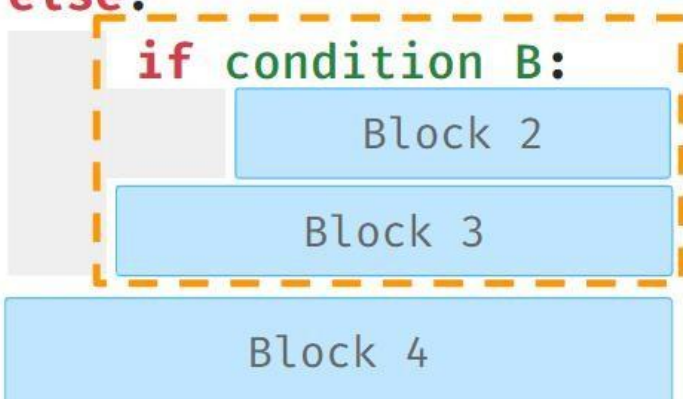
We can also write nested conditions in Else Statement.

In the below example *Block 2* is a nested conditional block.

if condition A:



else:



Code

```
a = 2
b = 3
c = 1

is_a_greatest = (a > b) and (a > c)
if is_a_greatest:
    print(a)
else:
    is_b_greatest = (b > c)
    if is_b_greatest:
        print(b)
```

Output

3

Elif Statement

Use the elif statement to have multiple conditional statements between if and else.

The elif statement is optional.

if condition A:

Block 1

elif condition B:

Block 2

else:

Block 3

Multiple Elif Statements

We can add any number of

elif
statements after
if
conditional block.

if condition A:

Block 1

elif condition B:

Block 2

elif condition C:

Block 3

else:

Block 4

Execution of Elif Statement

Python will execute the elif block whose expression evaluates to true.

If elif multiple conditions are true, then only the first elif block which is True will be executed.

if condition A:

Block 1

elif condition B:

Block 2

elif condition C:

Block 3

else:

Block 4

Optional Else Statement

Else statement is not compulsory after

if - elif statements.

if condition A:

Block 1

elif condition B:

Block 2

elif condition C:

Block 3

Code number = 5 is_divisible_by_10 =

(number % 10 == 0) is_divisible_by_5 =

(number % 5 == 0) if is_divisible_by_10:

print("Divisible by 10")

elif is_divisible_by_5:

print("Divisible by 5")

else:

print("Not Divisible by 10 or 5")

Output

Divisible by 5

Possible Mistake

Cannot write an elif statement after

Else statement.

if condition A:

Block 1

elif condition B:

Block 2

else:

Block 3

elif condition C:

Block 4



Loops

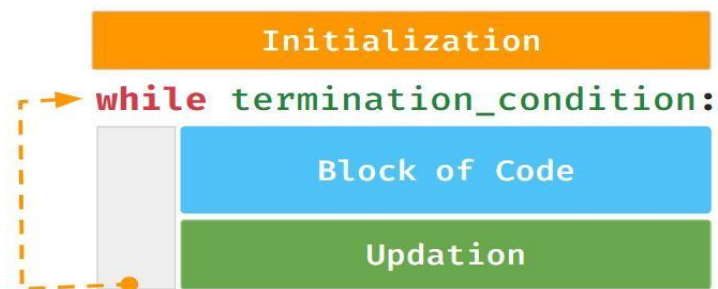
So far we have seen that Python executes code in a sequence and each block of code is executed once.

Loops allow us to execute a block of code several times.

While Loop

Allows us to execute a block of code several times as long as the condition is

True



While Loop Example

The following code snippet prints the next three consecutive numbers after a given number.

Code

```
a = int(input())
```

```
counter = 0 while
```

```
counter < 3: a = a
```

```
+ 1
```

```
    print(a)
```

```
    counter = counter + 1
```

Input

4

Output

5

6

7

Possible Mistakes

1. Missing Initialization

Code

```
a = int(input())
```

```
while counter < 3:
```

```
    a = a + 1
```

```
    print(a)
```

```
    counter = counter + 1
```

```
print("End") Input
```

5

Output

NameError: name 'counter' is not defined

2. Incorrect Termination Condition

Code

```
a = int(input()) counter =  
0 condition = (counter <  
3) while condition: a = a  
+ 1  
    print(a)  
    counter = counter + 1
```

Input

10

Output

Infinite Loop

While block will keep repeating as the value in condition variable is

True

3. Not Updating Counter Variable

Code

```
a = int(input())  
counter = 0 while  
counter < 3: a = a  
+ 1  
    print(a)  
print("End")
```

Input

10

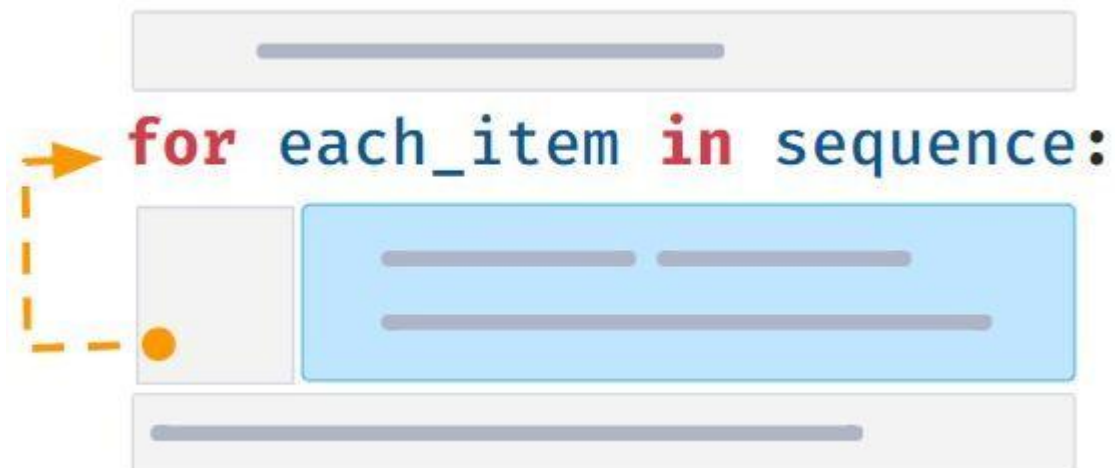
Output

Infinite Loop

As the value of counter is not updating.

For Loop

statement iterates over each item of a sequence.



Examples of sequences:

Sequence of Characters (string)

Sequence of numbers, etc.

Code

For Syntax word = "Python" for each_char in word:

```
    print(each_char)
```

Output

```
~
P
y
t
h
o
n
```

Range

Generates a sequence of integers starting from 0. Syntax:

`range(n)` Stops before n (n is not included).

Code

for number in range(3):

```
    print(number)
```

Output

```
~
0
1
2
```

Range with Start and End

Generates a sequence of numbers starting from

`start`

Syntax:

`range(start, end)`

Stops before End (end is not included). Code for number in range(5, 8):

```
print(number)
```

Output

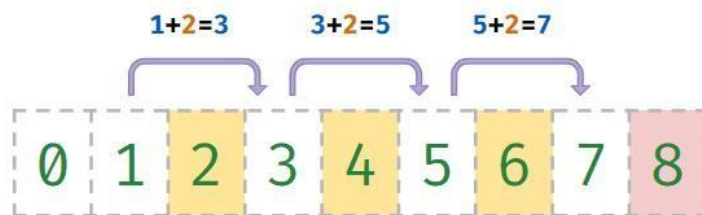
```
5
6
7
```

Extended Slicing and String Methods

Extended Slicing

Syntax:

```
variable[start_index:end_index:step]
```



```
[1:8:2]
```

Step determines the increment between each index for slicing.

Code

```
a = "Waterfall"
part = a[1:6:3]
print(part)
```

Output

```
ar
```

Methods

Python has a set of built-in reusable utilities. They simplify the most commonly performed operations are: String Methods

```
isdigit()
```

```
strip()
```

```
lower()
```

```
upper()
```

```
startswith()
```

```
endswith()
```

```
replace()
```

```
and more...
```

```
isdigit
```

Syntax:

```
str_var.isdigit()
```

```
False
```

Gives `True` if all the characters are digits. Otherwise,

```
Code
```

```
is_digit = "4748".isdigit()
```

```
print(is_digit)
```

```
Output
```

```
True
```

```
Strip
```

Syntax:

```
str_var.strip()
```

Removes all the leading and trailing spaces from a string.

```
Code
```

```
mobile = " 9876543210 "
```

```
mobile = mobile.strip()
```

```
print(mobile)
```

```
Output
```

```
9876543210
```

```
Strip - Specific characters
```

Syntax:

```
str_var.strip(chars)
```

We can also specify characters that need to be removed.

```
Code
```

```
name = "Ravi." name
```

```
= name.strip(".")
```

```
print(name)
```

```
Output
```

```
Ravi
```

```
Strip - Multiple Characters
```

Removes all spaces, comma(,) and full stop(.) that lead or trail the string. Code:

```
name = ", . , , ravi , . . "
```

```
name = name.strip(",.")
```

```
print(name)
```

```
Output
```

```
.
```

```
ravi
```

```
Replace
```

Syntax: `str_var.replace(old,new)`

Gives a new string after replacing all the occurrences of the old substring with the new substring.

Code

```
sentence = "teh cat and teh dog"
sentence = sentence.replace("teh","the")
print(sentence)
```

Output

the cat and the dog

Startswith

Syntax:

`str_var.startswith(value)`

Gives `True` if the string starts with the specified value. Otherwise,

`False`

Code

```
url = "https://onthegomodel.com"
is_secure_url = url.startswith("https://")
print(is_secure_url)
```

Output

True

Endswith

Syntax:

`str_var.endswith(value)`

Gives `True` if the string ends with the specified value. Otherwise,

`False`

Code

```
gmail_id = "rahul123@gmail.com" is_gmail =
gmail_id.endswith("@gmail.com")
print(is_gmail)
```

Output

True

Upper

Syntax:

`str_var.upper()`

Gives a new string by converting each character of the given string to uppercase.

Code

```
name = "ravi"
print(name.upper())
```

Output

RAVI

Lower

Syntax:

`str_var.lower()`

Gives a new string by converting each character of the given string to lowercase.

Code

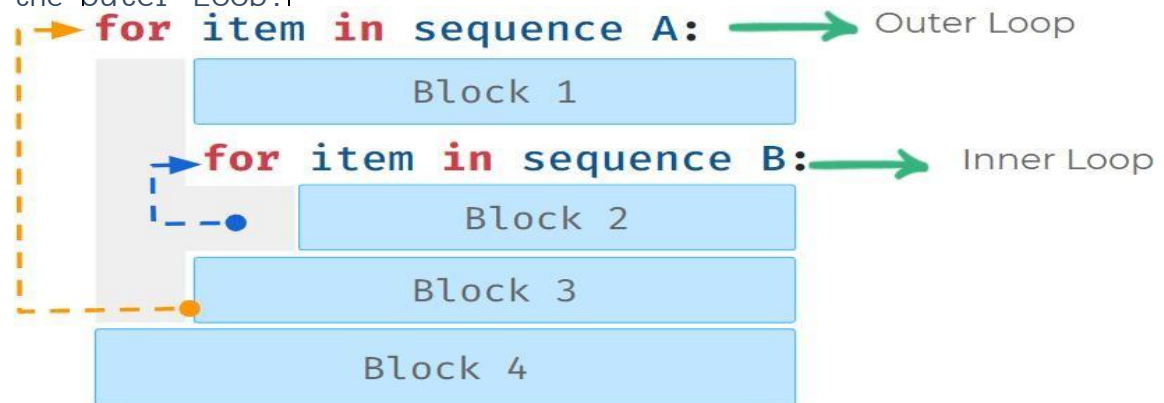
```
name = "RAVI"  
print(name.lower())
```

Output

ravi

Nested Loops

The **Inner Loop** will be executed one time for each iteration of the **Outer Loop**.



Code

An inner loop within the repeating block of an outer loop

is called Nested Loop. `for i in range(2): print("Outer: " +`

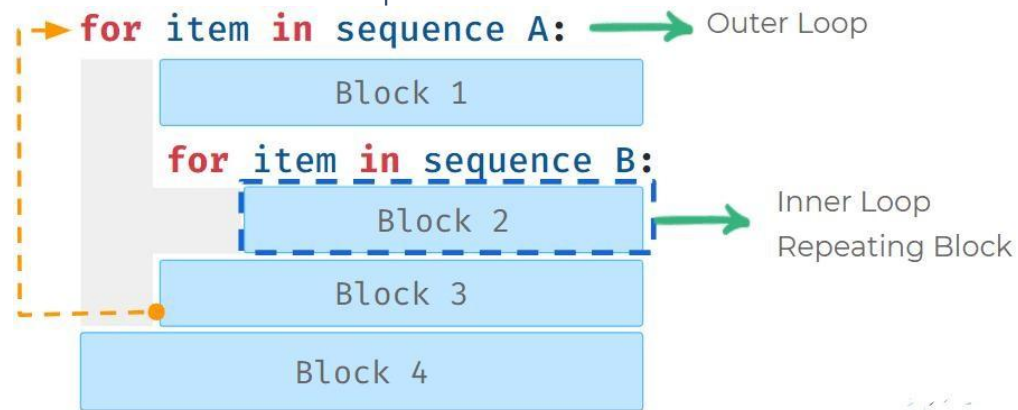
`str(i)) for j in range(2): print(" Inner: " + str(j))`

Output.

```
Outer: 0  
Inner: 0  
Inner: 1  
Outer: 1  
Inner: 0  
Inner: 1
```

Nested Repeating Block

The one highlighted in the blue dotted line is the **repeating** block of the inner loop.



Code

```
for i in range(2):
    print("Outer: " + str(i))
    for j in range(2):
        print(" Inner: " + str(j))
print("END")
```

In the above example, the below line is the repeating block of the nested loop.

Code

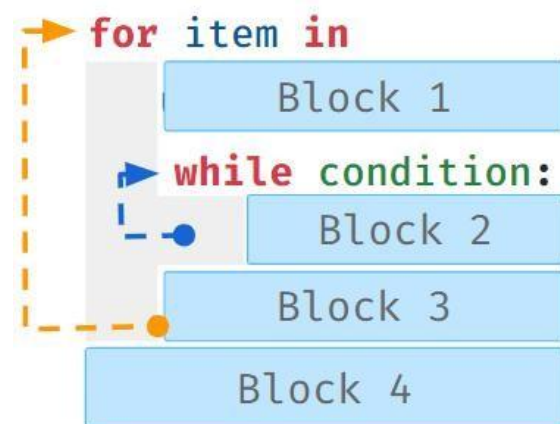
```
print(" Inner: " + str(j))
```

Output

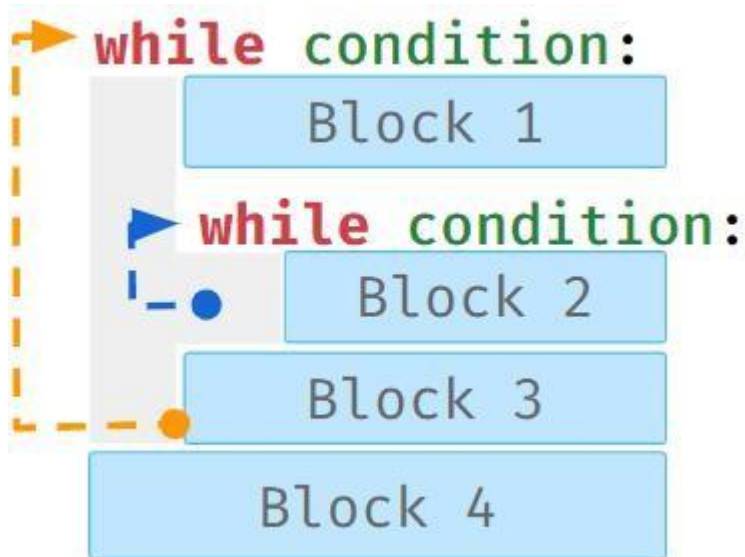
```
Outer: 0
Inner: 0
Inner: 1
Outer: 1
Inner: 0
Inner: 1
END
```

Examples - Nested Loops

Example - 1: While loop inside a For loop



Example - 2: While loop inside a while loop



Loop Control Statements

Control statements alter the sequential execution of a program.

Examples

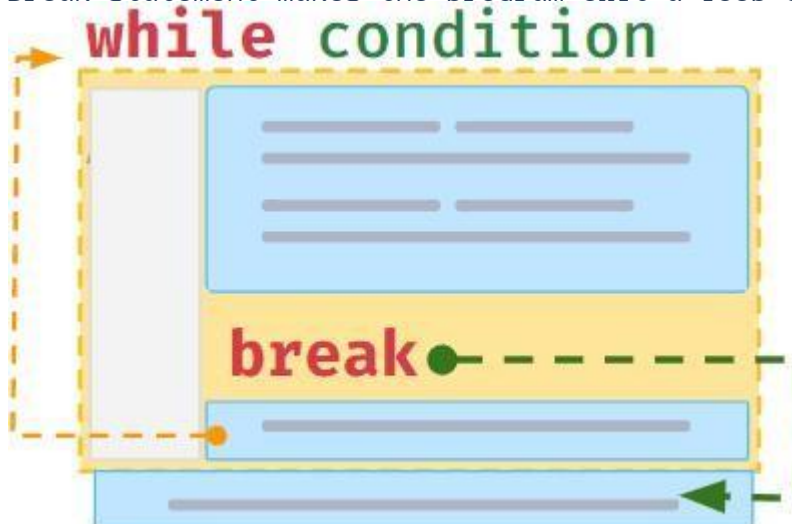
if-elif-else

while, for

break, continue

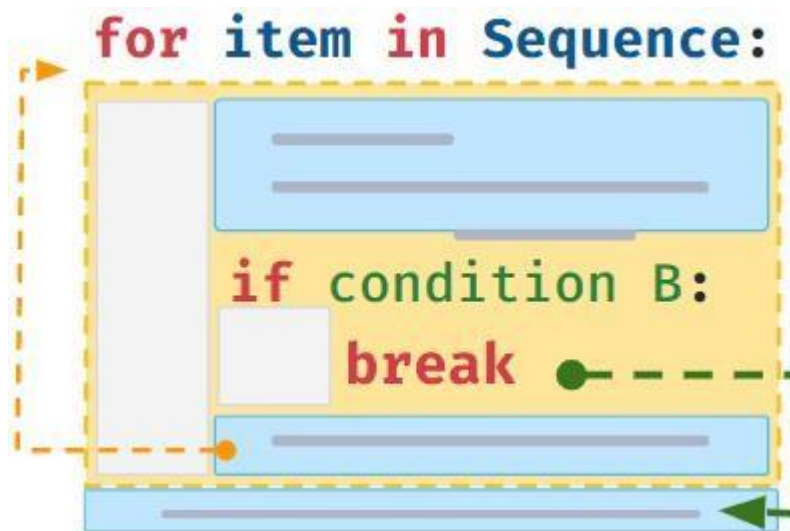
Break

Break statement makes the program exit a loop early.



Using Break

Generally, break is used to exit a loop when a condition is satisfied.



In the below example, when the variable `i` value equals to `3` the `break` statement gets executed and stops the execution of the loop further.

Code

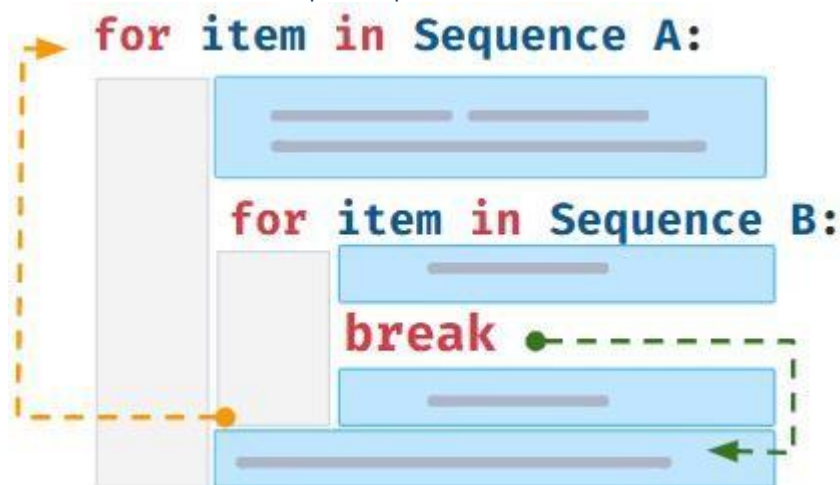
```
for i in range(5):
    if i == 3:
        break
    print(i)
print("END")
```

Output

```
0
1
2
END
```

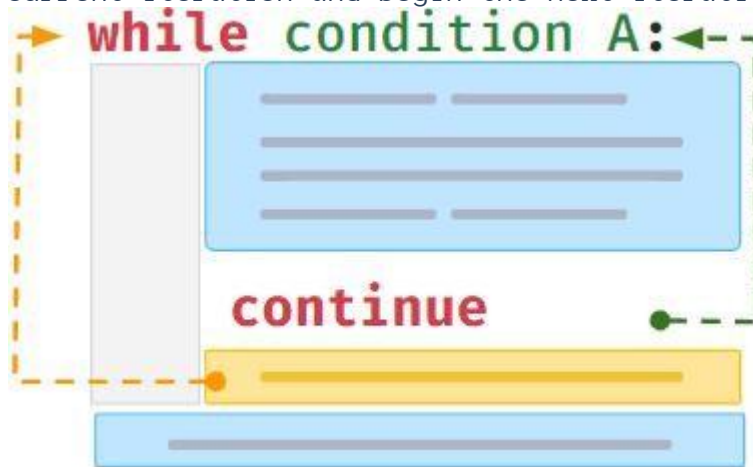
Break in Nested Loop

Break in inner loop stops the execution of the inner loop.



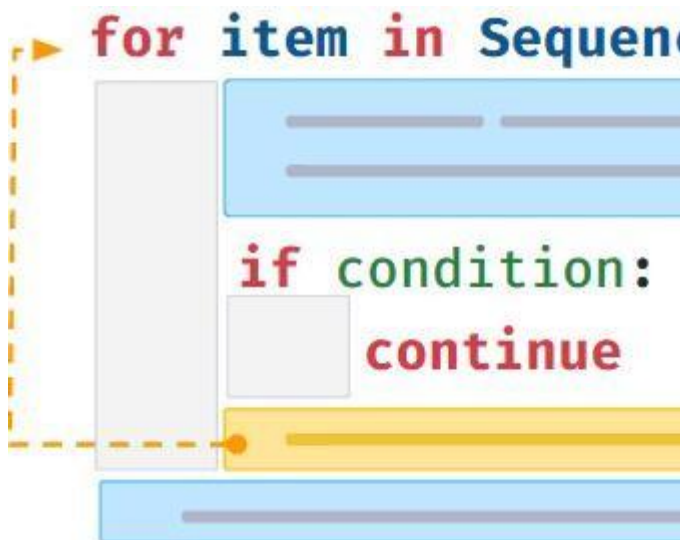
Continue

Continue makes the program skip the remaining statements in the current iteration and begin the next iteration.



Using Continue

Generally, continue is used to skip the remaining statements in the current iteration when a condition is satisfied.



3 In the below example, when the variable value equals to the next statements in the loop body are skipped.

Code

```
for i in
range(5):
    if i == 3:
        continue print(i)
print("END")
```

Output

```
.
0
1
2
4
END
```

Pass

Pass statement is used as a syntactic placeholder. When it is executed, nothing happens.
Generally used when we have to test the code before writing the complete code.

if condition A:

Block 1

elif condition B:

pass

else:

Block 3

Empty Loops

We can use pass statements to test code written so far, before writing loop logic.

while condition A:

pass

for item in Sequence:

pass

Comparing Strings

Computer internally stores characters as numbers.
Every character has a unique Unicode value.

Ord

`ord()`

To find the Unicode value of a character, we use the

`ord(character)`

gives unicode value of the character.

Code

```
unicode_value = ord("A")  
print(unicode_value)
```

Output

65

chr

To find the character with the given Unicode value, we use the

`chr()`

`chr(unicode)` gives character with the unicode value.

Code `char = chr(75)`

`print(char)`

Output

K

Unicode Ranges

`48 - 57` -> Number Digits (0 - 9)

`65 - 90` -> Capital Letters (A - Z)

`97 - 122` -> Small Letters (a - z)

`Rest` -> Special Characters, Other Languages

Printing Characters

The below code will print the characters

`A` from to `Z`

Code `for unicode_value in
range(65,91):
print(chr(unicode_value))`

Output

.

A

B

C

D

E

F

G

H

I

J

K

Comparing Strings

In Python, strings are compared considering unicode.

Code

`print("A" < "B")`

Output

True

As unicode value `A` is 65 and `B` is 66, which internally compares

65 <. So the output should be True
66

Character by Character Comparison

In Python, String Comparison is done character by character.

Code

```
print("BAD" >= "BAT")
```

Output

False

Code

```
print("98" < "984")
```

Output

True

Best Practices

Naming Variables Rule #1

Use only the below characters

Capital Letters (A – Z)

Small Letters (a – z)

Digits (0 – 9)

Underscore (_)

Examples:

age, total_bill

Naming Variables Rule #2

Below characters cannot be used

Blanks ()

Commas (,)

Special Characters

(~ ! @ # \$ % ^ . ? , etc.)

Naming Variables Rule #3

Variable name must begin with

Capital Letters (A – Z)

Small Letters (a – z)

Underscore (_)

Naming Variables Rule #4

Cannot use Keywords, which are reserved for special meaning

int

str

print

etc.,

Keywords

Words which are reserved for special meaning

Code help("keywords") Output:(list)

Case Styles

Camel case: totalBill

Pascal case: TotalBill

Snake case:
total_bill

Snake case is preferred for naming the variables in Python

Round

Rounding Numbers

```
round(number, digits(optional))
```

Rounds the float value to the given number of decimal digits.

`digit` -> define the number of decimal digits to be considered

`5` for rounding. when not specified default is `0` Code

```
a = round(3.14, 1)
```

```
print(a) a =
```

```
round(3.14)
```

```
print(a)
```

Output

```
3.1
```

```
3
```

Floating Point Approximation

Float values are stored approximately.

Code

```
print(0.1 + 0.2)
```

Output

```
0.30000000000000004
```

Floating Point Errors

Sometimes, floating point approximation gives unexpected results.

Code `print((0.1 + 0.2) == 0.3)`

Output

```
False
```

To avoid these unexpected results, we can use `round()`

Code

```
a = round((0.1 + 0.2), 1)
```

```
print(a) print(a == 0.3)
```

Output

```
0.3
True
```

Comments

Comment starts with a hash `#`.
It can be written in its own line next to a statement of code.

Code

```
n = 5
# Finding if Even
even = (n % 2 == 0)
print(even) # prints boolean value
```

Output

False

Floor Division Operator

To find integral part of quotient we use Floor Division Operator

```
//
```

```
a // b
```

Code print(3

```
// 2)
```

Compound Assignment Operators

Output

1

a += 1



Compound

Assignment Operator

Different compound assignment operators are

`+=`, `-=`, `*=`, `/=`, `%=`

`a += 1` is similar to `a = a + 1`

Code

```
a = 10
a -= 2
print(a)
```

Examples of Compound Assignment Operators

Code

Output

```
a = 10
```

```
a /= 2
```

```
print(a)
```

Output

5.0

Code

```
a = 10
```

```
a %= 2
```

```
print(a)
```

Output

0

Escape Characters

Single And Double Quotes

String is a sequence of characters enclosed within quotes.

Code sport = 'Cricket' print(type(sport)) sport = "Cricket"

```
print(type(sport))
```

Output

```
<class 'str'>
```

```
<class 'str'>
```

Code

```
is_same = ('Cricket' == "Cricket")
```

```
print(is_same)
```

Output

True

Escape Characters

Escape Characters are a sequence of characters in a string that are interpreted differently by the computer. We use escape

characters to insert characters that are illegal in a string.

Code

```
print("Hello\nWorld")
```

Output

```
Hello
```

```
World
```

We got a new line by adding

```
\n
```

escape character.

Examples - Escape Characters

Escape Characters start with a backslash in Python

```
\n
```

-> New Line

```
\t
```

-> Tab Space

```
\\
```

-> Backslash

```
\'
```

-> Single Quote

```
\"
```

-> Double Quote

Passing Strings With Quotes

The backslash \

character here tells Python not to consider the next character as the ending of the string. Code:

```
print('It\'s Python')
```

Output

```
It's Python
```

Code

```
print("It's Python")
```

Output

```
It's Python
```

Data Structures

Data Structures allow us to store and organize data efficiently. This will allow us to easily access and perform operations on the data.

In Python, there are four built-in data structures

List

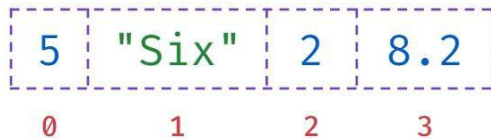
Tuple

Set

Dictionary

List

List is the most versatile python data structure. Holds an ordered sequence of items.



Creating a List

Created by enclosing elements within [square] brackets. Each item is separated by a comma. [Code](#)

```
a = 2
list_a = [5, "Six", a, 8.2]
print(type(list_a))
print(list_a)
```

[Output](#)

```
<class 'list'>
[5, 'Six', 2, 8.2]
```

Creating a List of Lists

[Code](#)

```
a = 2
list_a = [5, "Six", a, 8.2]
list_b = [1, list_a]
print(list_b) Output
```

```
[1, [5, 'Six', 2,
8.2]]
```

Length of List

[Code](#)

```
a = 2
list_a = [5, "Six", a, 8.2]
print(len(list_a))
```

[Output](#)

```
4
```

Accessing List Items

To access elements of a list, we use Indexing.

[Code](#)

```
a = 2
```

```
list_a = [5, "Six", a, 8.2]
print(list_a[1])
```

Output

Six

Iterating Over a List

Code

```
a = 2
list_a = [5, "Six", a, 8.2]
for item in list_a:
    print(item)
```

Output

5

Six

2

8.2

List

+ Concatenation

Similar to strings, operator concatenates lists.

```
list_a = [1, 2, 3]
list_b = ["a", "b", "c"]
list_c = list_a +
list_b
print(list_c)
```

Output

[1, 2, 3, 'a', 'b', 'c']

Adding Items to List

Code

```
list_a = []
print(list_a)
for i
in range(1,4):
    list_a += [i]
print(list_a)
```

Output

[]

[1, 2, 3]

Repetition

* Operator repeats lists.

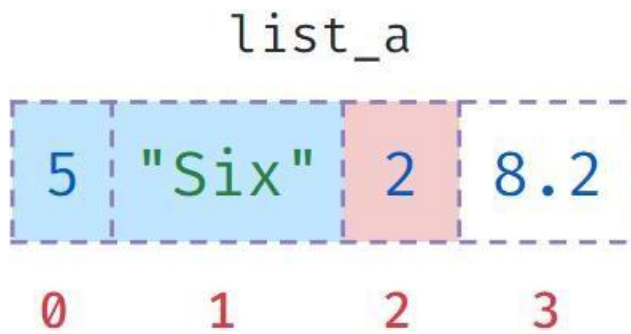
```
list_a = [1,
2]
list_b = list_a
* 3
print(list_b)
```

Output

[1, 2, 1, 2, 1, 2]

List Slicing

Obtaining a part of a list is called List Slicing.



Code

```
list_a = [5, "Six", 2, 8.2]
list_b = list_a[:2]
print(list_b)
```

Output

```
[5, 'Six']
```

Extended Slicing

Similar to string extended slicing, we can extract alternate items using step. Code

```
list_a = ["R", "B", "G", "O", "W"]
list_b = list_a[0:5:3]
print(list_b)
```

Output

```
['R', 'O']
```

list(sequence) Converting to List

`list()` takes a sequence and converts it into list.

```
color = "Red"
list_a = list(color)
print(list_a)
```

Output

```
['R', 'e', 'd']
```

Code

```
list_a = list(range(4))
print(list_a)
```

Output

```
[0, 1, 2, 3]
```

Lists are Mutable

Lists can be modified.

Items at any position can be updated.

```
list_a = [1, 2, 3, 5]
```

```
print(list_a)
```

```
list_a[3] = 4
```

```
print(list_a)
```

Output

```
[1, 2, 3, 5]
```

```
[1, 2, 3, 4]
```

Strings are Immutable

Strings are Immutable (Can't be modified).

Code

```
message = "see you  
soon" message[2] = "e"  
print(message)
```

Output

TypeError: 'str' object does not support it.

Working with Lists

Object

In general, anything that can be assigned to a variable in Python is referred to as an object.

Strings, Integers, Floats, Lists etc. are all objects.

Examples

"A"

1.25

[1, 2, 3]

Identity of an Object

Whenever an object is created in Python, it will be given a unique identifier (id).

This unique id can be different for each time you run the program. Every object that you use in a Python Program will be stored in Computer Memory.

The unique id will be related to the location where the object is stored in the Computer Memory.

"A"

Id - 140035229724336

1 2 3

Id - 139630925071104

Finding Id

id()

0

We can use `id()` to find the id of an object.

Code

```
print(id("Hello"))
```

Output

```
140589419285168
```

Id of Lists

```
list_a = [1, 2, 3]
```

```
list_b = [1, 2, 3]
```

```
print(id(list_a))
```

```
print(id(list_b))
```

Output

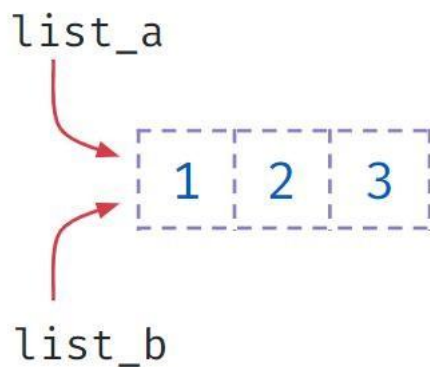
```
139637858236800
```

```
139637857505984
```

Modifying Lists

Modifying Lists - 1

When assigned an existing list both `list_a` and `list_b` the variables will be referring to the same object.



Code

```
list_a = [1, 2, 3]
```

```
list_b = list_a
```

```
print(id(list_a))
```

```
print(id(list_b))
```

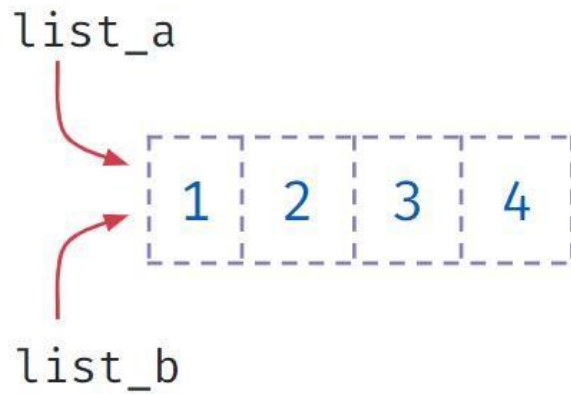
Output

```
140334087715264
```

```
140334087715264
```

Modifying Lists - 2

When assigned an existing list both the variables `list_a` and `list_b` will be referring to the same object.



Code:

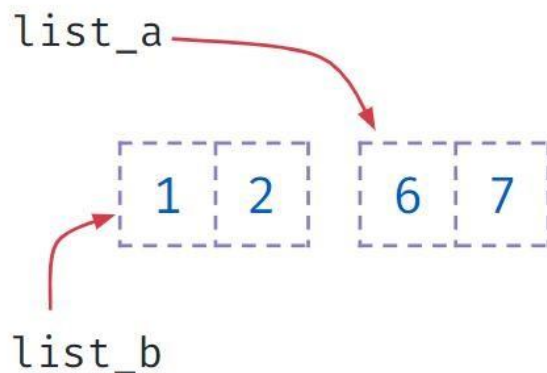
```
list_a = [1, 2, 3, 5] list_b =  
list_a list_b[3] = 4  
print("list a : " + str(list_a))  
print("list b : " + str(list_b))
```

Output

```
list a : [1, 2, 3, 4]  
list b : [1, 2, 3, 4]
```

Modifying Lists - 3

The assignment will update the reference to new object.



Code:

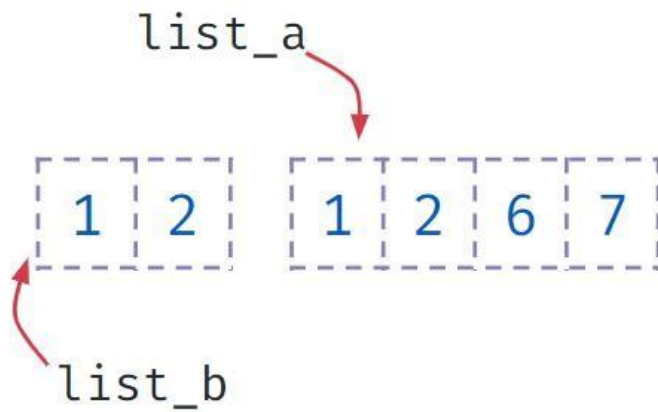
```
list_a = [1, 2] list_b = list_a  
list_a = [6, 7] print("list a :  
" + str(list_a)) print("list b :  
" + str(list_b))
```

Output

```
list a : [6, 7]  
list b : [1, 2]
```

Modifying Lists - 4

The assignment will update the reference to a new object.



Code:

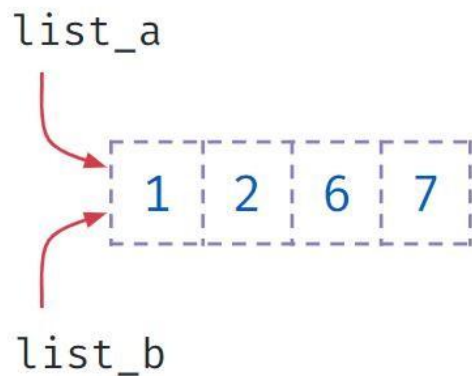
```
list_a = [1, 2] list_b = list_a
list_a = list_a + [6, 7]
print("list a : " + str(list_a))
print("list b : " + str(list_b))
```

Output

```
list a : [1, 2, 6,
7]
list b : [1,
2]
```

Modifying Lists - 5

Compound assignment will update the existing list instead of creating a new object.



Code:

```
list_a = [1, 2] list_b = list_a
list_a += [6, 7] print("list a :
" + str(list_a)) print("list b :
" + str(list_b))
```

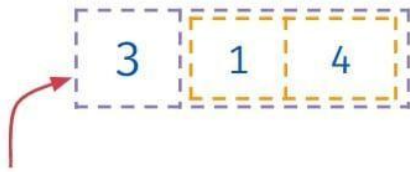
Output

```
list a : [1, 2, 6, 7]
list b : [1, 2, 6, 7]
```

Modifying Lists - 6

Updating mutable objects will also effect the values in the list, as the reference is changed.

list_a



list_b

Code:

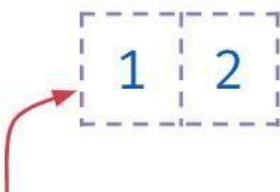
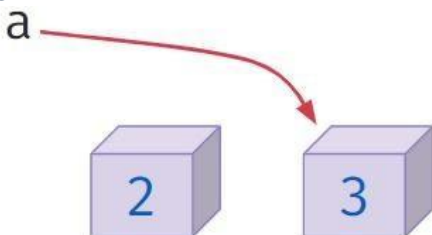
```
list_a = [1,2]
list_b = [3, list_a]
list_a[1] = 4
print(list_a)
print(list_b)
```

Output

```
[1,
4]
[3, [1,
4]]
```

Modifying Lists - 7

Updating immutable objects will not effect the values in the list, as the reference will be changed.



list_a

Code:

```
a = 2
list_a = [1,a]
print(list_a)
a = 3
print(list_a)
```

Output:

```
[1, 2]
[1, 2]
```

Lists and Strings

Splitting

`str_var.split(separator)`

Splits a string into a list at every specified separator. If no separator is specified, default separator is whitespace. [Code](#)

```
nums = "1 2 3 4"
num_list = nums.split()
print(num_list)
```

[Output](#)

```
['1', '2', '3', '4']
```

Multiple WhiteSpaces

Multiple whitespaces are considered as single when splitting.

[Code](#)

```
nums = "1 2 3 4 "
num_list = nums.split()
print(num_list)
```

[Output](#)

```
['1', '2',
'3', '4']
```

New `\n` and tab space `\t` lineare also whitespace.

[Code](#)

```
nums = "1\n2\t3
4"
num_list =
nums.split()
print(num_list)
```

[Output](#)

```
['1', '2', '3', '4']
```

Using Separator

Example- Breaks up a string at the specified separator. 1

Code

```
nums =  
"1,2,3,4" num_list =  
nums.split(',')  
print(num_list)
```

Output

```
['1', '2',  
'3', '4']
```

Example-

Code

2

```
nums = "1,2,,3,4,"  
num_list = nums.split(',')  
print(num_list)
```

Output

```
['1', '2', '', '3', '4', '']
```

Space as Separator

Code

```
nums = "1 2 3 4 "  
num_list = nums.split(" ")  
print(num_list)
```

Output

```
['1', '', '2', '3', '4', '']
```

String as Separator

Example-1

Code

```
string_a = "Python is a programming language"  
list_a = string_a.split('a')  
print(list_a)
```

Output

```
['Python is ', ' progr', 'mming l', 'ngu', 'ge']
```

Example-2

```
string_a = "step-by-step execution of code"  
list_a = string_a.split(' step')  
print(list_a)
```

Output

```
['', '-by-', ' execution of code']
```

Joining

```
str.join(sequence)
```

Takes all the items in a sequence of strings and joins them into one string.

Code

```
list_a = ['Python is ', ' progr', 'mming l', 'ngu', 'ge']  
string_a = "a".join(list_a) print(string_a)
```

Output

```
Python is a programming language
```

Joining Non String Values

Sequence should not contain any non-string values.

Code

```
list_a = list(range(4))  
string_a = ",".join(list_a)  
print(string_a)
```

Output

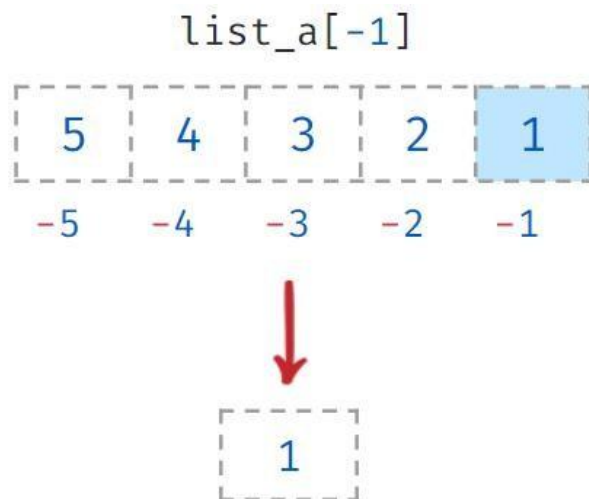
```
TypeError: sequence item 0: expected str instance, int found
```

Negative Indexing

Using a negative index returns the nth item from the end of list.

Last item in the list can be accessed with index

```
-1
```



Reversing a List

-1

for step will reverse the order of items in the list.

Code

```
list_a = [5, 4, 3, 2, 1]
list_b = list_a[::-1]
print(list_b)
```

Output

[1, 2, 3, 4, 5]

Accessing List Items

Example-1

Code

```
list_a = [5, 4, 3, 2, 1]
item = list_a[-1]
print(item)
```

Output

1

Example-2

Code

```
list_a = [5, 4, 3, 2, 1]
item = list_a[-4]
print(item)
```

Output

4

Slicing With Negative Index

You can also specify negative indices while slicing a List.

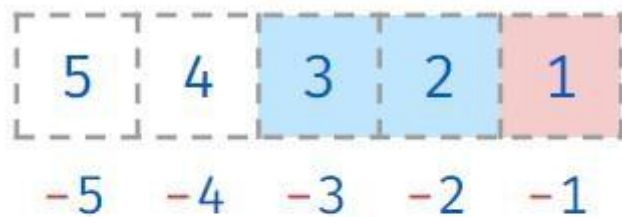
Code

```
list_a = [5, 4, 3, 2, 1]
list_b = list_a[-3:-1]
print(list_b)
```

Output

[3, 2]

`list_a[-3:-1]`



Out of Bounds Index

While slicing, Index can go

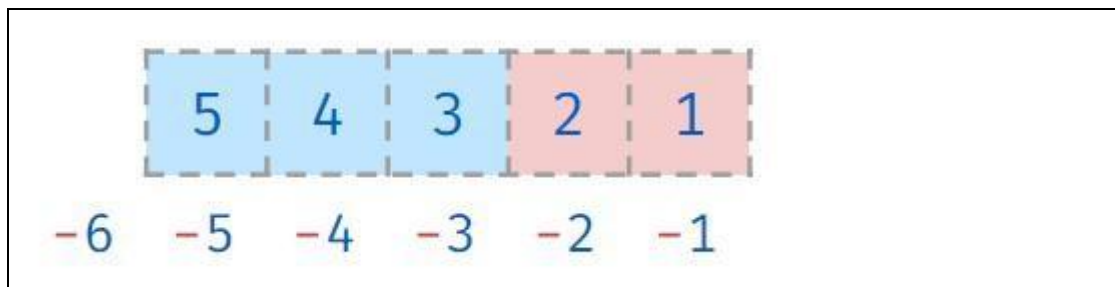
Code

out of bounds.

```
list_a = [5, 4, 3, 2, 1]
list_b = list_a[-6:-2]
print(list_b)
```

Output

[5, 4, 3]



Negative Step Size

`variable[start:end:negative_step]`

Negative Step determines the decrement between each index for slicing.

Start index should be greater than the end index in this case

	start	>
	end	

Negative Step Size Examples

Example-1

Code

```
list_a = [5, 4, 3, 2, 1]
list_b = list_a[4:2:-1]
print(list_b)
```

Output

[1, 2]

Example-2

Negative step requires the start to be greater than end.

Code

```
list_a = [5, 4, 3, 2, 1]
list_b = list_a[2:4:-1]
print(list_b)
```

Output

[]

Reversing a List

-1

for step will reverse the order of items in the list.

```
list_a = [5, 4, 3, 2, 1]
```

```
list_b = list_a[::-1]
```

```
print(list_b)
```

Output

```
[1, 2, 3, 4, 5]
```

Reversing a String

```
-1
```

for step will reverse the order of the characters.

Code

```
string_1 = "Program"
```

```
string_2 = string_1[::-1]
```

```
print(string_2)
```

Output

```
mgorP
```

Negative Step Size - Strings

Code

```
string_1 = "Program"
```

```
string_2 = string_1[6:0:-2]
```

```
print(string_2)
```

Output

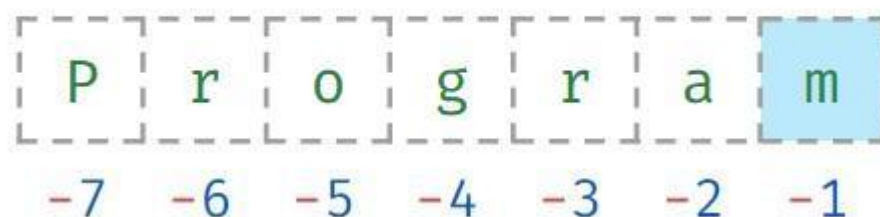
```
mro
```

Indexing & Slicing - Strings

Example - 1

Code

```
string_1[-1]
```



```
string_1 = "Program"
string_2 = string_1[-1]
print(string_2)
```

Output

m

Example-2

Code

```
string_1 = "Program"
string_2 = string_1[-4:-1]
print(string_2)
```

Output

gra

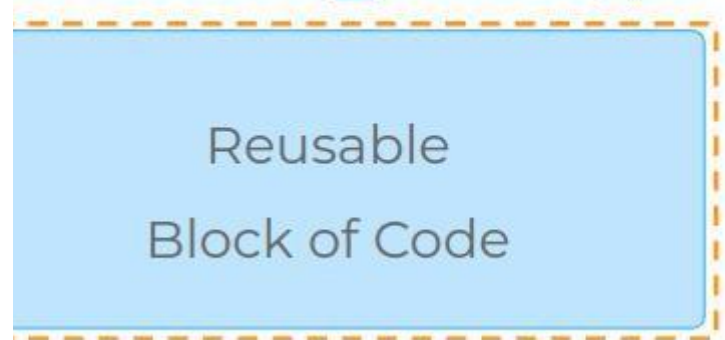
Functions

Block of reusable code to perform a specific action.

Reusing Code code without writing it every time we need.

Using an existing

unction_name():



Code

```
def greet():
    print("Hello")
```

```
name = input()
print(name)
```

Input

Teja

Output

Teja

Defining a Function

Function is uniquely identified by the
`function_name`

```
def function_name():
```

Reusable
Block of Code

Code

```
def greet():  
    print("Hello")
```

```
name = input()  
print(name)
```

Input

Teja

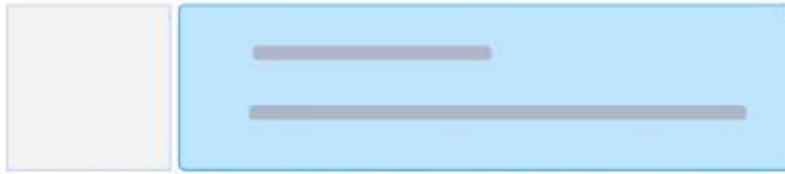
Output

Teja

Calling a Function

The functional block of code is executed only when the function is called.

```
def function_name():
```



```
function_name()
```

```
function_name()
```

Code

```
def greet():  
    print("Hello")
```

```
name = input()  
greet()  
print(name)
```

Input

Teja

Output

Hello

Teja

Defining & Calling a Function

A function should be defined before it is called.

Code

```
name = input() greet() print(name)
```

```
def greet():  
    print("Hello")
```

Input

Teja

Output

NameError: name 'greet' is not defined

Printing a Message

Consider the following scenario, we want to create a function, that prints a custom message, based on some variable that is defined outside the function. In the below code snippet, we want to access the value in the variable

name

at line 2 in place of the

?

.

Code

```
def greet(): msg =  
    "Hello " + ?  
    print(msg)
```

```
name = input()
```

```
greet()
```

Input

Teja

Desired Output

Hello Teja

We use the concept of Function Arguments for these types of scenarios.

Function With Arguments

We can pass values to a function using an Argument.

```
def function_name(args):
```



Reusable
Block of Code

Code

```
def greet(word): msg =  
    "Hello " + word  
    print(msg)
```

```
name = input() greet(word=name)
```

Input

Teja

Output

Hello Teja

Variables Inside a Function

A variable created inside a function can only be used in it.

Code

```
def greet(word):  
    msg = "Hello " + word
```

```
name = input()  
greet(word=name) print(msg)
```

Input

Teja

Output

```
NameError: name 'msg' is not defined
```

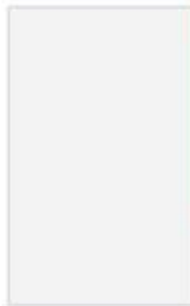
Returning a Value

To return a value from the function use

```
return
```

keyword.

```
def function_name(args):
```



Reusable
Block of Code

```
return value
```

Exits from the function when return statement is executed.

Code

```
def greet(word): msg =  
    "Hello " + word  
    return msg
```

```
name = input() greeting =  
greet(word=name)  
print(greeting)
```

Input

Teja

Output

Hello Teja

Code written after

```
return
```

statement will not be executed.

Code

```
def greet(word): msg =  
    "Hello "+word  
    return msg  
    print(msg)
```

```
name = input() greeting =  
greet(word=name)  
print(greeting)
```

Input

Output

Built-in Functions

We are already using functions which are pre-defined in Python. Built-in functions are readily available for reuse

Function Arguments

A function can have more than one argument.

```
def function_name(arg_1, arg_2):
```



Reusable
Block of Code

Keyword Arguments

Passing values by their names.

Code

```
def greet(arg_1, arg_2):  
    print(arg_1 + " " + arg_2)  
  
greeting = input() name = input()  
greet(arg_1=greeting, arg_2=name)
```

Input

Good Morning

Ram

Output

Good Morning Ram

Possible Mistakes - Keyword Arguments

Code

```
def greet(arg_1, arg_2):  
    print(arg_1 + " " + arg_2)  
  
greeting = input()  
name = input()  
greet(arg_2=name)
```

Input

Good Morning

Ram

Output

TypeError: greet() missing 1 required positional argument: 'arg_1'

Positional Arguments

Values can be passed without using argument names.

These values get assigned according to their position.

Order of the arguments matters here.

Code

```
def greet(arg_1, arg_2):  
    print(arg_1 + " " + arg_2)
```

```
greeting = input()  
name = input()  
greet(greeting, name)
```

Input

Good Morning

Ram

Output

Good Morning Ram

Possible Mistakes - Positional Arguments

Mistake - 1

Code

```
def greet(arg_1, arg_2):  
    print(arg_1 + " " + arg_2)
```

```
greeting = input()  
name = input()  
greet(greeting)
```

Input

Good Morning

Ram

Output

`TypeError: greet() missing 1 required positional argument: 'arg_2'`

Mistake - 2

Code

```
def greet(arg_1, arg_2):  
    print(arg_1 + " " + arg_2)
```

```
greeting = input()
name = input() greet()
```

Input

Good Morning

Ram

Output

TypeError: greet() missing 2 required positional arguments

Default Values

Example-1

Code

```
def greet(arg_1="Hi", arg_2="Ram"):
    print(arg_1 + " " + arg_2)
```

```
greeting = input()
name = input() greet()
```

Input

Hello

Teja

Output

Hi Ram

Example-2

Code

```
def greet(arg_1="Hi", arg_2="Ram"):
    print(arg_1 + " " + arg_2)
```

```
greeting = input()
name = input()
greet(greeting)
```

Input

Hello

Teja

Output

Hello Ram

Example-3

Code

```
def greet(arg_1="Hi", arg_2="Ram"):  
    print(arg_1 + " " + arg_2)
```

```
greeting = input()  
name = input()  
greet(name)
```

Input

Hello

Teja

Output

Teja Ram

Example-4

Code

```
def greet(arg_1="Hi", arg_2="Ram"):  
    print(arg_1 + " " + arg_2)
```

```
greeting = input()  
name = input()  
greet(arg_2=name)
```

Input

Hello

Teja

Output

Hi Teja

Example-5

Code

```
def greet(arg_1="Hi", arg_2):  
    print(arg_1 + " " + arg_2)
```

```
greeting = input()  
name = input()  
greet(arg_2=name)
```

Input

Hello

Teja

Output

SyntaxError:non-default argument follows default argument

Non-default arguments cannot follow default arguments.

Example-6

Code

```
def greet(arg_2, arg_1="Hi"):  
    print(arg_1 + " " + arg_2)
```

```
greeting = input()
```

```
name = input() greet(arg_2=name)
```

Input

Hello

Teja

Output

Hi Teja

Passing Immutable Objects

Code

```
def increment(a):  
    a += 1  
  
a = int(input())  
increment(a) print(a)
```

Input

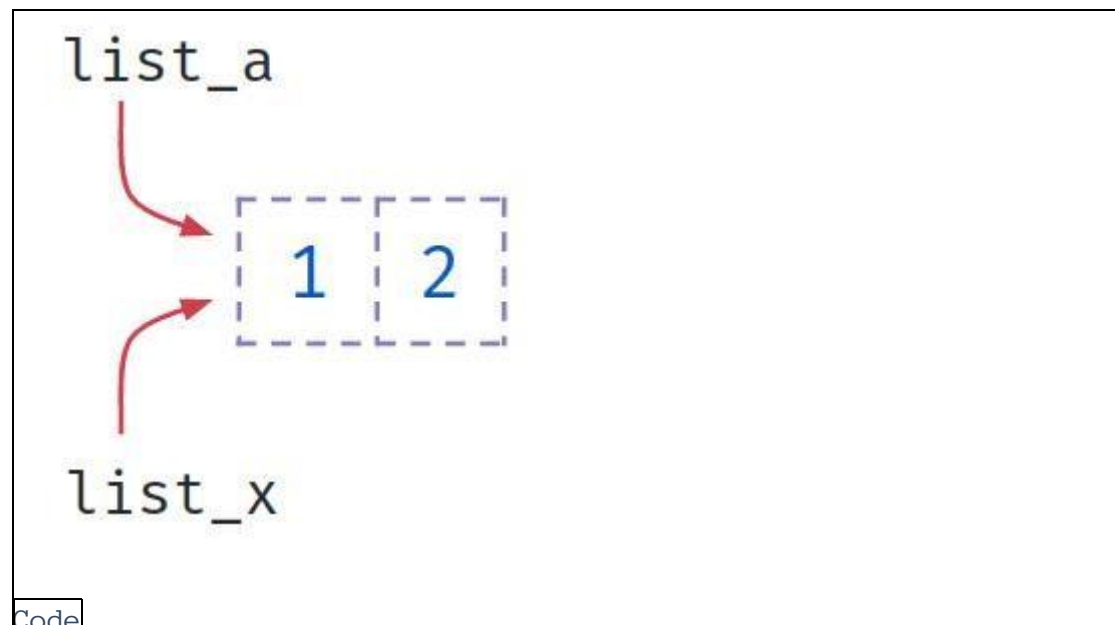
Output

Even though variable names are same, they are referring to two different objects. Changing the value of the variable inside the function will not affect the variable outside.

Passing Mutable Objects

The same object in the memory is referred by both

and



Code

```
def add_item(list_x):  
    list_x += [3]  
  
list_a = [1,2]  
add_item(list_a) print(list_a)
```

Output

[1, 2, 3]

Code

```
def add_item(list_x):  
    list_x = list_x + [3]
```

```
list_a = [1,2]  
add_item(list_a)  
print(list_a)
```

Output

[1, 2]

Default args are evaluated only once when the function is defined, not each time the function is called.

Code

```
def add_item(list_x=[]): list_x += [3] print(list_x)
```

```
add_item()  
add_item([1,2])  
add_item()
```

Output

[3]

[1, 2, 3]

[3, 3]

Built-in functions

Built-in functions are readily available for reuse.

We are already using functions which are pre-defined in Python

`print()`

`int()`

`str()`

len()

Finding Minimum

min()

returns the smallest item in a sequence or smallest of two or more arguments. min(sequence) min(arg1, arg2, arg3 ...)

Example-1

Code

```
smallest= min(3,5,4)
print(smallest)
```

Output

3

Example-2

Code

```
smallest = min([1,-2,4,2]) print(smallest)
```

Output

-2

Minimum of Strings

min(str_1, str_2)

Strings are compared character by character using unicode values.

P - 80(unicode)

J - 74(unicode)

Code

```
smallest = min("Python", "Java")
print(smallest)
```

Output

Java

Finding Maximum

max()

returns the largest item in a sequence or largest of two or more arguments. `max(sequence)` `max(arg1, arg2, arg3 ...)`

Example-1

Code

```
largest = max(3, 5, 4)
print(largest)
```

Output

5

Example-2

Code

```
largest = max([1, -2, 4, 2])
print(largest)
```

Output

4

Finding Sum

`sum(sequence)`

returns sum of items in a sequence.

Code

```
sum_of_numbers = sum([1, -2, 4, 2])
print(sum_of_numbers)
```

Output

5

Ordering List Items

`sorted(sequence)`

returns a new sequence with all the items in the given sequence ordered in increasing order.

Code

```
list_a = [3, 5, 2, 1, 4, 6]
list_x = sorted(list_a)
print(list_x)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

Ordering List Items - Reverse

```
sorted(sequence, reverse=True)
```

returns a new sequence with all the items in the given sequence ordered in decreasing order.

Code

```
list_a = [3, 5, 2, 1, 4, 6]  
list_x = sorted(list_a, reverse=True)  
print(list_x)
```

Output

```
[6, 5, 4, 3, 2, 1]
```

Function Call Stack & Recursion

Stack

Stack is a data structure that stores items in an Last-In/First-Out manner.



Calling a Function

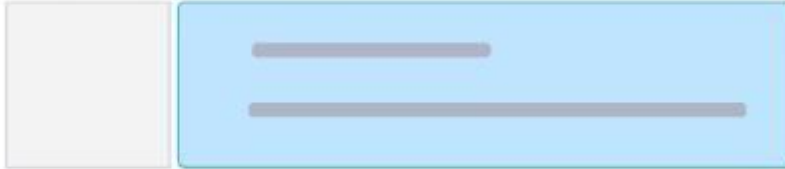
Calling

```
function_1()
```

inside

function_2()

def function_1():



def function_2():



Code

```
def get_largest_sqr(list_x):  
    len_list = len(list_x)  
    for i in range(len_list):  
        x = list_x[i] list_x[i]  
        = x * x largest =  
        max(list_x) return  
        largest
```

```
list_a = [1,-3,2] result =  
get_largest_sqr(list_a)
```

Output

9

In the above code calling functions are

len()

and

max()

inside

get_largest_sqr()

Sum of Squares of List Items

get_sum_of_sqrs([1,2,3])

Code

```
def get_sqrd_val(x):  
    return (x * x)  
  
def get_sum_of_sqrs(list_a):  
    sqrs_sum = 0  
    for i in list_a:  
        sqrs_sum += get_sqrd_val(i)  
    return sqrs_sum
```

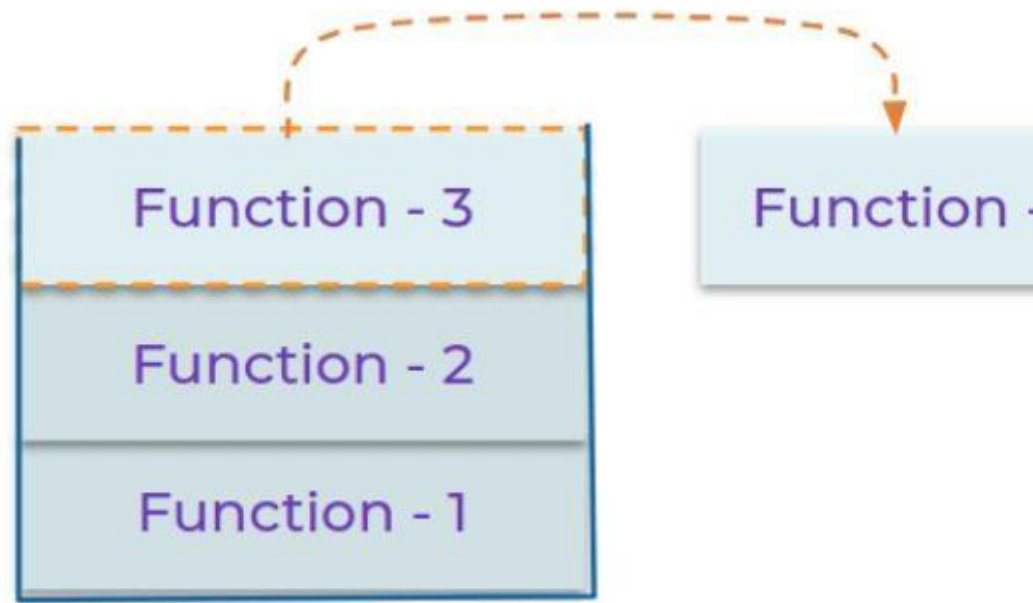
```
list_a = [1, 2, 3]
```

Output

14

Function Call Stack

Function Call Stack keeps track of function calls in progress

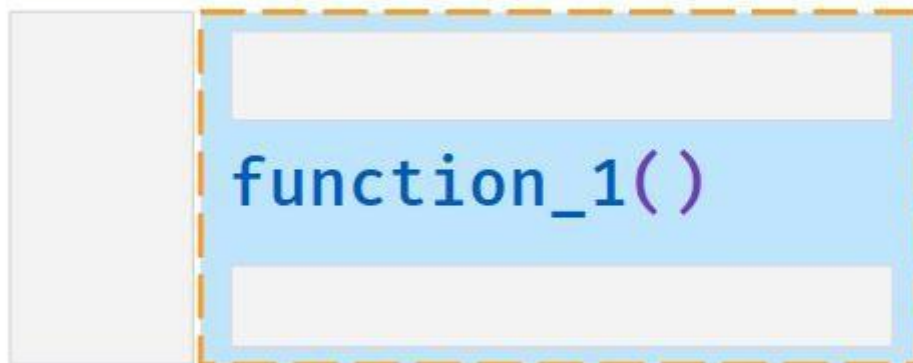


NXT
WA

Recursion

A function calling itself is called a Recursion

```
def function_1():
```



NXT
WAVE

Powered by
IB HUBS

Let's understand recursion with a simple example of multiplying N numbers

Multiply N Numbers

```
def factorial(n): # Recursive Function
```

```
    if n == 1: # Base Case return
        1
    return n * factorial(n - 1) # Recursion
num = int(input())
result = factorial(num)
print(result)
```

Base Case

A recursive function terminates when base condition is met

Input

3

Output

6

Without Base case

Code

```
def factorial(n):
    return n * factorial(n - 1)
num = int(input())
result = factorial(num)
print(result)
```

Input

3

Output

RecursionError: maximum recursion depth exceeded

List Methods

Python provides list methods that allow us to work with lists.

Let's learn few among them

append()

extend()

insert()

`pop()`

`clear()`

`remove()`

`sort()`

`index()`

Append

`list.append(value)`

Adds an element to the end of the list.

Code

```
list_a = []
for x
in range(1,4):
    list_a.append(x)
print(list_a)
```

Output

`[1, 2, 3]`

Extend

`list_a.extend(list_b)`

Adds all the elements of a sequence to the end of the list.

Code

```
list_a = [1, 2, 3]
list_b = [4, 5, 6]
list_a.extend(list_b)
print(list_a)
```

Output

`[1, 2, 3, 4, 5, 6]`

Insert

`list.insert(index,value)`

Element is inserted to the list at specified index.

```
list_a = [1,
```

Code

```
        2, 3]  
list_a.insert(1,4)  
print(list_a)
```

Output

```
[1, 4, 2, 3]
```

Pop

```
list.pop()
```

Removes last element.

Code

```
list_a = [1, 2, 3]  
list_a.pop()  
print(list_a)
```

Output

```
[1, 2]
```

Remove

```
list.remove(value)
```

Removes the first matching element from the list.

Code

```
list_a = [1, 3, 2, 3]  
list_a.remove(3)  
print(list_a)
```

Output

```
[1, 2, 3]
```

Clear

```
list.clear()
```

Removes all the items from the list.

```
        2, 3]  
list_a.clear()  
print(list_a)
```

Output

```
list_a = [1,
```

Code

```
[1
```

Index

```
list.index(value)
```

Returns the index at the first occurrence of the specified value.

Code

```
list_a = [1, 3, 2, 3]
index = list_a.index(3)
print(index)
```

Output

```
1
```

Count

```
list.count(value)
```

Returns the number of elements with the specified value.

Code

```
list_a = [1, 2, 3]
count = list_a.count(2)
print(count)
```

Output

```
1
```

Sort

```
list.sort()
```

Sorts the list.

```
3, 2]
```

```
list_a.sort()
print(list_a)
```

Output

```
[1, 2, 3]
```

Sort & Sorted

```
sort()
```

Modifies the list

```
list_a = [1,
```

Code

Code

```
list_a = [1, 3, 2]
list_a.sort()
print(list_a)
```

Output

```
[1, 3, 2]
```

Tuples and Sequences

None

None

is an object which is a datatype of its own (NoneType).

Used to define no value or nothing.

Code

```
var = None
print(var)
print(type(var))
```

Output

```
None
```

```
<class 'NoneType'>
```

Function Without Return

Functions assigned to a variable, when function does not have a

`return`

statement, the variable will get the value

`None`

Code

```
def increment(a):  
    a += 1
```

```
a = 55
```

```
result = increment(a)
```

```
print(result)
```

Output

`None`

Function That Returns Nothing

When a function returns no value, the default value will be

`None`

Example-1

Code

```
def increment(a):  
    a += 1  
    return
```

```
a = 55
```

```
result = increment(a)
```

```
print(result)
```

Output

`None`

Example-2

Code

```
def increment(a):  
    a += 1  
    return None
```

```
a = 5  
result = increment(a)  
print(result)
```

Output	
--------	--

None

Example-3

Code

```
result = print("Hi")  
print(result)
```

Output	
--------	--

Hi
None

Tuple

Holds an ordered sequence of items.

Tuple is immutable object, where as list is a mutable object.

Code

```
result = print("Hi") print(result)
```

Output	
--------	--

Hi
None

Tuple

Holds an ordered sequence of items.

Tuple is immutable object, where as list is a	
---	--

mutable
object.

Code

```
a = 2 tuple_a = (5, "Six", a,  
8.2)
```

Creating a Tuple

Created by enclosing elements within (round) brackets.

Each item is separated by a comma.

Code

```
a = 2  
tuple_a = (5, "Six", a, 8.2)  
print(type(tuple_a))  
print(tuple_a)
```

Output

```
<class 'tuple'>  
(5, 'Six', 2, 8.2)
```

Tuple with a Single Item

Code

```
a = (1,)  
print(type(a))  
print(a)
```

Output

```
<class 'tuple'>  
(1,)
```

Accessing Tuple Elements

Accessing Tuple elements is also similar to string and list accessing and slicing.

Code

```
a = 2  
tuple_a = (5, "Six", a, 8.2)  
print(tuple_a[1])
```

Output

Six

Tuples are Immutable

Tuples does not support modification.

Code

```
tuple_a = (1, 2, 3, 5)
tuple_a[3] = 4
print(tuple_a)
```

Output

TypeError: 'tuple' object does not support item assignment

Operations can be done on Tuples

len()

Itera

g

Slicing

Extended Slicing

Converting to Tuple

tuple(sequence)

Takes a sequence and converts it into tuple.

String to Tuple

Code

```
color = "Red"
tuple_a = tuple(color)
print(tuple_a)
```

Output

('R', 'e', 'd')

List to Tuple

Code

```
list_a = [1, 2, 3]
tuple_a = tuple(list_a)
print(tuple_a)
```

Output

```
(1, 2, 3)
```

Sequence to Tuple

Code

```
tuple_a = tuple(range(4))
print(tuple_a)
```

Output

```
(0, 1, 2, 3)
```

Membership Check

Check if given data element is part of a sequence or not.

Membership Operators

`in`

`not in`

Example-1

Code

```
tuple_a = (1, 2, 3, 4)
is_part = 5 in tuple_a
print(is_part)
```

Output

```
False
```

Example-2

Code

```
tuple_a = (1, 2, 3, 4)
is_part = 1 not in tuple_a
print(is_part)
```

Output

```
False
```

List Membership

Code

```
list_a = [1, 2, 3, 4]
is_part = 1 in list_a
print(is_part)
```

Output

True

String Membership

Code

```
word = 'Python'
is_part = 'th' in word
print(is_part)
```

Output

True

Packing & Unpacking

Unpacking

Values of any sequence can be directly assigned to variables.

Number of variables in the left should match the length of sequence.

Code

```
tuple_a = ('R', 'e', 'd')
(s_1, s_2, s_3) = tuple_a
print(s_1)
print(s_2)
print(s_3)
```

Output

R

e

d

Errors in Packing

Code

```
tuple_a = ('R', 'e', 'd')
s_1, s_2 = tuple_a
print(s_1)
print(s_2)
```

Output

```
ValueError: too many values to unpack (expected 2)
```

Code

```
tuple_a = ('R', 'e', 'd')
s_1, s_2, s_3, s_4 = tuple_a
print(s_1)
```

Output

```
ValueError: not enough values to unpack (expected 4, got 3)
```

Tuple Packing

()

brackets are optional while creating tuples.

In Tuple Packing, Values separated by commas will be packed into a tuple.

Code

```
a = 1, 2, 3
print(type(a))
print(a)
```

Output

```
<class 'tuple'> (1,
2, 3)
```

Code

```
a = 1,
print(type(a))
print(a)
```

Output

```
<class 'tuple'> (1,)
```

Code

```
a, = 1,  
print(type(a))  
print(a)
```

Output

```
<class 'int'>  
1
```

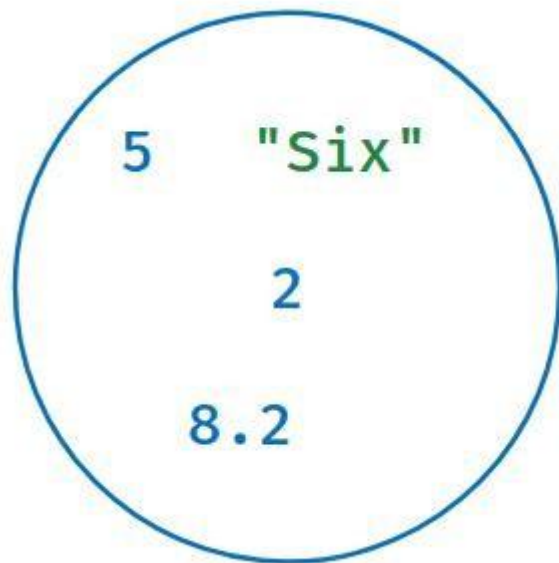
Sets

Unordered collection of items.

Every set element is

Unique (no duplicates)

Must be immutable



Creating a Set

Created by enclosing elements {curly} brackets.
within

Each item is separated by a
comma.

Code

```
a = 2
```

```
set_a = {5, "Six", a, 8.2}
print(type(set_a))
print(set_a)
```

Output

```
<class 'set'>
{8.2, 2, 'Six', 5}
```

Need not be in the same order as defined.

No Duplicate Items

Sets contain unique elements

Code

```
set_a = {"a", "b", "c", "a"}
print(set_a)
```

Output

```
{'b', 'a', 'c'}
```

Immutable Items

Set items must be immutable. As List is mutable, Set cannot have list as an item.

Code

```
set_a = {"a", ["c", "a"]}
```

```
print(set_a)
```

Output

```
TypeError: unhashable type: 'list'
```

Creating Empty Set

We use

```
set()
```

to create an empty set.

Code

```
set_a = set()
print(type(set_a))
print(set_a)
```

Output

```
<class 'set'> set()
```

Converting to Set

`set(sequence)`

takes any sequence as argument and converts to set, avoiding duplicates

List to Set

Code

```
set_a = set([1, 2, 1])
print(type(set_a))
print(set_a)
```

Output

```
<class 'set'>
```

```
{1, 2}
```

String to Set

Code

```
set_a = set("apple")
print(set_a)
```

Output

```
{'l', 'p', 'e', 'a'}
```

Tuple to Set

Code

```
set_a = set((1, 2, 1))
print(set_a)
```

Output

```
{1, 2}
```

Accessing Items

As sets are unordered, we cannot access or change an item of a set using

Indexing

Slicing

Code

```
set_a = {1, 2, 3}
print(set_a[1])
print(set_a[1:3])
```

Output

```
TypeError: 'set' object is not subscriptable
```

Adding Items

```
set.add(value)
```

adds the item to the set, if the item is not present already.

Code

```
set_a = {1, 3, 6, 2, 9}
set_a.add(7)
print(set_a)
```

Output

```
{1, 2, 3, 6, 7, 9}
```

Adding Multiple Items

```
set.update(sequence)
```

adds multiple items to the set, and duplicates are avoided.

Code

```
set_a = {1, 3, 9}
set_a.update([2, 3])
print(set_a)
```

Output

```
[2, 1, 3, 9]
```

Removing Specific Item

```
set.discard(value)
```

takes a single value and removes if present.

Code

```
set_a = {1, 3, 9}
set_a.discard(3)
print(set_a)
```

Output

```
{1, 9}
```

```
set_a.remove(value)
```

takes a value and remove if it present or raise an error.

Code

```
set_a = {1, 3, 9}
set_a.remove(5)
print(set_a)
```

Output

```
KeyError: 5
```

Operations on Sets

You can perform the following operations on Sets

```
clear()
```

```
len()
```

Iterating

Membership Check

Set Operations

Set objects also support mathematical operations like union, intersection, difference, and symmetric difference.

Union

Union of two sets is a set containing all elements of both sets.

```
set_a | set_b
```

or

```
set_a.union(sequence)  
union()
```

converts sequence to a set, and performs the union.

Code

```
set_a = {4, 2, 8}  
set_b = {1, 2}  
union = set_a | set_b  
print(union)
```

Output

```
{1, 2, 4, 8}
```

Code

```
set_a = {4, 2, 8} list_a =  
[1, 2] union =  
set_a.union(list_a)  
print(union)
```

Output

```
{1, 2, 4, 8}
```

Intersection

Intersection of two sets is a set containing common elements of both sets.

```
set_a & set_b
```

or

```
set_a.intersection(sequence)  
intersection()
```

converts sequence to a set, and perform the intersection.

Code

```
set_a = {4, 2, 8} set_b =  
{1, 2} intersection = set_a  
& set_b print(intersection)
```

Output

```
{2}
```

Code

Output

```
{2}
```

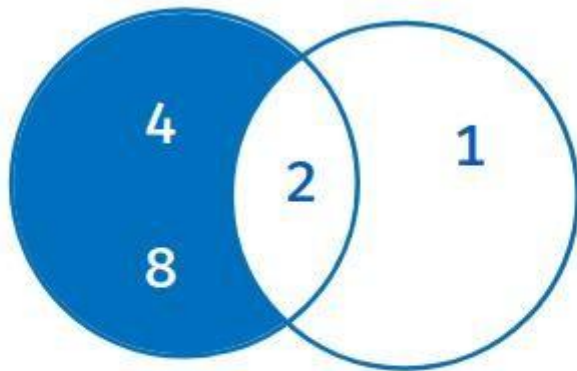
Code

Output

```
{2}
```

Difference

Difference of two sets is a set containing all the elements in the first set but not second.



```
set_a - set_b = {8, 4}
```

```
set_a - set_b
```

or

```
set_a.difference(sequence)  
difference()
```

converts sequence to a set.

Code

```
set_a = {4, 2, 8}  
set_b = {1, 2} diff  
= set_a - set_b  
print(diff)
```

Output

```
{8, 4}
```

Code

```
set_a = {4, 2, 8} tuple_a = (1,
2) diff =
set_a.difference(tuple_a)
print(diff)
```

Output

```
{8, 4}
```

Symmetric Difference

Symmetric difference of two sets is a set containing all elements which are not common to both sets.

```
set_a ^ set_b
```

or

```
set_a.symmetric_difference(sequence)
symmetric_difference()
```

converts sequence to a set.

Code

```
set_a = {4, 2, 8} set_b = {1,
2} symmetric_diff = set_a ^
set_b print(symmetric_diff)
```

Output

```
{8, 1, 4}
```

Code

```
set_a = {4, 2, 8}
set_b = {1, 2}
diff = set_a.symmetric_difference(set_b)
print(diff)
```

Output

```
{8, 1, 4}
```

Set Comparisons

Set comparisons are used to validate whether one set fully exists within another

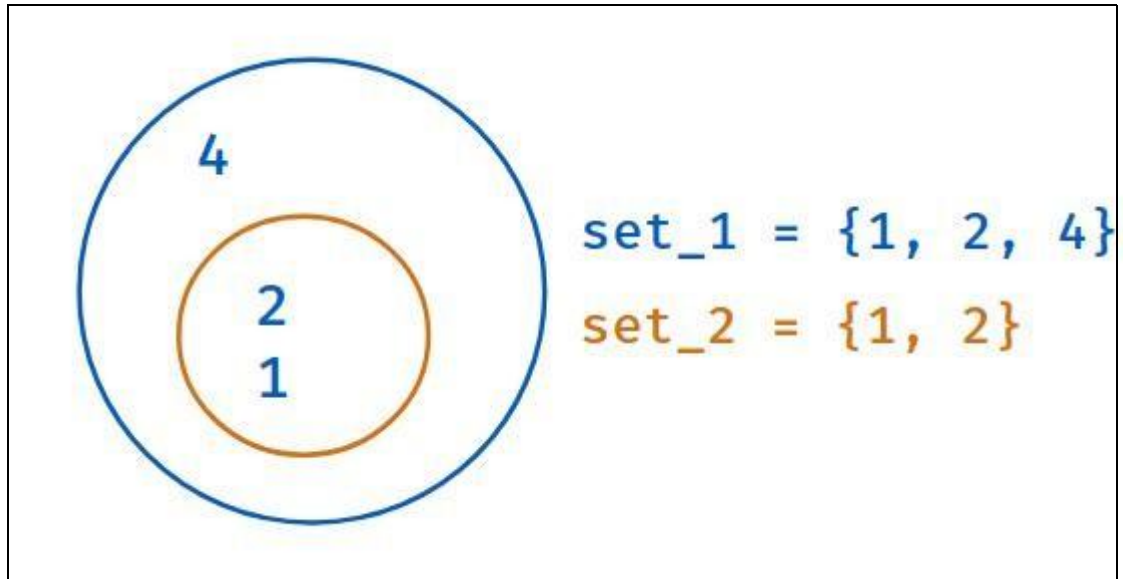
```
issubset()
```

```
issuperset
```

```
()
```

```
isdisjoint()
```

Subset



```
set2.issubset(set1)
```

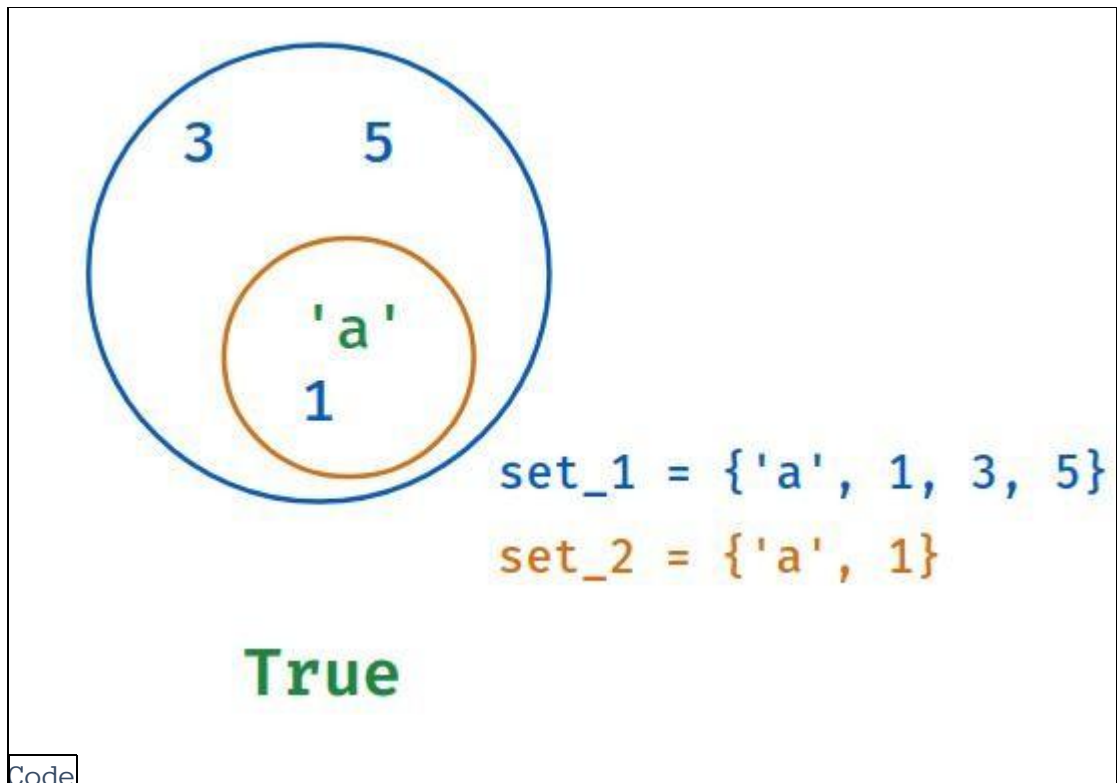
Returns

```
True
```

if all elements of second set are in first set. Else,

```
False
```

Example-1

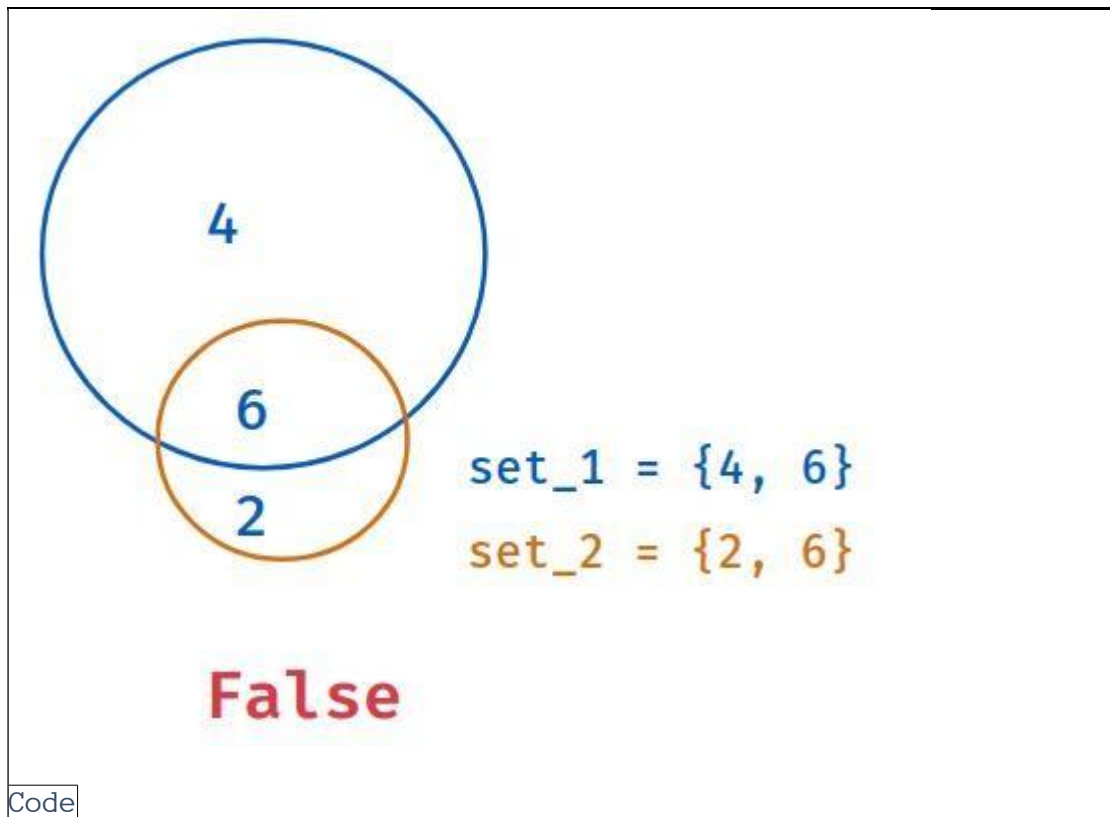


```
set_1 = {'a', 1, 3, 5} set_2 =  
{ 'a', 1} is_subset =  
set_2.issubset(set_1)  
print(is_subset)
```

Output

True

Example-2

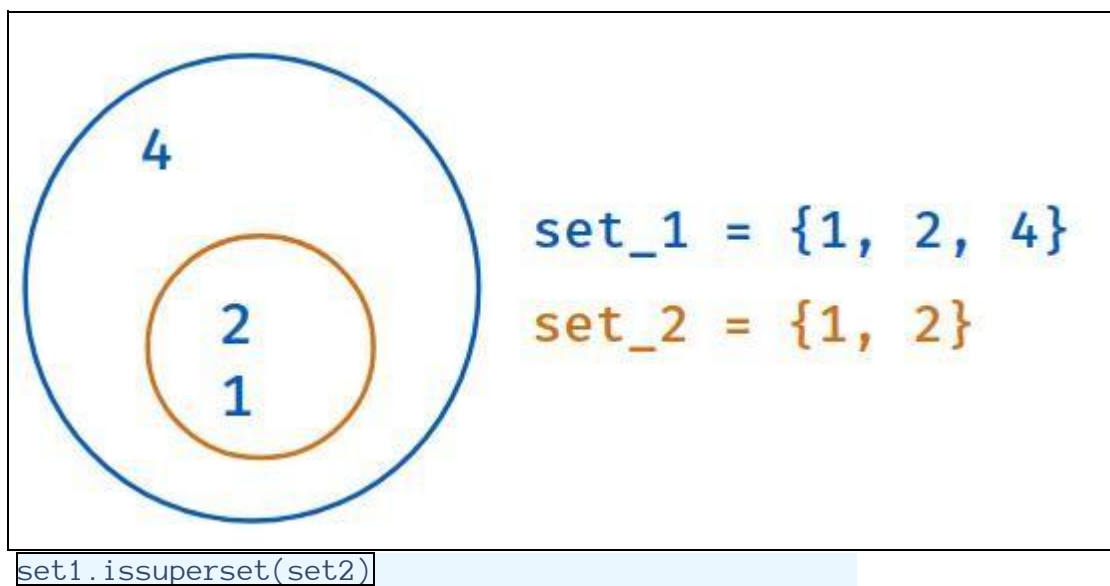


```
set_1 = {4, 6} set_2 = {2, 6}  
is_subset = set_2.issubset(set_1)  
print(is_subset)
```

Output

False

SuperSet



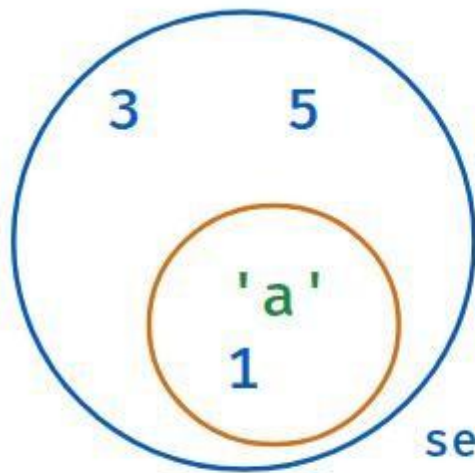
Returns

True

if all elements of second set are in first set. Else,

False

Example - 1



set_1 = {'a', 1, 3, 5}

set_2 = {'a', 1}

True

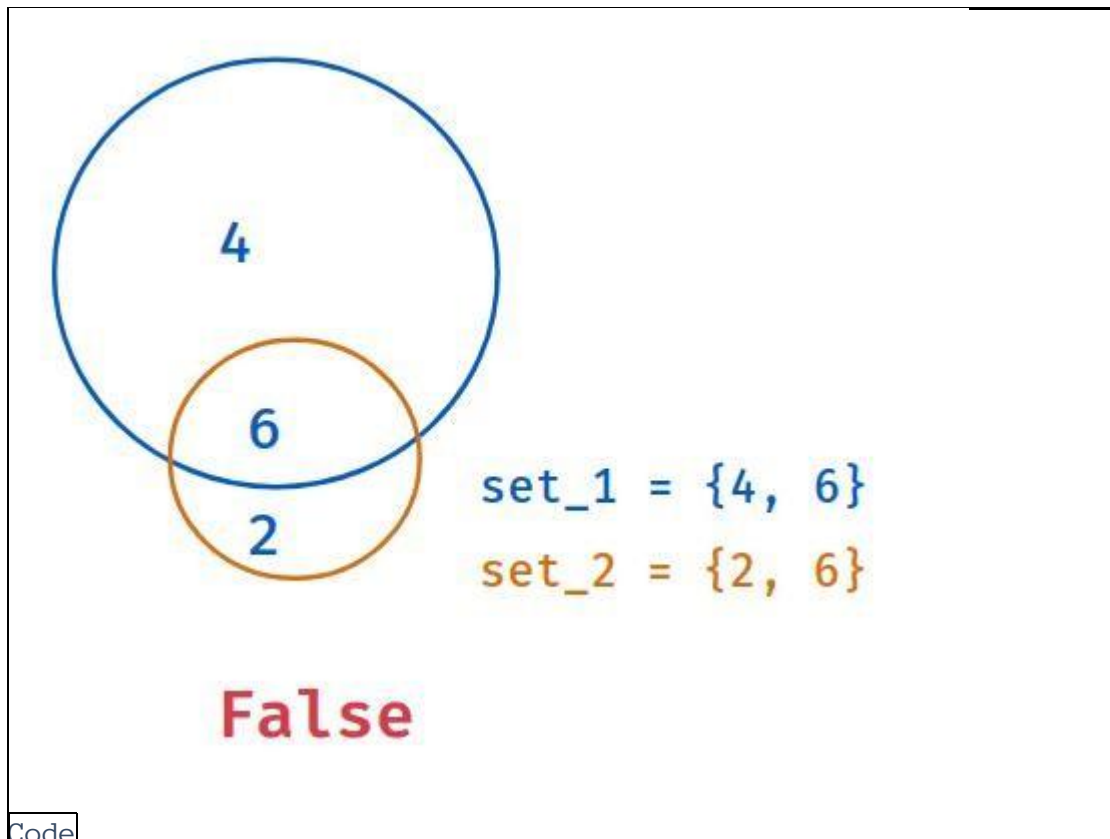
Code

```
set_1 = {'a', 1, 3, 5} set_2 = {'a',  
1} is_superset =  
set_1.issuperset(set_2)  
print(is_superset)
```

Output

True

Example-2



```
set_1 = {4, 6} set_2 = {2, 6} is_superset =  
set_1.issuperset(set_2) print(is_superset)
```

Output

False

Disjoint Sets

```
set1.isdisjoint(set2)
```

Returns

```
True
```

when they have no common elements. Else,

```
False
```

Code

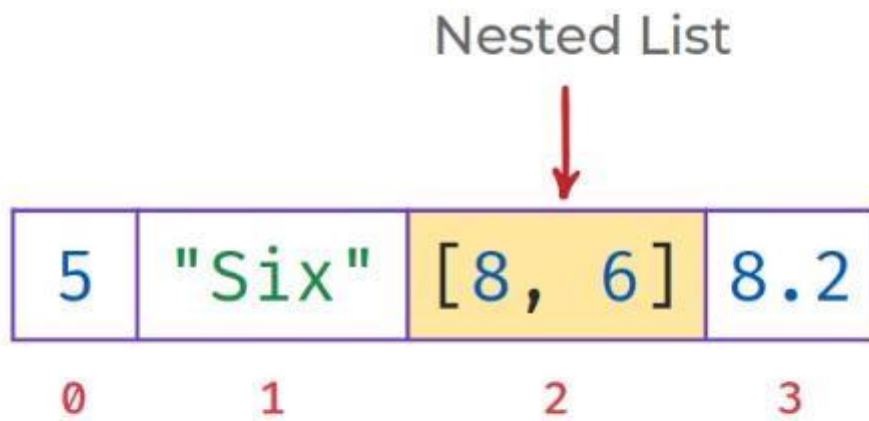
```
set_a = {1, 2} set_b = {3, 4} is_disjoint =  
set_a.isdisjoint(set_b) print(is_disjoint)
```

Output

True

Nested Lists & String Formatting

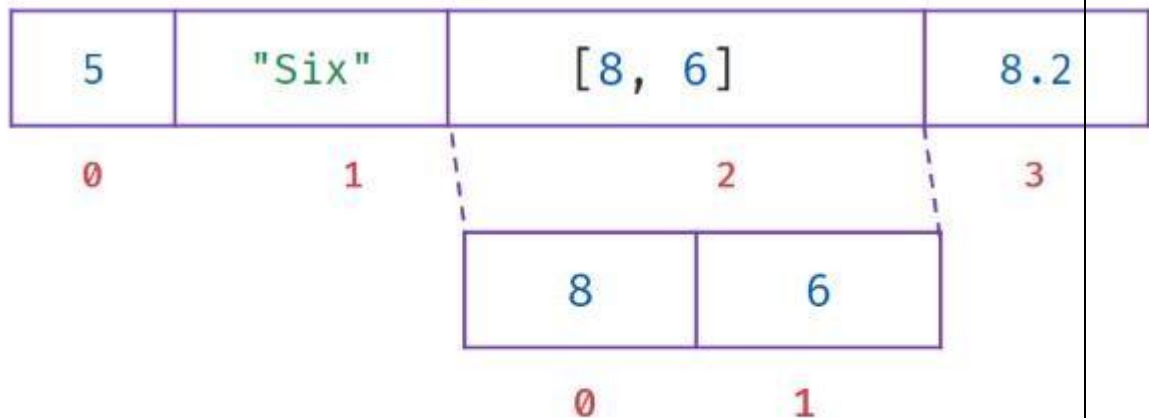
A list as an item of another list.



NXT
WAVE

Powered by
IB HUBS

Accessing Nested List



NXT
WAVE

Powered by
IB HUBS

Code

```
list_a = [5, "Six", [8, 6], 8.2] print(list_a[2])
```

Output	
--------	--

```
[8, 6]
```

Accessing Items of Nested List

Example-1

Code

```
list_a = [5, "Six", [8, 6], 8.2] print(list_a[2][0])
```

Output	
--------	--

```
8
```

Example-2

Code

```
list_a = ["Five", "Six"] print(list_a[0][1])
```

Output	
--------	--

```
i
```

String Formatting

Code	
------	--

```
name = input()
```

```
age = int(input())
```

```
msg = ("Hi " + name + ". You are " + str(age) + " years old.")
```

```
print(msg)
```

String formatting simplifies this concatenation.

It increases the readability of code and type conversion is not required.

Add Placeholders

Add placeholders

```
{}
```

where the string needs to be formatted. `msg = "Hi
{}. You are {} years old." msg.format(val_1,
val_2,..)`

Inserts values inside the string's placeholder

0

Code

```
name = "Raju" age
= 10
msg = "Hi {}. You are {} years old." print(msg.format(name, age))
```

Output

Hi Raju. You are 10 years old.

Number of Placeholders

Code

```
name = "Raju" age
= 10
msg = "Hi {}. You are {} years old {}." print(msg.format(name, age))
```

Output

IndexError: Replacement index 2 out of range for positional args tuple

Numbering Placeholders

Numbering placeholders, will fill values according to the position of arguments.

Code

```
name = input() age = int(input()) msg = "Hi
{0}. You are {1} years old."
print(msg.format(name, age))
```

Hi Raju. You are 10 years old.

Code

```
name = input() age = int(input()) msg = "Hi
{1}. You are {0} years old."
print(msg.format(name, age))
```

Output

Hi 10. You are Raju years old.

Naming Placeholder

Naming placeholders will fill values according to the keyword

Output

arguments.

Code

```
name = input() age = int(input()) msg = "Hi {name}.  
You are {age} years old."  
print(msg.format(name=name, age=age))
```

Output

```
Hi Raju. You are 10 years old.
```

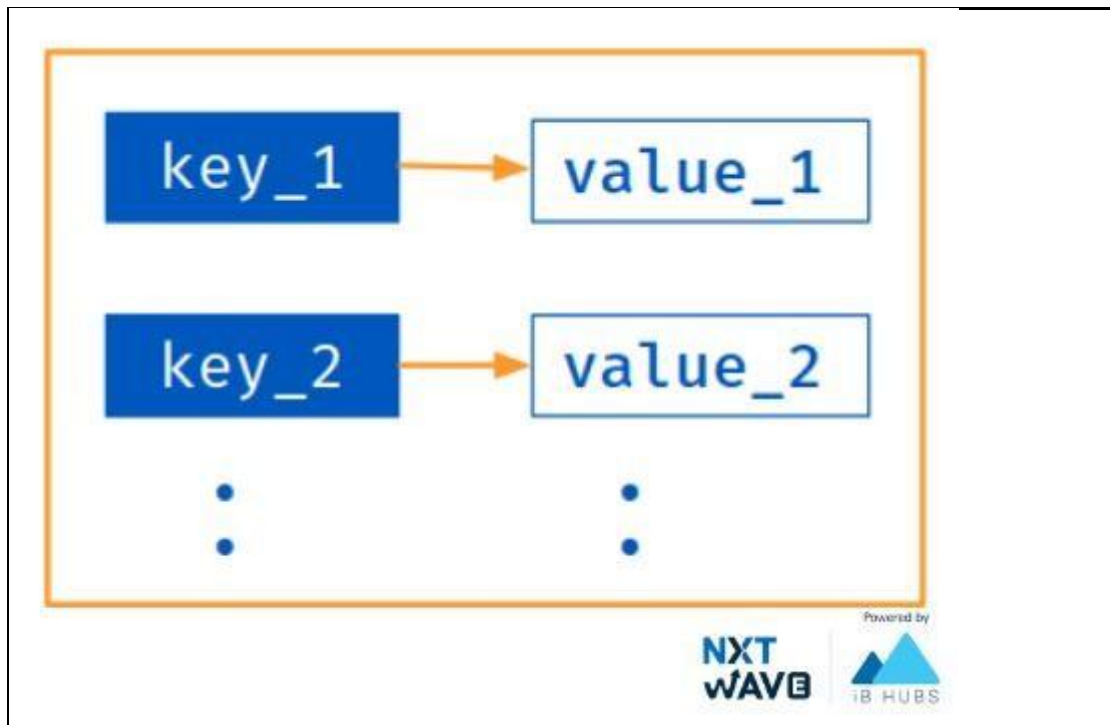
Dictionaries

Unordered collection of items.

Every dictionary item is a

Key-value

pair.



Creating a Dictionary

Created by enclosing items within

{curly}

brackets

Each item in dictionary has a key - value pair separated by a comma .

Code

```
dict_a = {  
    "name": "Teja",  
    "age": 15  
}
```

Key - Value Pairs

Code

```
dict a = {  
    "name": "Teja",  
    "age": 15  
}
```

In the above dictionary, the

keys are

name and Age

values are

Teja

and

15

Collection of Key-Value Pairs

Code

```
dict_a = { "name": "Teja",  
           "age": 15 }  
print(type(dict_a))  
print(dict_a)
```

Output

```
<class 'dict'> {'name': 'Teja', 'age':  
15}
```

Immutable Keys

Keys must be of immutable type and must be unique.

Values can be of any data type and can repeat.

Code

```
dict_a = {  
    "name": "Teja",  
    "age": 15,  
    "roll_no": 15 }
```

Creating Empty Dictionary

Code - 1

```
dict_a = dict() print(type(dict_a))  
print(dict_a)
```

Output

```
<class 'dict'>  
{}
```

Code - 2

```
dict_a = {}  
print(type(dict_a)) print(dict_a)
```

Output

```
<class 'dict'>  
{}
```

Accessing Items

To access the items in dictionary, we use square bracket

`[`

along with the

`key`

to obtain its value.

Code

```
dict_a = {  
    'name': 'Teja',  
    'age': 15 }  
print(dict_a['name'])
```

Output

Teja

Accessing Items - Get

The

`get()`

method returns

`None`

if the key is not found.

Code

```
dict_a = {  
    'name': 'Teja',  
    'age': 15 }  
print(dict_a.get('name'))
```

Output

Teja

```
'name': 'Teja',  
'age': 15
```

```
print(dict_a.get('city'))
```

Code

```
dict_a = {  
}
```

Output

None

KeyError

When we use the square brackets

```
{
```

to access the key-value, `KeyError` is raised in case a key is not found in the dictionary.

Code

```
dict_a = {'name': 'Teja', 'age': 15 }  
print(dict_a['city'])
```

Output

```
KeyError: 'city'
```

Quick Tip

If we use the square brackets

```
{
```

```
},
```

```
KeyError
```

is raised in case a key is not found in the dictionary. On the other hand, the

```
get()
```

method returns

```
None
```

if the key is not found.

Membership Check

Checks if the given key exists.

Code

```
dict_a = {  
    'name': 'Teja',  
    'age': 15 }  
result = 'name' in dict_a  
print(result)
```

Output

```
True
```

Operations on Dictionaries

We can update a dictionary by

Adding a key-value pair

Modifying existing item

s

Deleting existing items

Adding a Key-Value Pair

Code

```
dict_a = {'name': 'Teja', 'age': 15 }  
dict_a['city'] = 'Goa'  
print(dict_a)
```

Output

```
{'name': 'Teja', 'age': 15, 'city': 'Goa'}
```

Modifying an Existing Item

As dictionaries are mutable, we can modify the values of the keys.

Code

```
dict_a = {  
    'name': 'Teja',  
    'age': 15  
}  
dict_a['age'] = 24  
print(dict_a)
```

Output

```
{'name': 'Teja', 'age': 24}
```

Deleting an Existing Item

We can also use the

`del`

keyword to remove individual items or the entire dictionary itself.

Code

```
dict_a = {  
    'name': 'Teja',  
    'age': 15 }  
del dict_a['age']  
print(dict_a)
```

Output

```
{'name': 'Teja'}
```

Dictionary Views

They provide a dynamic view on the dictionary's entries, which means that when the dictionary changes, the view reflects these changes.

Dictionary Methods

`dict.keys()`

o returns dictionary Keys

`dict.values()`

o returns dictionary Values

`dict.items()`

o returns dictionary items(key-value) pairs

The objects returned by

`keys()`

,
`values()`

&

`items()`

are View Objects.

Getting Keys

The

`keys()`

method returns a view object of the type `dict_keys` that holds a list of all keys.

Code

```
dict_a = {  
    'name': 'Teja',  
    'age': 15 }  
print(dict_a.keys())
```

Output

`dict_keys(['name', 'age'])`

Getting Values

The

```
values()
```

method returns a view object that displays a list of all the values in the dictionary.

Code

```
dict_a = {  
    'name': 'Teja',  
    'age': 15 }  
print(dict_a.values())
```

Output

```
dict_values(['Teja', 15])
```

Getting Items

The

```
items()
```

method returns a view object that displays a list of dictionary's (key, value) tuple pairs.

Code

```
dict_a = {  
    'name': 'Teja',  
    'age': 15 }  
print(dict_a.items())
```

Output

```
dict_items([('name', 'Teja'), ('age', 15)])
```

Iterate over Dictionary Views

Example - 1

Code

```
dict_a = {  
    'name': 'Teja',  
    'age': 15  
}  
for key in dict_a.keys():  
    print(key)
```

Output

```
name age
```

Example - 2

Code

```
dict_a = {  
    'name': 'Teja',  
    'age': 15 }  
keys_list = list(dict_a.keys()) print(keys_list)
```

Output

```
[ 'name', 'age' ]
```

Example - 3

Code

```
dict_a = {  
    'name': 'Teja',  
    'age': 15 }  
for value in dict_a.values():  
    print(value)
```

Output

```
Teja
```

```
15
```

Example - 4

Code

```
dict_a = {  
    'name': 'Teja',  
    'age': 15 }  
for key, value in dict_a.items():  
    pair = "{} {}".format(key, value)  
    print(pair)
```

Output

```
name Teja age  
15
```

Dictionary	View	Objects
------------	------	---------

```
keys()
```

,

```
values()
```

&

```
items()
```

are called Dictionary Views as they provide a dynamic view on the dictionary's items.

Code

```
dict_a = {  
    'name': 'Teja',  
    'age': 15 }  
view = dict_a.keys()  
print(view)  
dict_a['roll_no'] = 10  
print(view)
```

Output

```
dict_keys(['name', 'age']) dict_keys(['name', 'age', 'roll_no'])
```

Converting to Dictionary

```
dict(sequence)
```

takes any number of key-value pairs and converts to dictionary.

Code	
	<pre>list_a = [("name", "Teja"), ["age", 15], ("roll_no", 15)] dict_a = dict(list_a) print(dict_a)</pre>
Output	

```
{'name': 'Teja', 'age': 15, 'roll_no': 15}
```

Code	
	<pre>list_a = ["name", "Teja", 15] dict_a = dict(list_a) print(dict_a)</pre>
Output	

```
ValueError: dictionary update sequence element #0 has length 4; 2 is required
```

Type of Keys

A dictionary key must be of a type that is immutable.

Type	Example	Can be used as key?
Integers	1000	Yes
Floats	10.25	Yes
Strings	'Hello'	Yes
Lists	[1, 5]	No (Mutable)
Sets	{'a', 'b'}	No (Mutable)
Dictionaries	{'a': 'b'}	No (Mutable)
Tuples	(1, 5)	Yes. Only if all items are immutable

Working with Dictionaries

Dictionary Methods

Python provides dictionary methods that allow us to work with dictionaries.

copy() get()
update()
fromkeys()

and more..

Let's learn few among them

Referring Same Dictionary Object

Code	
<pre>dict_a = { 'name': 'Teja', 'age': 15 } dict_b = dict_a dict_b['age'] = 20 print(dict_a) print(id(dict_a)) print(id(dict_b))</pre>	
Output	

```
{'name': 'Teja', 'age': 20}  
  
140170705626624  
  
140170705626624
```

Copy of Dictionary	
dict.copy()	

returns copy of a dictionary.

Code	
<pre>dict_a = { 'name': 'Teja', 'age': 15 } dict_b = dict_a.copy() dict_b['age'] = 20 print(dict_a) print(id(dict_a)) print(id(dict_b))</pre>	
Output	

```
{'name': 'Teja', 'age': 15}  
  
140664418952704  
  
140664418952896
```

Copy of List	
--------------	--

Code

```
list_a = ['Teja', 15] list_b =  
list_a.copy() list_b.extend([20])  
print(list_a) print(id(list_a))  
print(id(list_b))
```

Output

```
['Teja', 15]
```

```
139631861316032
```

```
139631860589504
```

Operations on Dictionaries

`len()` `clear()`

Membership Check

Code

```
dict_a = { 'name':  
'Teja',  
'age': 15 }  
print(len(dict_a)) # length of dict_a  
if 'name' in dict_a: # Membership Check print("True")  
dict_a.clear() # clearing dict_a print(dict_a)
```

Output

```
2
```

```
True
```

```
{}
```

Iterating

Cannot add/remove dictionary keys while iterating the dictionary.

Code

```
dict_a = {'name': 'Teja', 'age': 15} for k in  
dict_a.keys():  
    if k == 'name':  
        del dict_a[k]  
print(dict_a)
```

Output

```
RuntimeError: dictionary changed size during iteration
```

Arbitrary Function Arguments

Passing Multiple Values

We can define a function to receive any number of arguments.

We have already seen such functions

```
max(*args) max(1,2,3..)
```

```
min(*args) min(1,2,3..)
```

```
def func(*args):
```



Variable Length Arguments

Variable Length Arguments

Variable length arguments are packed as tuple.

Code

```
def more_args(*args):  
    print(args)
```

```
more_args(1, 2, 3, 4)
```

```
more_args()
```

Output

```
(1, 2, 3, 4)
```

```
()
```

Unpacking as Arguments

If we already have the data required to pass to a function as a sequence, we can

unpack it with

```
*
```

while passing.

Code

```
def greet(arg1="Hi", arg2="Ram"):
    print(arg1 + " " + arg2)
data = ["Hello", "Teja"]
greet(*data)
```

Output	
--------	--

Hello Teja

Multiple Keyword Arguments

We can define a function to receive any number of keyword arguments.

Variable length kwargs are packed as dictionary.

```
def func(**kwargs):
```



Variable Length
Keyword Arguments

Code

```
def more_args(**kwargs): print(kwargs)
more_args(a=1, b=2) more_args()
```

Output	
--------	--

{'a': 1, 'b': 2}

{}}

Iterating

`kwargs` is a dictionary. We can iterate over them like any other dictionary.

Code

```
def more_args(**kwargs):
```

```
for i, j in kwargs.items():
    print('{}:{}'.format(i,j)) more_args(a=1,
b=2)
```

Output	
--------	--

a:1 b:2

Unpacking as Arguments

Code - 1	
----------	--

```
def greet(arg1="Hi", arg2="Ram"):
    print(arg1 + " " + arg2)
data = {'arg1':'Hello', 'arg2':'Teja'}
greet(**data)
```

Output	
--------	--

Hello Teja

Code - 2	
----------	--

```
def greet(arg1="Hi", arg2="Ram"):
    print(arg1 + " " + arg2)
data = {'msg':'Hello', 'name':'Teja'}
greet(**data)
```

Output	
--------	--

TypeError: greet() got an unexpected keyword argument 'msg'

Cheat Sheet

Introduction to OOP

Good Software

Before jumping into Object Oriented Programming, let's understand the word **Software**.

Software is an easily changeable tool/product that performs a specific task.

The ease of changing or making changes to the software is referred as its softness.



A good software should keep the users happy, by delivering what they need.

A good software should also keep the developers happy. Ease of making changes (softness) keeps developers happy. So is should be:

- o Easy to understand and make changes.

Object-Oriented Programming System (OOPS) is a way of designing, developing software that is easy to change. approachin

- o Easy to fix bugs and add new features within the scope.

g,

Note

Keep in mind that building software differs from solving the coding questions (that you are doing in practice).

A fundamental difference is that you move on to the next problem once you solve a coding problem. But with software, once you build it, you are often required to add new features and fix bugs

that need you to make changes to the code written by you or your colleagues. So unlike the coding questions, with software, you have to keep working with the same code for a long time.

Therefore, ease of understanding (code-readability) and ease of making changes (code-maintainability) to the code become crucial in software development.

The techniques and concepts you learn in this topic are developed over time based on software developers' experiences to make working code easier.

OOPs

Object-Oriented Programming is a way of approaching, designing and developing software, so that the components of the software and the interactions between them resembles real-life objects and their interactions.

Proper usage of OOPS concepts helps us build well-organized systems that are easy to use and extend.

Describing Real-Life Objects

In Object Oriented Programming, we model software after real-life objects. To be good at modeling software after real-life objects, we should be able to properly describe them.

Let us try describing these real-life objects



Object 1



Object 2

NXT
WAVE

Powered by
IB HUBS

The following description is a bad way of describing, as the information of an object scattered and unorganized.

Object 1 is a car and it has four tyres.

Object 2 is a dog and it has four legs.

Object 1 has four doors.

Object 1 can make sound.

Object 2, barks.

Object 1 is in blue color.

Object 2 is in brown color.

Organized Description

	In the below description we are grouping the information related to an object.	
--	--	--

Object 1 is a car and it has four tyres.
Object 1 has four doors.
Object 1 can make sound.
Object 1 is in blue color.
Object 2 is a dog and it has four legs.
Object 2, barks.
Object 2 is in brown color

	In the below approach, we further organize the information into	
	o What the object is?	
	o	
	o What the object has?	
		What the object can do?

Object 1 is a car
Object 1 has
Four tyres
Four seats
Four doors
and so on ... Object 1 can
Sound horn
Move Accelerate

Expand

The above description shows a simple framework where we describe object by specifying the properties that the object has and actions that the object can do. Organized Description should be

A clear separation of objects.	
A clear grouping of what object has and what it does.	

Cheat Sheet

Describing Similar Objects

Sometimes, the objects that we are describing are so similar that only values of the properties differ.



Object 3



Object 4

NXT
WAVE

Powered by
IB HUBS

Object 3 is a Mobile

Properties camera : 13 MP

storage : 16 GB battery life :

21 Hrs ram : 3 GB and so

on ...

Object 4 is a Mobile

Properties camera : 64 MP

Expand

In this case, the objects that we describe have completely the same set of properties like camera, storage, etc.

Template

For objects that are very similar to each other (objects that have the same set of actions and properties), we can create a standard **Form** or **Template** that can be used to describe different objects in that category.

Mobile Template

Model : Camera:

Storage:

Does it have a Face Unlock? Yes | No

Filled Template

Model : iPhone 12 Pro

Camera: 64MP

Storage: 128GB

Does it have a Face Unlock? Yes

Bundling Data

While modeling real-life objects with object oriented programming, we ensure to bundle related information together to clearly separate information of different objects.

Bundling of related properties and actions together is called Encapsulation.

Classes can be used to bundle related properties and actions.



Defining a Class

To create a class, use the keyword

class

class ClassName:



Properties
Actions

Special Method

In Python, a special method

init

is used to assign values to properties.

Code

```
class Mobile:
    def init (self, model, camera):
        self.model = model
        self.camera = camera
```

Properties & Values

Code

```
class Mobile:
    def __init__(self, model, camera):
        self.model = model
        self.camera = camera
    def make_call(self, number):
        print("calling..")
```

In the above example,

model

and

camera

are the properties and values are which passed to the

init

method.

Action

```
class Mobile:
    def __init__(self, model, camera):
        self.model = model
        self.camera = camera
    def make_call(self, number):
        print("calling..")
```

In the above example, the below function is an action

```
def make_call(self, number):
    print("calling..")
```

In OOP terminology, properties are referred as

attributes

actions are referred as methods

Using a Class

To use the defined like a blueprint, actual values.

class, we have to instantiate it. A class is while its instance is based on that class with

Instance of Class

Syntax for creating an instance of class looks similar to function call.

An instance of class is Object.

Code

```
class Mobile:
    def __init__(self, model, camera):
        self.model = model
        self.camera = camera
```

```
mobile_obj = Mobile(
    "iPhone 12 Pro",
    "12 MP")
print(mobile_obj)
```

Class Object

An object is simply a collection of attributes and methods that act on those data.

```
class Mobile:
    def __init__(self, model, camera):
        self.model = model
        self.camera = camera
```

```
mobile_obj = Mobile(
    "iPhone 12 Pro",
    "12 MP")
print(mobile_obj)
```

Method Arguments & Return Values

Similar to functions, Methods also support positional, keyword & default arguments and also return values.

Code

```
class Mobile:
    def __init__(self, model, camera): self.model = model
        self.camera= camera
    def make_call(self,number): return "calling..{}".format(number)
```

Instance Methods of Class

For instance method, we need to first write

`self`

in the function definition and then the other arguments.

Code

```
class Mobile:
    def __init__(self, model, camera):
        self.model = model self.camera= camera
    def make_call(self,number): print("calling..{}".format(number))
mobile_obj = Mobile("iPhone 12 Pro", "12 MP")
mobile_obj.make_call(9876543210)
```

Output

calling..9876543210

Multiple Instances

Code

```
class Mobile:
    def __init__(self, model, camera):
        self.model = model self.camera= camera
    def make_call(self,number): print("calling..{}".format(number))
mobile_obj1 = Mobile("iPhone 12 Pro", "12 MP")
print(id(mobile_obj1))
mobile_obj2 = Mobile("Galaxy M51", "64 MP")
print(id(mobile_obj2))
```

Output

139685004996560

139685004996368

Type of Object

The class from which object is created is considered to be the type of object.

Code

```
class Mobile:
    def __init__(self, model, camera):
        self.model = model self.camera = camera
obj_1 = Mobile("iPhone 12 Pro", "12 MP")
print(type(obj_1))
```

Output

```
<class '__main__.Mobile'>
```

Cheat Sheet Classes and Objects

Attributes of an Object

Attributes can be set or accessed using

(dot) character.

Code

```
class Mobile:
    def __init__(self, model, storage):
        self.model = model self.storage = storage

obj = Mobile("iPhone 12 Pro",
"128GB")
print(obj.model)
```

Output

```
iPhone 12 Pro
```

Accessing in Other Methods

We can also access and update properties in other methods.

Code

```
class Mobile:
    def __init__(self, model):
        self.model = model

    def get_model(self):
        print(self.model)

obj_1 = Mobile("iPhone 12
Pro")
obj_1.get_model()
```

Output

```
iPhone 12 Pro
```

Updating Attributes

It is recommended to update attributes through methods.

Code

```
class Mobile:
    def __init__(self, model):
        self.model = model

    def update_model(self, model):
        self.model = model

obj_1 = Mobile("iPhone 12") print(obj_1.model)
obj_1.update_model("iPhone 12 Pro") print(obj_1.model)
```

Output

```
iPhone 12 iPhone
12 Pro
```

Modeling Class

Let's model the scenario of shopping cart of ecommerce site.

The features a cart should have

can add an item

can remove an item from cart
update quantity of an item

to show list of items in cart

to show total price for the items in the cart

Code

```
class Cart:
    def __init__(self):
        self.items = {}
        self.price_details = {"book": 500, "laptop": 30000}

    def add_item(self, item_name, quantity):
        self.items[item_name] = quantity

    def remove_item(self, item_name):
        del self.items[item_name]

    def update_quantity(self, item_name, quantity):
        self.items[item_name] = quantity

    def get_cart_items(self):
        cart_items = list(self.items.keys())
        return cart_items

    def get_total_price(self):
        total_price = 0
        for item, quantity in self.items.items():
            total_price += quantity * self.price_details[item]
        return total_price
```

```
cart_obj = Cart()
cart_obj.add_item("book", 3)
cart_obj.add_item("laptop", 1)
print(cart_obj.get_total_price())
```

```
cart_obj.remove_item("laptop") print(cart_obj.get_cart_items())
cart_obj.update_quantity("book", 2) print(cart_obj.get_total_price())
```

Output	
--------	--

31500

['book']

1000

Cheat Sheet

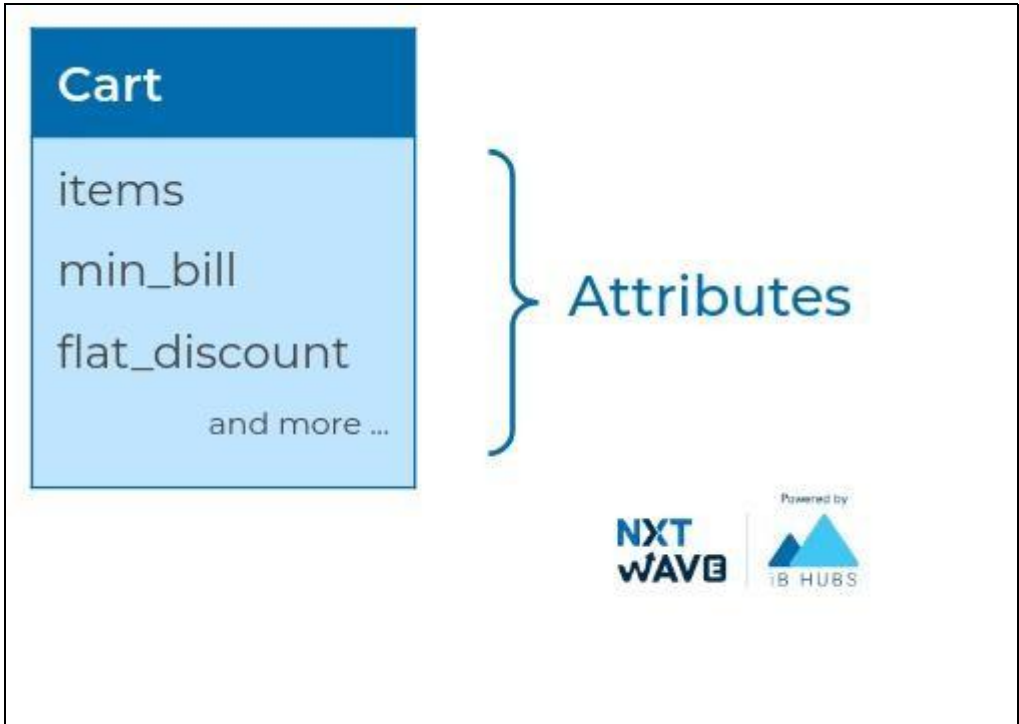
Attributes & Methods

Shopping Cart	
	Users can add different items to their shopping cart and checkout. value of the cart should be more than a minimum Rs. 100/- for the checkout.
	The total amount (
	During Offer Sales, all users get a flat discount on their cart and the minimum cart value will be Rs. 200/-.

Attributes

Broadly, attributes can be categorized as

Instance Attributes	
Class Attributes	



Instance Attributes

ffer for each instance of class are

Attributes whose value can modeled as inst attributes.

Ex: Items in Cart



Cart Object - B

Class Attributes

stay common for all the objects are

flat Discount

Attributes whose values modelled
Attributes.

Ex: Minimum Cart Bill,

instance
attribute

Cart

items

min_bill

flat_discount

Class
Attribute

NXT
WAVE

Power
to
P

Accessing Instance Attributes

Code

```

class Cart:
    flat_discount = 0
    min_bill = 100
    def __init__(self):
        self.items = {}
    def add_item(self, item_name, quantity):
        self.items[item_name] = quantity
    def display_items(self):
        print(self.items)

a = Cart()
a.display_items()

```

Output

NameError: name 'items' is not defined

Instance attributes can only be accessed using instance of class.

Self

self

passed to method contains the object, which is an instance of class.

Code

```

class Cart:
    flat_discount = 0
    min_bill = 100
    def __init__(self):
        self.items = {}
    def add_item(self, item_name, quantity):
        self.items[item_name] = quantity

```

```
def display_items(self): print(self)
```

Output	
--------	--

```
<__main__.Cart object at 0x7f6f83c9dfd0>
```

```
<__main__.Cart object at 0x7f6f83c9dfd0>
```

Accessing Using Self

Code

```
class Cart:
    flat_discount = 0
    min_bill = 100
    def __init__(self):
        self.items = {}
    def add_item(self, item_name, quantity):
        self.items[item_name] = quantity
    def display_items(self):
        print(self.items)

a = Cart()
a.add_item("book", 3)
a.display_items()
```

Output	
--------	--

```
{"book": 3}
```

Accessing Using Object

Code

```
class Cart:
    flat_discount = 0
    min_bill = 100
    def __init__(self):
        self.items = {}
    def add_item(self, item_name, quantity):
        self.items[item_name] = quantity
    def display_items(self):
        print(self.items)

a = Cart()
a.add_item("book", 3)
print(a.items)
```

Output	
--------	--

```
{"book": 3}
```

Accessing Using Class

Code

```
class Cart:
    flat_discount = 0
    min_bill = 100
    def __init__(self):
        self.items = {}
    def add_item(self, item_name, quantity):
        self.items[item_name] = quantity
```

```
def display_items(self):
    print(self.items)
print(Cart.items)
```

Output	
--------	--

AttributeError: type object 'Cart' has no attribute 'items'

Accessing Class Attributes

Example1

Code

```
class Cart:
    flat_discount = 0
    min_bill = 100
    def __init__(self):
        self.items = {}
    print(Cart.min_bill)
```

Output	
--------	--

100

Example2

Code

```
class Cart:
    flat_discount = 0
    min_bill = 100
    def __init__(self):
        self.items = {}
    def print_min_bill(self):
        print(Cart.min_bill)
a = Cart()
a.print_min_bill()
```

Output	
--------	--

100

Updating Class Attribute

Code

```
class Cart:
    flat_discount = 0
    min_bill = 100
    def print_min_bill(self):
        print(Cart.min_bill)
a = Cart()
b = Cart()
Cart.min_bill = 200
print(a.print_min_bill())
print(b.print_min_bill())
```

Output	
--------	--

200

Method

Broadly, methods can be categorized as

Instance Methods

Class Methods

Static Methods

Cart

items

min_bill

flat_discount

add_item()

display_items()

update_flat_discount()

Methods

NXT
WAVE

Powered by
IB HUBS

Instance Methods

Instance methods can access all attributes of the instance and have self as a parameter.

Cart

items

min_bill

flat_discount

add_item(self)

display_items(self)

update_flat_discount()

Instance
Methods

NXT
WAVE

Powered by
IB HUBS

Example 1

Code

```
class Cart:
    def __init__(self):
        self.items = {}
    def add_item(self, item_name, quantity):
        self.items[item_name] = quantity
    def display_items(self):
        print(self.items)
```

```
a = Cart()
a.add_item("book", 3)
a.display_items()
```

Output

```
{'book': 3}
```

Example2

Code

```
class Cart:
    def __init__(self):
```

```

        self.items = {}
    def add_item(self, item_name, quantity): self.items[item_name]
        = quantity self.display_items()
    def display_items(self):
        print(self.items)
a = Cart()
a.add_item("book", 3)

```

Output

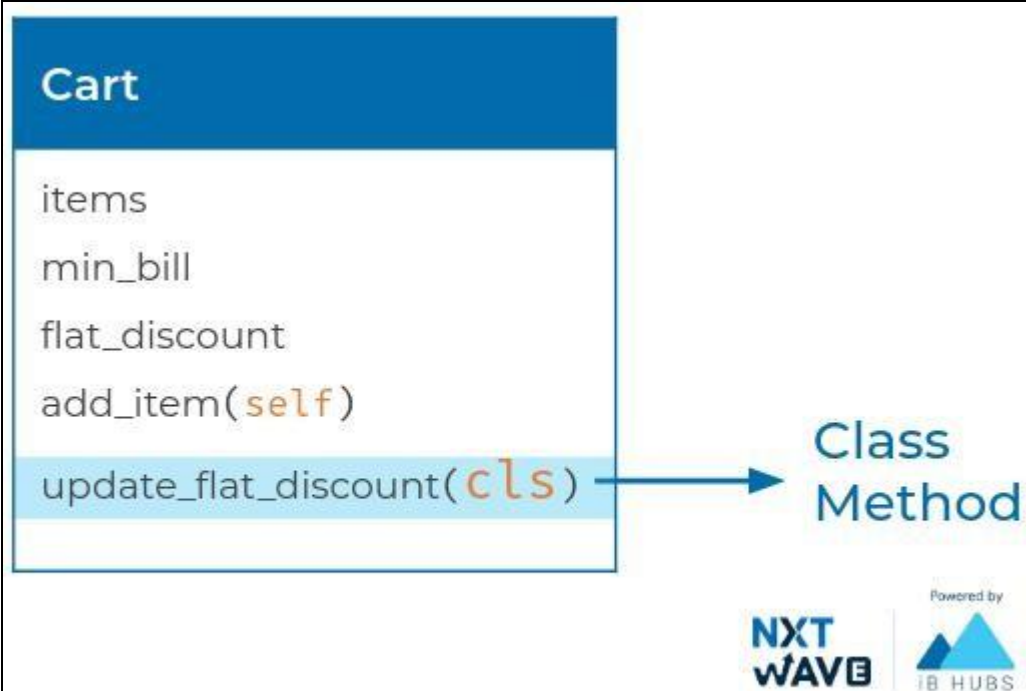
```
{'book': 3}
```

Class Methods

Methods which need access to class attributes but not instance attributes are marked as Class Methods. For class methods, we send

```
cls
```

as a parameter indicating we are passing the class.



Code

```

class Cart:
    flat_discount = 0
    min_bill = 100
    @classmethod
    def update_flat_discount(cls, new_flat_discount):
        cls.flat_discount = new_flat_discount
Cart.update_flat_discount(25)

```

```
print(Cart.flat_discount)
```

Output	
--------	--

25

```
@classmethod
```

decorator marks the method below it as a class method.

We will learn more about decorators in upcoming sessions.

Accessing Class Method

Code

```
class Cart:
    flat_discount = 0
    min_bill = 100
    @classmethod
    def update_flat_discount(cls, new_flat_discount):
        cls.flat_discount = new_flat_discount

    @classmethod
    def increase_flat_discount(cls, amount):
        new_flat_discount = cls.flat_discount + amount
        cls.update_flat_discount(new_flat_discount)

Cart.increase_flat_discount(50)
print(Cart.flat_discount)
```

Output	
--------	--

50

Static Method

We might need some generic methods that don't need access to either instance or class attributes. These type of methods are called Static Methods.

Usually, static methods are used to create utility functions which make more sense to be part of the class.

```
@staticmethod
```

decorator marks the method below it as a static method.

We will learn more about decorators in upcoming sessions.

Cart

items
min_bill
flat_discount
add_item(**self**)
update_flat_discount(**cls**)
greet()

Static Method

NXT WAVE

Powered by

IB HUBS

Code

```
class Cart:
```

```
    @staticmethod def greet():
        print("Have a Great Shopping")
```

```
Cart.greet()
```

Output

```
Have a Great Shopping
```

Overview of Instance, Class & Static Methods

Instance Methods	Class Methods	Static Methods
self as parameter	cls as parameter	No cls or self as parameters
No decorator required	Need decorator @classmethod	Need decorator @staticmethod
Can be accessed through object(instance of class)	Can be accessed through class	Can be accessed through class

Cheat Sheet

Inheritance

Products

Lets model

Electronics,

-commerce site having different products like Kids Wear, Grocery, etc.

Electronic Item

Following are few

Electronic Item

attributes & methods for an Electronic product.

Name

Price

Deal Price

Rating

Warranty in Months

Specifications

display_product_de

get_warranty(self

al

Grocery Item

Similarly, attribute

& methods for a Grocery item.

Grocery Item
Name
Price
Deal Price
Rating
Packed Date
Expiry Date
display_product_details(self)
get_expiry_date(self)



Common Attributes & Methods

All these products Electronics, Kids Wear, common attributes & methods.

Electronic Item
Name
Price
Deal Price
Rating
Warranty in Months
Specifications
display_product_details(self)
get_warranty(self)

Grocery Item
Name
Price
Deal Price
Rating
Packed Date
Expiry Date
display_product_details(self)
get_expiry_date(self)

Grocery etc.. have few

Specific Attributes & Methods		& methods of its own.																	
Also, each product has specific attributes																			
<table><tr><th>Electronic Item</th></tr><tr><td>Name</td></tr><tr><td>Price</td></tr><tr><td>Deal Price</td></tr><tr><td>Rating</td></tr><tr><td>Warranty in Months</td></tr><tr><td>Specifications</td></tr><tr><td>display_product_details(self)</td></tr><tr><td>get_warranty(self)</td></tr></table>	Electronic Item	Name	Price	Deal Price	Rating	Warranty in Months	Specifications	display_product_details(self)	get_warranty(self)	<table><tr><th>Grocery Item</th></tr><tr><td>Name</td></tr><tr><td>Price</td></tr><tr><td>Deal Price</td></tr><tr><td>Rating</td></tr><tr><td>Packed Date</td></tr><tr><td>Expiry Date</td></tr><tr><td>display_product_details(self)</td></tr><tr><td>get_expiry_date(self)</td></tr></table>	Grocery Item	Name	Price	Deal Price	Rating	Packed Date	Expiry Date	display_product_details(self)	get_expiry_date(self)
Electronic Item																			
Name																			
Price																			
Deal Price																			
Rating																			
Warranty in Months																			
Specifications																			
display_product_details(self)																			
get_warranty(self)																			
Grocery Item																			
Name																			
Price																			
Deal Price																			
Rating																			
Packed Date																			
Expiry Date																			
display_product_details(self)																			
get_expiry_date(self)																			
Electronic & Grocery Items		all																	

Electronic Item & Grocery Item will have which are common to all products. Lets attributes & methods as Product

Electronic Item

ie
e
Price
ng
ay_produc
anty in Mor
fications
arranty(se

attributes & methods

Modelling Classes

ite the common

Product

Name
Price
Deal Price
Rating
display_product_details(self)

Electronic Item

Product

Warranty in Months
Specifications
get_warranty(self)

Grocery Item

Product

Packed Date
Expiry Date
get_expiry_date(self)

NXT
WAVE

Post
18

Advantages of Modelling Classes as above

- Reusability
- Clear Separation
- More Organized

Inheritance

Inheritance is a and methods from

With Inheritance, mechanism by which a class inherits attributes another class. we can have

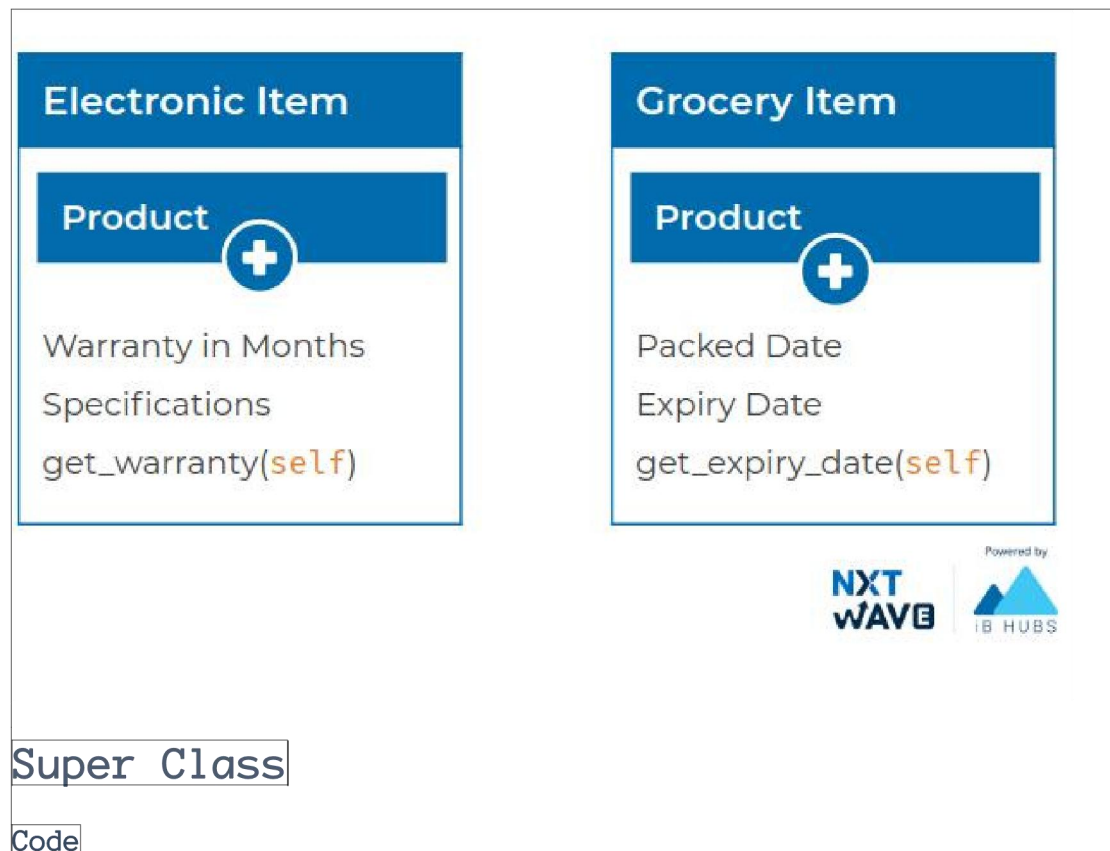
ElectronicItem

inherit the attributes & methods from

Product

instead of defining them again.

Product is Super/Base/Parent Class and ElectronicItem is Sub/Derived/Child Class.



```
class Product:
    def __init__(self, name, price, deal_price, ratings):
        self.name = name
        self.price = price
        self.deal_price = deal_price
        self.ratings = ratings
        self.you_save = price - deal_price
    def display_product_details(self):
        print("Product: {}".format(self.name))
        print("Price: {}".format(self.price))
        print("Deal Price: {}".format(self.deal_price))
        print("You Saved: {}".format(self.you_save))
        print("Ratings: {}".format(self.ratings))
```

```
p = Product("Shoes", 500, 250, 3.5)
```

```
p.display_product_details()
```

Output

Product: Shoes

Price: 500

Deal Price: 250

You Saved: 250

Ratings: 3.5

Sub Class

The subclass automatically inherits all the attributes & methods from its superclass.

Example1

Code

```
class Product:
    def __init__(self, name, price, deal_price, ratings):
        self.name = name
        self.price = price
        self.deal_price = deal_price
        self.ratings = ratings
        self.you_save = price - deal_price
    def display_product_details(self):
        print("Product: {}".format(self.name))
        print("Price: {}".format(self.price))
        print("Deal Price: {}".format(self.deal_price))
        print("You Saved: {}".format(self.you_save))
        print("Ratings: {}".format(self.ratings))

class ElectronicItem(Product):
    pass

class GroceryItem(Product):
    pass

e = ElectronicItem("TV", 45000, 40000, 3.5)
e.display_product_details()
```

Output

```
Product: TV

Price: 45000

Deal Price: 40000

You Saved: 5000

Ratings: 3.5
```

Example2

Code

```
class Product:
    def __init__(self, name, price, deal_price, ratings):
        self.name = name
        self.price = price
        self.deal_price = deal_price
        self.ratings = ratings
        self.you_save = price - deal_price
    def display_product_details(self):
        print("Product: {}".format(self.name))
        print("Price: {}".format(self.price))
        print("Deal Price: {}".format(self.deal_price))
        print("You Saved: {}".format(self.you_save))
        print("Ratings: {}".format(self.ratings))

class ElectronicItem(Product):
    pass

class GroceryItem(Product):
    pass

e = GroceryItem("milk", 25, 20, 3)
```

```
e.display_product_details()
```

Output

Product: milk

Price: 25

Deal Price: 20

You Saved: 5

Ratings: 3

Example3

Code

```
class Product:
    def __init__(self, name, price, deal_price, ratings):
        self.name = name
        self.price = price
        self.deal_price = deal_price
        self.ratings = ratings
        self.you_save = price - deal_price
    def display_product_details(self):
        print("Product: {}".format(self.name))
        print("Price: {}".format(self.price))
        print("Deal Price: {}".format(self.deal_price))
        print("You Saved: {}".format(self.you_save))
        print("Ratings: {}".format(self.ratings))

class ElectronicItem(Product):
    def set_warranty(self, warranty_in_months):
        self.warranty_in_months = warranty_in_months

    def get_warranty(self):
        return self.warranty_in_months

e = ElectronicItem("TV", 45000, 40000, 3.5)
e.set_warranty(24)
print(e.get_warranty())
```

Output

24

In the above example, calling

set_warranty

will create an attribute

warranty_in_months

Super Class & Sub Class

Superclass cannot access the methods and attributes of the subclass.

Code

```
class Product:
    def __init__(self, name, price, deal_price, ratings):
```

```

        self.name = name self.price = price self.deal_price =
        deal_price self.ratings = ratings self.you_save = price -
        deal_price
    def display_product_details(self):
        print("Product: {}".format(self.name)) print("Price: {}".format(self.price))
        print("Deal Price: {}".format(self.deal_price)) print("You Saved:
        {}".format(self.you_save)) print("Ratings: {}".format(self.ratings))

class ElectronicItem(Product):
    def set_warranty(self, warranty_in_months): self.warranty_in_months =
        warranty_in_months

    def get_warranty(self): return self.warranty_in_months
p = Product("TV",45000, 40000, 3.5)
p.set_warranty(24)

```

Output	
--------	--

```

AttributeError: 'Product' object has no attribute 'set_warranty'

```

Sub Class Method

Code	
------	--

```

class Product:
    def __init__(self, name, price, deal_price, ratings): self.name = name self.price = price
        self.deal_price = deal_price self.ratings = ratings

```

```

        self.you_save = price - deal_price
    def display_product_details(self):
        print("Product: {}".format(self.name)) print("Price: {}".format(self.price))
        print("Deal Price: {}".format(self.deal_price)) print("You Saved:
        {}".format(self.you_save)) print("Ratings: {}".format(self.ratings))

class ElectronicItem(Product):
    def set_warranty(self, warranty_in_months):
        self.warranty_in_months = warranty_in_months

    def get_warranty(self): return self.warranty_in_months
e = ElectronicItem("TV",45000, 40000, 3.5)
e.set_warranty(24)
e.display_product_details()

```

Output	
--------	--

```

Product: TV

Price: 45000

Deal Price: 40000

You Saved: 5000

Ratings: 3.5

```

Calling Super Class Method

We can call methods defined in superclass from the methods in the subclass.

Code	
------	--

```

class Product:
    def __init__(self, name, price, deal_price, ratings):
        self.name = name
        self.price = price
        self.deal_price = deal_price
        self.ratings = ratings
        self.you_save = price - deal_price
    def display_product_details(self):
        print("Product: {}".format(self.name)) print("Price: {}".format(self.price))
        print("Deal Price: {}".format(self.deal_price)) print("You Saved:
        {}".format(self.you_save)) print("Ratings: {}".format(self.ratings))

class ElectronicItem(Product):
    def set_warranty(self, warranty_in_months):
        self.warranty_in_months = warranty_in_months
    def get_warranty(self):
        return self.warranty_in_months

    def display_electronic_product_details(self):
        self.display_product_details()
        print("Warranty {} months".format(self.warranty_in_months))
e = ElectronicItem("TV",45000, 40000, 3.5)
e.set_warranty(24)
e.display_electronic_product_details()

```

Output	
--------	--

	Product: TV
	Price: 45000
	Deal Price: 40000
	You Saved: 5000
	Ratings: 3.5
	Warranty 24 months

Inheritance - Part 2

How would you design and implement placing order with the	
details of all the products bought?	

Composition

Modelling instances of one class as attributes of another class is called <u>Composition</u>
Code

```
class Product:
    def __init__(self, name, price, deal_price, ratings):
        self.name = name
        self.price = price
        self.deal_price = deal_price
        self.ratings = ratings
        self.you_save = price - deal_price

    def display_product_details(self):
        print("Product: {}".format(self.name))
        print("Price: {}".format(self.price))
        print("Deal Price: {}".format(self.deal_price))
        print("You Saved: {}".format(self.you_save))
        print("Ratings: {}".format(self.ratings))

    def get_deal_price(self):
        return self.deal_price

class ElectronicItem(Product):
    def set_warranty(self, warranty_in_months):
        self.warranty_in_months = warranty_in_months

    def get_warranty(self):
        return self.warranty_in_months

class GroceryItem(Product):
    pass

class Order:
    def __init__(self, delivery_speed, delivery_address):
        self.items_in_cart = []
        self.delivery_speed = delivery_speed
        self.delivery_address = delivery_address

    def add_item(self, product, quantity):
        self.items_in_cart.append((product, quantity))

    def display_order_details(self):
        for product, quantity in self.items_in_cart:
            product.display_product_details()
            print("Quantity: {}".format(quantity))

    def display_total_bill(self):
```

```
total_bill = 0
for product, quantity in self.items_in_cart:
    price = product.get_deal_price() * quantity
    total_bill += price
print("Total Bill: {}".format(total_bill))
```

```
milk = GroceryItem("Milk", 40, 25, 3.5)
tv = ElectronicItem("TV", 45000, 40000, 3.5)
order = Order("Prime Delivery", "Hyderabad")
order.add_item(milk, 2)
order.add_item(tv, 1)
order.display_order_details()
order.display_total_bill()
```

Output	
--------	--

Product: Milk
Price: 40
Deal Price: 25
You Saved: 15
Ratings: 3.5
Quantity: 2
Product: TV

Price: 45000

Deal Price: 40000

You Saved: 5000

Ratings: 3.5

In the above example, we are modelling

Product

as attribute of

Order

Overriding Methods

Sometimes, we require a method in the instances of a sub class to behave differently from the method in instance of a superclass.

Code

```
class Product:
```

```
    def __init__(self, name, price, deal_price, ratings):
        self.name = name
        self.price = price
        self.deal_price = deal_price
        self.ratings = ratings
        self.you_save = price - deal_price
```

```
    def display_product_details(self):
        print("Product: {}".format(self.name))
        print("Price: {}".format(self.price))
        print("Deal Price: {}".format(self.deal_price))
        print("You Saved: {}".format(self.you_save))
        print("Ratings: {}".format(self.ratings))
```

```
    def get_deal_price(self):
        return self.deal_price
```

```
class ElectronicItem(Product):
```

```
    def display_product_details(self):
        self.display_product_details()
        print("Warranty {} months".format(self.warranty_in_months))
```

```
    def set_warranty(self, warranty_in_months):
        self.warranty_in_months = warranty_in_months
```

```
    def get_warranty(self):
        return self.warranty_in_months
```

```
e = ElectronicItem("Laptop", 45000, 40000, 3.5)
```

```
e.set_warranty(10)
```

```
e.display_product_details()
```

Output

RecursionError: maximum recursion depth exceeded

Because

```
self.display_product_details()
```

in ElectronicItem class does not call the method in the superclass.

Super

Accessing Super Class's Method

`super()`

allows us to call methods of the superclass (Product) from the subclass.

Instead of writing and methods to access and modify warranty we can override

`init`

Let's add warranty of ElectronicItem.

Code

```
class Product:
    def __init__(self, name, price, deal_price, ratings):
        self.name = name
        self.price = price
        self.deal_price = deal_price
        self.ratings = ratings
        self.you_save = price - deal_price

    def display_product_details(self):
        print("Product: {}".format(self.name))
        print("Price: {}".format(self.price))
        print("Deal Price: {}".format(self.deal_price))
        print("You Saved: {}".format(self.you_save))
        print("Ratings: {}".format(self.ratings))

    def get_deal_price(self):
        return self.deal_price

class ElectronicItem(Product):
    def display_product_details(self):
        super().display_product_details()
        print("Warranty {} months".format(self.warranty_in_months))

    def set_warranty(self, warranty_in_months):
        self.warranty_in_months = warranty_in_months

    def get_warranty(self):
        return self.warranty_in_months

e = ElectronicItem("Laptop", 45000, 40000, 3.5)
e.set_warranty(10)
e.display_product_details()
```

Output	
--------	--

Product: Laptop

Price: 45000

Deal Price: 40000

You Saved: 5000

Ratings: 3.5

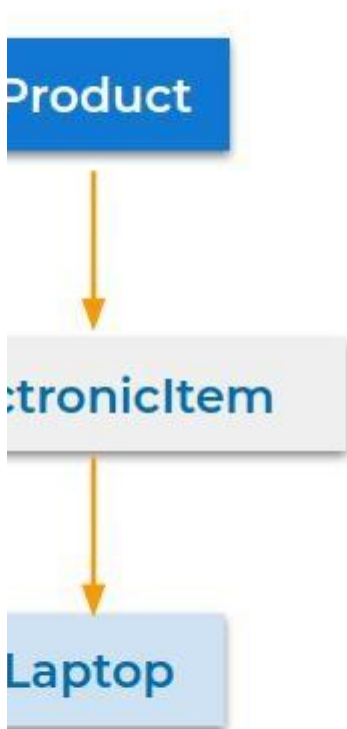
Warranty 10 months

MultiLevel Inheritance

We can also inherit from a subclass. This is called
Inheritance

MultiLevel

We can continue such inheritance to any depth in Python.



```
class Product:
    pass
```

ElectronicItem(Product):

class

```
class Laptop(ElectronicItem): Pass
```

Inheritance & Composition

When to use Inheritance?

When the classes have an **IS-A**

Prefer modeling with inheritance when relationship.

Car is a Vehicle

Truck is a Vehicle

Car is a Product

Order is a Product

When to use Composition?

Prefer modeling with inheritance when the classes have an **HAS-A** relationship.

Car has a Tyre



Order has a Product



Product has a Order



Car has a Vehicle



NXT
WAVE

Powered by
IB HUBS

Cheat Sheet

Standard Library

Built-in Functions

Built-in functions are Readily available for reuse.

Some of the built Functions are

- 1.
- 2.

`print()`

- 3.
- 4.
- 5.
- 6.

`max()`

- 7.
- 8.

9.
10.

`min()`

11.
12.
13.
14.

`len()`

15. 16. `and many more..`

Standard Library

Python provides several such useful values (constants), classes and functions.

This collection of predefined utilities is referred as the

Python Standard Library

All these functionalities are organized into different modules.

In Python context, any file containing a Python code is
called a `module`

These modules are further organized into folders known
as `packages`

Different modules are:

`collections random datetime`
`math and many more..`

Working with Standard Library

To use a functionality defined in a module we need to import that module in our program.

`import module name`

Math Module

`math` module provides us to access some common `math functions` and
`constants`.

Variable / Function / Class
defined in the module

`math.factorial(5)`

Module Name

NXT
WAVE

Powered by
IB HUBS

Code

```
import math print(math.factorial(5)) print(math.pi)
```

Output

120

3.141592653589793

Importing module

Importing a module and giving it a new name (aliasing)

Code

```
import math as m1  
print(m1.factorial(5))
```

Output

120

Importing from a Module

We can import just a specific definition from a module.

Code

```
from math import factorial
print(factorial(5))
```

Output

120

Aliasing Imports

We can also import a specific definition from a module and alias it

Code

```
from math import factorial as fact
print(fact(5))
```

Output

120

Random module

Randomness is useful in whenever uncertainty is required.

Forexample : Rolling a dice, flipping a coin, etc.,

`random`

module provides us utilities to create randomness.



NXT
WAVE

Powered by
IB HUBS

Randint

`randint()`

is a function in random module which returns a random integer in the given interval.

Code

```
import random
random_integer = random.randint(1, 10) print(random_integer)
```

Output

8

Choice

`choice()`

is a function in random module which returns a random element from the sequence.

Code

```
import random
random_ele = random.choice(["A","B","C"]) print(random_ele)
```

Output

B

To know more about [Python Standard Library](#), go through the authentic python documentation -

<https://docs.python.org/3/library/>

Map, Filter and Reduce

We worked with different sequences (list, tuples, etc.)

To simplify working with sequences we can use

`map()`

`filter()`

and

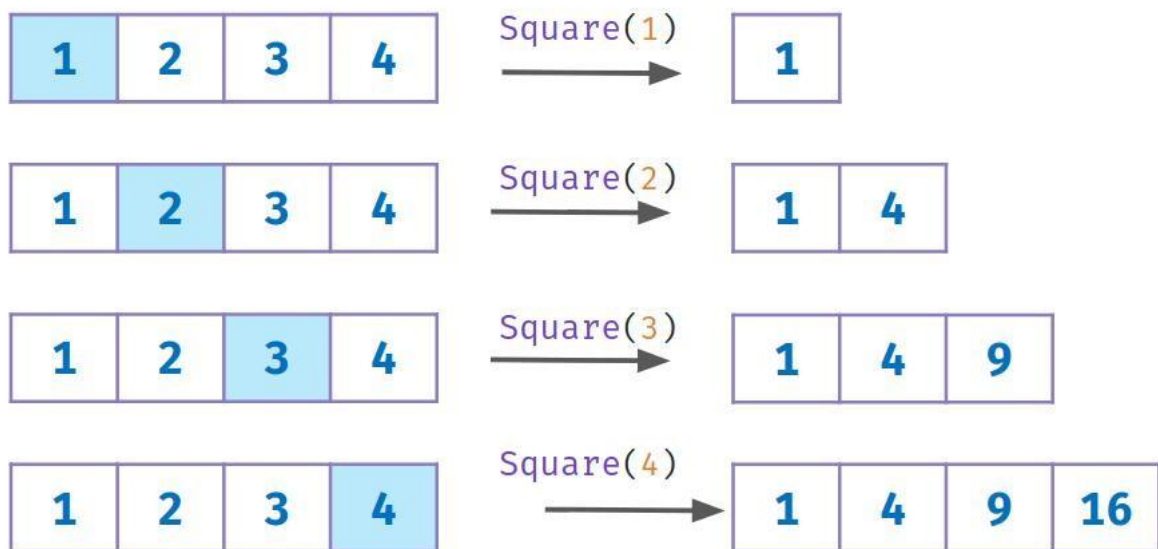
`reduce()`

functions.

Map

`map()`

applies a given function to each item of a sequence (list, tuple etc.) and returns a sequence of the results.



Example-1

Code

```
def square(n):  
    return n * n  
numbers = [1, 2, 3, 4] result = map(square,  
numbers) numbers_square = list(result)  
print(numbers_square)
```

Output

[1, 4, 9, 16]

Example-2

Code

```
numbers = list(map(int, input().split()))  
print(numbers)
```

Input

1 2 3 4

Output

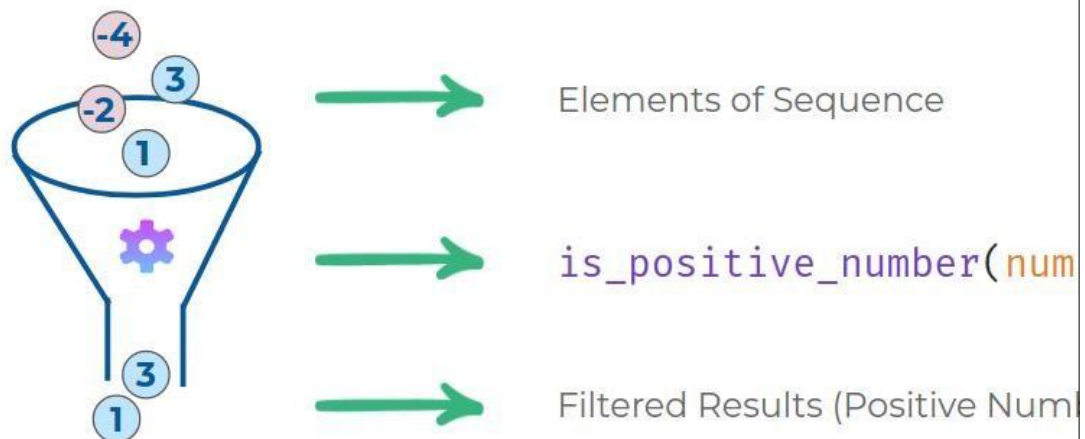
[1, 2, 3, 4]

Filter

`filter()`

method filters the elements of a given sequence based on the result of given function.

The function should return True/False



Code

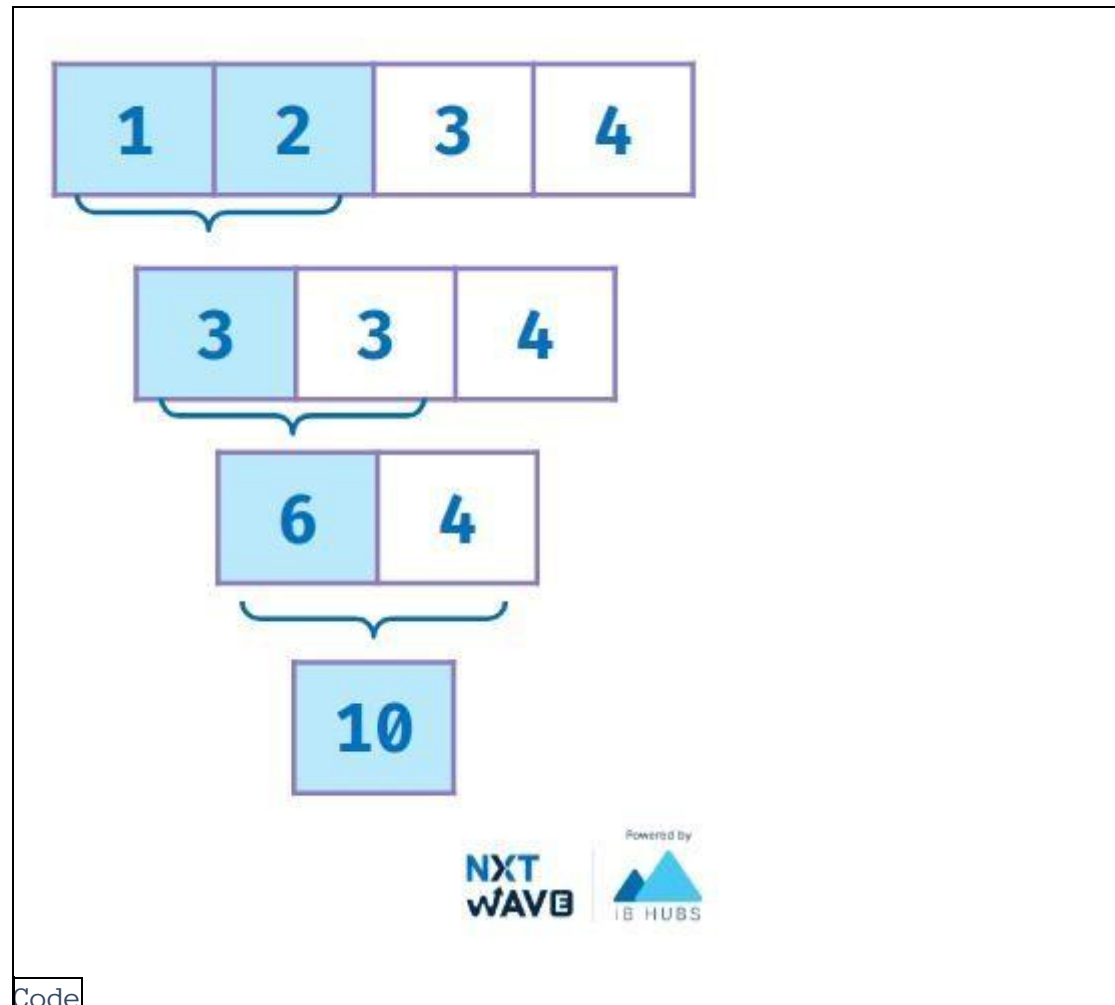
Output

[1, 3]

Reduce

`reduce()`

function is defined in the functools module.



Code

```
from functools import reduce
```

```
def sum_of_num(a, b):  
    return a+b  
list_a = [1, 2, 3, 4]  
sum_of_list = reduce(sum_of_num, list_a)  
print(sum_of_list)
```

Output

10

Scope & Namespaces

Object

In general, anything that can be assigned to a variable in Python is referred to as an `object`.

Strings, Integers, Floats, Lists, Functions, Module etc. are all

"A"

1.25

1 2 3

NXT
WAVE

Powered by
IB HUBS

objects.

Identity of an Object

Whenever an object is created in Python, it will be given a unique identifier (id). This unique id can be different for each time you run the program.

"A"

Id - 140035229724336

1 2 3

Id - 139630925071104

NXT
WAVE

Powered by
IB HUBS

Every object that you use in a Python Program will be stored in Computer Memory

The unique id will be related to the location where the object is stored in the .

Name of an Object

Name	or	Identifier	is simply a name given to an object.
------	----	------------	--------------------------------------

Code

```
a = "Hello"  
b = "World"
```

a

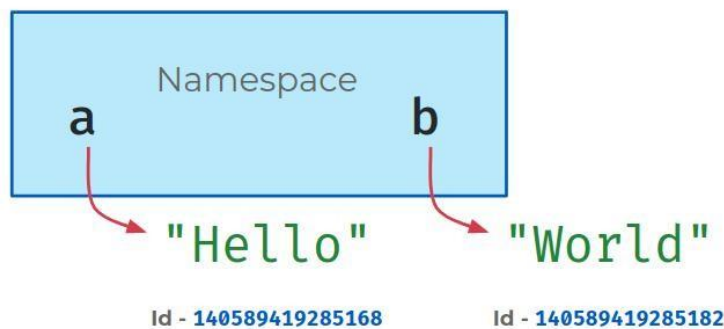
"H

Id - 140

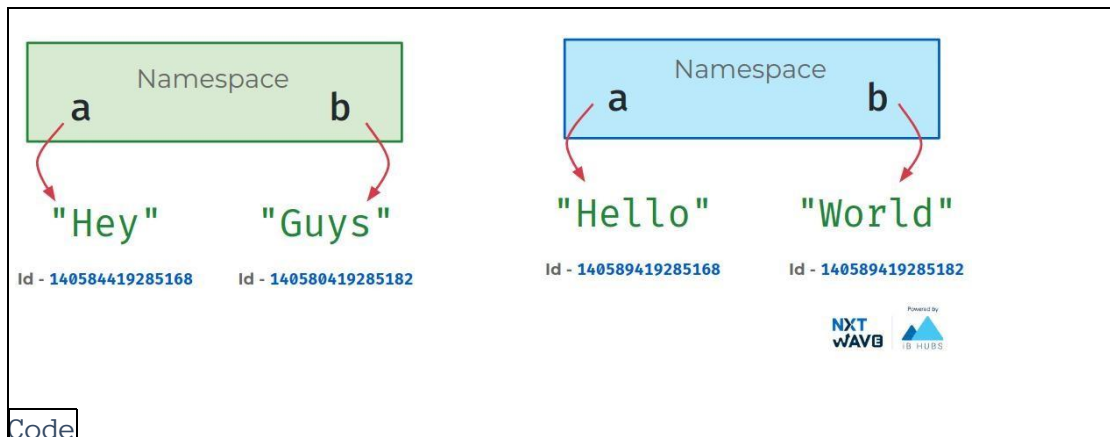
Namespaces

A namespace is a collection of currently defined names along with information about the object that the name references.

It ensures that names are unique and won't lead to any conflict.



Namespaces allow us to have the same name referring to different things in different namespaces.



```
def greet 1():
    a = "Hello"
    print(a)
    print(id(a))
```

```
def greet 2():
    a = "Hey"
    print(a)
    print(id(a))
```

```
print("Namespace -
1")
greet_1()
print("Namespace -
2")
greet_2()
```

Output	

```
Namespace - 1

Hello

140639382368176

Namespace - 2

Hey

140639382570608
```

Types of namespaces

As Python executes a program, it creates namespaces as necessary and forgets them when they are no longer needed.

Different namespaces are:

- Built-in
- Global
- Local

1.

Built-in Namespace

a Python program and exists as

g.

Created when we start executi-in functions like `id()`, `print()` to us from any part of the long as the program is program.

This is the reason that etc. are always available

Global Namespace

all

names defined directly in a module

This namespace is loaded
(outside of all functions)

is loaded, and it lasts until the

It is created when the
program ends.

math

():

3

nt(a + b)

NXT
WAVE



Namespace

can have various

functions

and

classes

.
A new local namespace is created when a function is called, which lasts until the function returns.

Code

```
import math  
a = 2  
  
def foo():  
    b = 3  
    print(a + b)  
  
foo()
```

NXT
WAVE

Powered by
IB HUBS

Scope of a Name

The scope of a name is the region of a program in which that name has meaning.

Python searches for a name from the inside out, looking in the

local

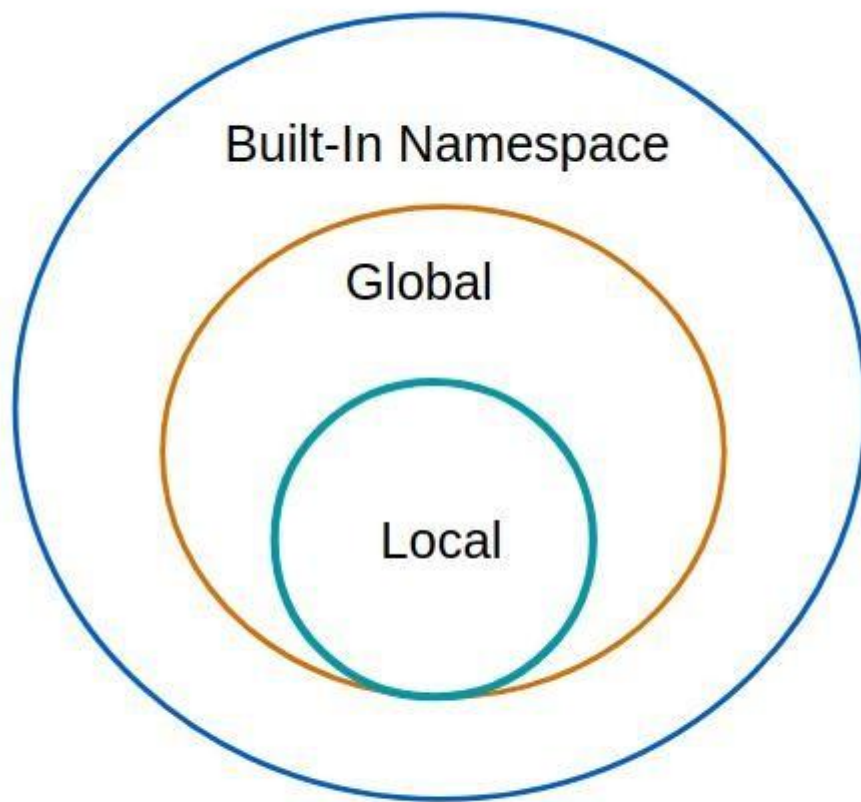
,

global

, and

finally

the built-in namespaces.



NXT
WAVE

Powered by
IB HUBS

Global variables

In Python, a variable defined outside of all functions is known as a **global variable**.

This variable name will be part of **Global Namespace**.

Example1

Code

```
x = "Global Variable"
```

```
print(x)
```

```
def foo():
```

```
    print(x)
```

```
foo()
```

Output

```
Global Variable
```

Global Variable

Example2

Code

```
def foo(): print(x)
```

```
x = "Global Variable"
```

```
foo()
```

Output

Global Variable

Local Variables

In Python, a variable defined inside a function is a local variable.

This variable name will be part of the Local Namespace which will be created when the function is called and lasts until the function returns.

Code

```
def foo():
```

```
    x = "Local Variable"
```

```
    print(x)
```

```
foo()
```

```
print(x)
```

Scope of x

NXT
WAVE

Powered by
IB HUBS

Code

```
def foo(): x = "Local Variable"
```

```
    print(x)
```

```
foo()
```

```
print(x)
```

Output

Local Variable

NameError: name 'x' is not defined

As,

x

is not declared before assignment, python throws an error.

Local Import

Code

```
def foo():  
    import math  
    print(math.pi)  
foo()  
print(math.pi)
```

Output

3.141592653589793

NameError: name 'math' is not defined

Local Variables & Global Variables

Code

```
x = "Global Variable"  
  
def foo(): x = "Local Variable"  
    print(x)  
print(x)  
foo()  
print(x)
```

Output

Global Variable

Local Variable

Global Variable

Modifying Global Variables

global

keyword is used to define a name to refer to the value in Global Namespace.

Code

```
x = "Global Variable"
```

```
def foo(): global x x = "Global
            Change" print(x)
print(x)
foo()
print(x)
Output
```

Global Variable
Global Change
Global Change

Errors & Exceptions

There are two major kinds of errors:

1. Syntax Errors
2. Exceptions

Syntax Errors

Syntax errors are parsing errors which occur when the code is not adhering to Python Syntax.

Code

```
if True print("Hello")
```

Output

SyntaxError: invalid syntax

When there is a syntax error, the program will not execute even if that part of code is not used.

Code

```
print("Hello")
```

```
def greet():
    print("World")
```

Output

SyntaxError: unexpected EOF while parsing

Notice that in the above code, the syntax error is inside the

`greet`

function, which is not used in rest of the code.

Exceptions

Even when a statement or expression is syntactically correct,

it may cause an **error** when an attempt is made to execute it.

Errors detected during execution are called **exceptions**.

Example Scenario

We wrote a program to download a Video over the Internet.

Internet is disconnected during the download

We do not have space left on the device to download the video

Example1

Division Example

Input given by the user

Code is not within expected values.

```
def divide(a, b):  
    return a / b
```

```
divide(5, 0)
```

Output

ZeroDivisionError: division by zero

Example2

Input given by the user is not within expected values.

Code

```
def divide(a, b):  
    return a / b
```

```
divide("5", "10")
```

Output

TypeError: unsupported operand type(s) for /: 'str' and 'str'

Example3

Consider the following code, which is used to update the quantity of items in store.

Code

```
class Store:  
    def __init__(self):  
        self.items = {  
            "milk" : 20, "bread" : 30, }  
  
    def add_item(self, name, quantity):  
        self.items[name] += quantity
```

```
s = Store()
s.add_item('biscuits', 10)
```

Output	
--------	--

KeyError: 'biscuits'

Working With Exceptions

What happens when your code runs into an exception during execution?

The application/program crashes.

End-User Applications

When you develop applications that are directly used by end-users, you need to **handle different possible exceptions** in your code so that the application will not crash.

Reusable Modules

When you develop modules that are used by other developers, you should **raise exceptions** for different scenarios so that other developers can handle them.

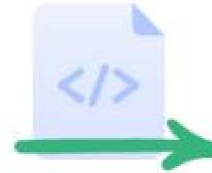
Money Transfer App Scenario

Let's consider we are creating an app that allows users to transfer money between them.



Dev 1

Reusable
Module

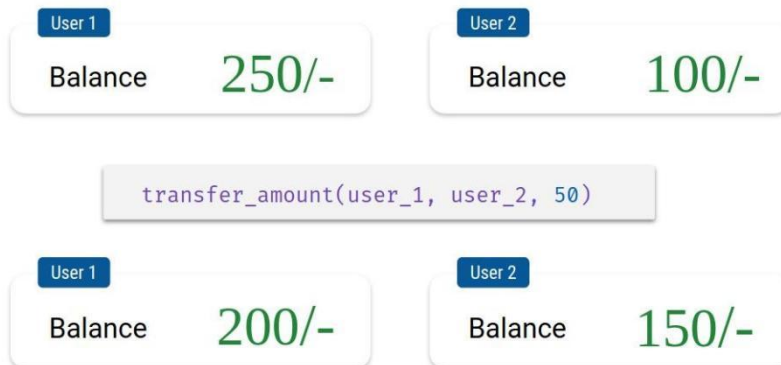


Develop a Class
to model Bank Account

Use Bank Account Class
to implement

Bank Account Class

Example 1



NXT
WAVE

Powered by
IS HUBS

Code

```
class BankAccount:
    def __init__(self, account_number):
        self.account_number = str(account_number)
        self.balance = 0
    def get_balance(self):
        return self.balance
    def withdraw(self, amount):
        if self.balance >= amount:
            self.balance -= amount
        else: print("Insufficient Fund")
    def deposit(self, amount):
```

```
self.balance += amount
```

```
:
```

```
def transfer_amount(acc_1, acc_2, amount):  
    acc_1.withdraw(amount)  
    acc_2.deposit(amount)
```

```
user_1 = BankAccount("001")  
user_2 = BankAccount("002")  
user_1.deposit(250)  
user_2.deposit(100)  
print("User 1 Balance: {}".format(user_1.get_balance()))  
print("User 2 Balance: {}".format(user_2.get_balance()))  
transfer_amount(user_1, user_2, 50)  
print("Transferring 50/- from User 1 to User 2")  
print("User 1 Balance: {}".format(user_1.get_balance()))  
print("User 2 Balance: {}".format(user_2.get_balance()))
```

Output	
--------	--

User 1 Balance: 250/-

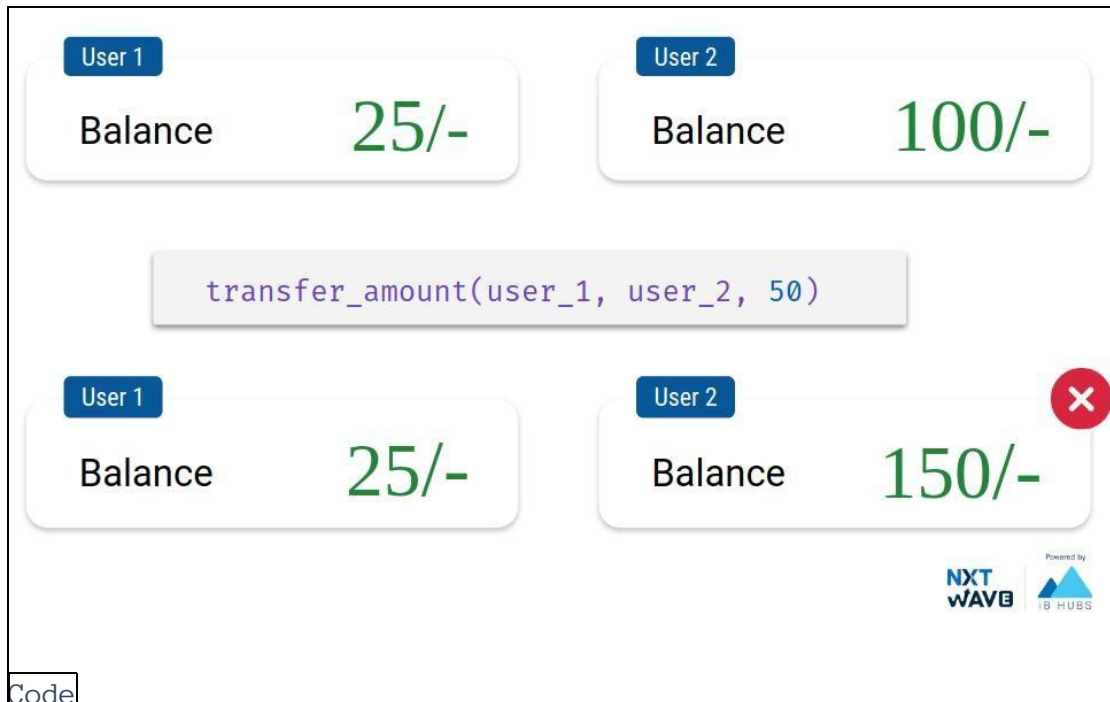
User 2 Balance: 100/-

Transferring 50/- from User 1 to User 2

User 1 Balance: 200/-

User 2 Balance: 150/-

Example2	
----------	--



Code

```
class BankAccount:
    def init (self, account number):
        self.account number = str(account_number)
        self.balance = 0
```

```
def get_balance(self):
    return self.balance
def withdraw(self, amount):
    if self.balance >= amount:
        self.balance -= amount
    else: print("Insufficient
Fund
s")
```

```
def deposit(self, amount)
    self.balance += amount :
```

```
def transfer_amount(acc_1, acc_2, amount):
    acc_1.withdraw(amount)
    acc_2.deposit(amount)
```

```
user_1 = BankAccount("001")
user_2 = BankAccount("002")
user_1.deposit(25)
user_2.deposit(100)
print("User 1 Balance: {}/-".format(user_1.get_balance()))
print("User 2 Balance: {}/-".format(user_2.get_balance()))
transfer_amount(user_1,
user_2, 50)
print("Transferring 50/- from User
1 to User 2")
print("User 1 Balance: {}/-".format(user_1.get_balance()))
print("User 2 Balance: {}/-".format(user_2.get_balance()))
```

```
".format(user_2.get_balance()))
```

Output

User 1 Balance: 25/User 2

Balance: 100/-

Insufficient Funds

Transferring 50/- from User 1 to User 2

User 1 Balance: 25/User 2

Balance: 150/-

Raising Exceptions

When your code enters an unexpected state,
communicate

raise an exception to



it.

Built-in Exceptions

Different exception classes which are raised in different scenarios.

- | | |
|---------------------|-------------------|
| → ZeroDivisionError | → KeyError |
| → EOFError | → Exception |
| → TypeError | → OSError |
| → ValueError | → ArithmeticError |

and many more ...



You can use the built-in exception classes with `raise` keyword to

raise an exception in the program.

Code

We pass message as argument .

```
raise ValueError("Unexpected Value!!")
```

Output

```
ValueError:Unexpected Value!!
```

Bank Account Class

Example1

User 1

Balance

25/-

User 2

Balance

100/-

```
transfer_amount(user_1, user_2, 50)
```

ValueError: Insufficient Funds



NXT
WAVE

Powered by
IB HUBS

Code

```
class BankAccount:
    def __init__(self, account_number):
        self.account_number = str(account_number)
        self.balance = 0

    def get_balance(self):
        return self.balance

    def withdraw(self, amount):
        if self.balance >= amount:
            self.balance -= amount
        else:
            raise ValueError("Insufficient Funds")

    def deposit(self, amount):
        self.balance += amount

def transfer_amount(acc_1, acc_2, amount):
    acc_1.withdraw(amount)
    acc_2.deposit(amount)
```

```
user_1 = BankAccount("001")
user_2 = BankAccount("002")
user_1.deposit(25)
user_2.deposit(100)
print("User 1 Balance: {}/-".format(user_1.get_balance()))
print("User 2 Balance: {}/-".format(user_2.get_balance()))
transfer_amount(user_1, user_2, 50)
print("Transferring 50/- from User 1 to User 2")
print("User 1 Balance: {}/-".format(user_1.get_balance()))
print("User 2 Balance: {}/-".format(user_2.get_balance()))
```

Output	
--------	--

User 1 Balance: 25/User 2
Balance: 100/-
ValueError: Insufficient Funds

Handling Exceptions

Python provides a way to catch the exceptions that were raised so that they can be properly handled.

Exceptions can be handled with	try-except	block.	
--------------------------------	------------	--------	--

Whenever an exception occurs at some line in try block, the execution stops at that line and jumps to except block.

```
try:
    # Write code that
    # might cause exceptions.
except:
    # The code to be run when
    # there is an exception.
```

Transfer Amount

Example 1

User 1 Balance 25/-

User 2 Balance 100/-

```
transfer_amount(user_1, user_2, 50)
```

User 1 Balance 25/-

User 2 Balance 100/-

NXT WAVE Powered by IB HUBS

Code

```
class BankAccount:
    def __init__(self, account_number):
        self.account_number = str(account_number)
        self.balance = 0
```

```
    def get_balance(self):
        return self.balance
    def withdraw(self, amount):
        if self.balance >= amount:
            self.balance -= amount
        else:
            raise ValueError("Insufficient Funds")
    def deposit(self, amount):
        self.balance += amount
```

```
def transfer_amount(acc_1, acc_2, amount):
    try:
        acc_1.withdraw(amount)
        acc_2.deposit(amount)
        return True
    except:
        return False
user_1 = BankAccount("001")
user_2 = BankAccount("002")
```

```
user_1.deposit(25)
user_2.deposit(100)
print("User 1 Balance: {}/-
".format(user_1.get_balance())) print("User 2 Balance:
{}/-".format(user_2.get_balance()))
print(transfer_amount(user_1, user_2, 50))
print("Transferring 50/- from User 1 to User 2")
print("User 1 Balance: {}/-
".format(user_1.get_balance())) print("User 2 Balance:
{}/-".format(user_2.get_balance()))
```

Output	
--------	--

User 1 Balance: 25/User 2
Balance: 100/-
False
Transferring 50/- from User 1 to User 2
User 1 Balance: 25/User 2
Balance: 100/-

Summary	
Reusable Modules	

While developing reusable modules, we need to raise Exceptions to stop our code from being used in a bad way.

End-User Applications	
-----------------------	--

While developing end-user applications, we need to handle Exceptions so that application will not crash when used.

Handling Specific Exceptions

We can specifically mention the exceptions of that specific type.	name of exception	to catch all
Syntax		

```
try:
    # Write code that
    # might cause exceptions.
except Exception:
    # The code to be run when
    # there is an exception.
```

Example 1

Code

```
try:
    a = int(input())
    b = int(input())
    c = a/b
    print(c)
except ZeroDivisionError:
    print("Denominator can't be 0")
except:
    print("Unhandled Exception")
```

Input

5
0

Output

Denominator can't be 0

Example2

Code

Input given by the user is not within expected values.

```
try: a =
    int(input()) b
    = int(input())
    c = a/b print(c)
except ZeroDivisionError:
    print("Denominator can't be 0")
except:
    print("Unhandled Exception")
```

Input

12
a

Output

Unhandled Exception

We can also access the handled exception in an

object

Syntax

```
try:
    # Write code that
    # might cause exceptions.
except Exception as e: #
    The code to be run when
    # there is an exception.
```

Code

```
class BankAccount:
    def __init__(self,
        account_number):
        self.account_number = str(account_number)
        self.balance = 0

    def get_balance(self):
        return self.balance

    def withdraw(self, amount):
        if self.balance >= amount:
            self.balance -= amount
        else:
            raise ValueError("In
                sufficient
                Funds")

    def deposit(self, amount):
        self.balance += amount
```

```
def transfer_amount(acc_1, acc_2, amount):
    try:
        acc_1.withdraw(amount)
        acc_2.deposit(amount)
        return True
    except ValueError as e:
        print(str(e))
        print(type(e))
        print(e.args)
    return False
```

```
user_1 = BankAccount("001")
user_2 = BankAccount("002")
user_1.deposit(25)
user_2.deposit(100)

print("User 1 Balance: {}".format(user_1.get_balance()))
print("User 2 Balance: {}".format(user_2.get_balance()))

print(transfer_amount(user_1, user_2, 50))

print("Transferring 50/- from User 1 to User 2")

print("User 1 Balance: {}".format(user_1.get_balance()))
print("User 2 Balance: {}".format(user_2.get_balance()))
```

Output

```
User 1 Balance: 25/User 2
Balance: 100/-

Insufficient Funds

<class 'ValueError'>

('Insufficient Funds',)

False

Transferring 50/- from User 1 to User 2

User 1 Balance: 25/User 2
Balance: 100/-
```

Handling Multiple Exceptions

We can write multiple exception blocks to handle different types of exceptions differently.

Syntax

```
try:
    # Write code that
    # might cause exceptions.
except Exception1:
    # The code to be run when
    # there is an exception.
except Exception2:
    # The code to be run when
    # there is an exception.
```

Example 1

Code

```
try:
    a = int(input())
    b = int(input())
    c = a/b
    print(c)
except ZeroDivisionError:
    print("Denominator can't be 0")
except ValueError:
    print("Input should be an integer")
except:
    print("Something went wrong")
```

Input

5
0

Output

Denominator can't be 0

Example2

Code

```
try:
    a = int(input())
    b =
    int(input()) c
    = a/b print(c)
except ZeroDivisionError:
    print("Denominator can't be 0")
except ValueError: print("Input
should be an integer")
except:
    print("Something went wrong")
```

Input

12
a

Output

Input should be an integer

Working With Dates & Times

Datetime

Python has a built-in `datetime` module which provides convenient objects to work with dates and times.

Code

```
import datetime
```

Datetime classes

Commonly used classes in the datetime module are:

`date class`

`time class`

`datetime class`

`timedelta class`

Working with 'date' class

Representing Date

A date object can be used to represent any valid `date` (year, month and day).

Code

```
import datetime
```

```
date_object =  
datetime.date(2019, 4, 13)  
print(date_object)
```

Output	
--------	--

2019-04-13

Date Object

Code

```
from datetime import date date_obj  
= date(2022, 2, 31) print(date_obj)
```

Output	
--------	--

ValueError: day is out of range for month

Today's Date

Class method

today()

returns a date object with today's date.

Code

```
import datetime
```

```
date_object =  
datetime.date.today()  
print(date_object)
```

Output	
--------	--

2021-02-05

Attributes of Date Object

Code

```
from datetime import date
```

```
date_object = date(2019, 4, 13)  
print(date_object.year)  
print(date_object.month)  
print(date_object.day)
```

Output	
--------	--

2019

4

13

Working with 'time' Class

Representing Time

A time object can be used to represent any valid `time` (hours, minutes and seconds).

Code

```
from datetime import time
time_object =
time(11, 34, 56)
print(time_object)
```

Output

```
11:34:56
```

Attributes of Time Object

Code

```
from datetime import time
time_object = time(11, 34,
56)
print(time_object)
print(time_object.hour)
print(time_object.minute)
print(time_object.second)
```

Output

```
11:34:56
```

```
11
```

```
34
```

```
56
```

Working with 'datetime' Class

Datetime

The datetime class represents a valid `date and time` together.

Example-1

Code

```
from datetime import datetime
date_time_obj = datetime(2018, 11,
28, 10, 15, 26)
print(date_time_obj.year)
```

```
print(date_time_obj.month)
print(date_time_obj.hour)
print(date_time_obj.minute)
```

Output

```
2018
11
10
15
```

Example-2 It gives the current date and time

Code

```
import datetime
datetime_object = datetime.datetime.now()
print(datetime_object)
```

Output

```
2021-02-05 09:26:08.077473
```

Date Time object

Code

```
from datetime import datetime
date_time_obj = datetime(2018, 11, 28)
print(date_time_obj)
```

Output

```
2018-11-28 00:00:00
```

Formatting Datetime

The datetime classes have

```
strftime(format)
```

method to format the datetime into any required format like

```
mm/dd/yyyy
```

```
dd-mm-yyyy
```

Format Specifier	Meaning	Example
%y	Year without century as a zero-padded decimal number	19, 20, ...
%Y	Year with century as a decimal number	2019, 2020, ...

%b	Month as abbreviated name	Jan, Feb, ...
%B	Month as full name	January, February
%m	Month as a zero-padded decimal number	01, 02, ..., 12
%d	Day of the month as a zero-padded decimal number	01, 02, ..., 31
%a	Weekday as abbreviated name	Sun, Mon, ...
Format Specifier	Meaning	Example
%A	Weekday as full name	Sunday, Monday, ...
%H	Hour (24-hour clock) as a zero-padded decimal number	00, 01, ..., 23
%I	Hour (12-hour clock) as a zero-padded decimal number	01, 02, ..., 12
%p	AM or PM	AM, PM
%M	Minute as a zero-padded decimal number	00, 01, ..., 59
%S	Second as a zero-padded decimal number	00, 01, ..., 59

Code

```
from datetime import datetime

now = datetime.now()
formatted_datetime_1 = now.strftime("%d %b %Y %I:%M:%S %p")
print(formatted_datetime_1)

formatted_datetime_2 =
now.strftime("%d/%m/%Y, %H:%M:%S")
print(formatted_datetime_2)
```

Output	
--------	--

05 Feb 2021 09:26:50 AM
05/02/2021, 09:26:50

Parsing Datetime

The class method

```
strptime()
```

creates a `datetime object` from a given string representing date and time.

```
Code
from datetime import datetime
date_string = "28 November, 2018"
print(date_string)

date_object =
datetime.strptime(date_string,
"%d %B, %Y")
print(date_object)
```

Output	
--------	--

28 November, 2018

2018-11-28 00:00:00

Working with 'timedelta' Class

Timedelta object represents

duration

Example1

Code

```
from datetime import timedelta
delta =
timedelta(days=365,
hours=4)
print(delta)
```

Output

365 days, 4:00:00

Example2

Code

```
from datetime import timedelta, datetime
delta = timedelta(days=365)
current_datetime = datetime.now()
print(current_datetime)
next_year_datetime = current_datetime + delta
print(next_year_datetime)
```

Output

2021-02-05 09:28:30.239095

2022-02-05 09:28:30.239095

Calculating Time Difference

Code

```
import datetime

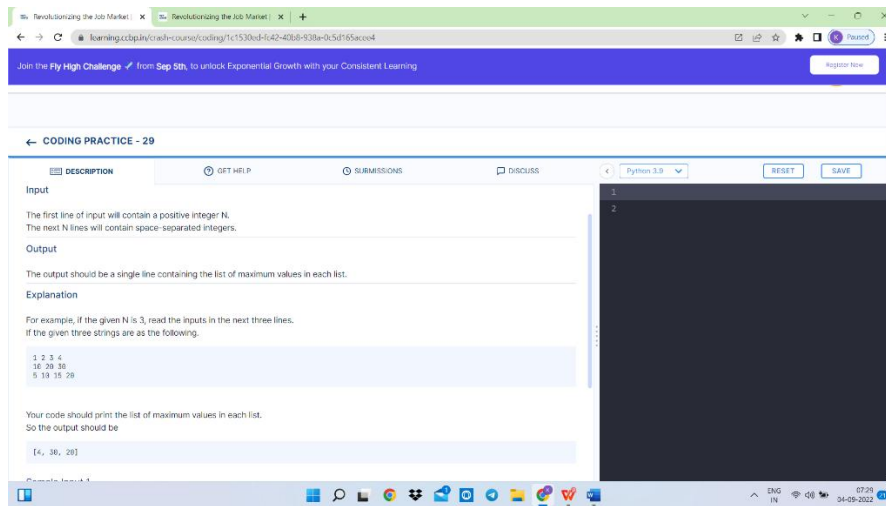
dt1 = datetime.datetime(2021, 2, 5)
dt2 = datetime.datetime(2022, 1, 1)
duration = dt2 - dt1
print(duration)
print(type(duration))
```

Output

330 days, 0:00:00

<class 'datetime.timedelta'>

List of maximum values

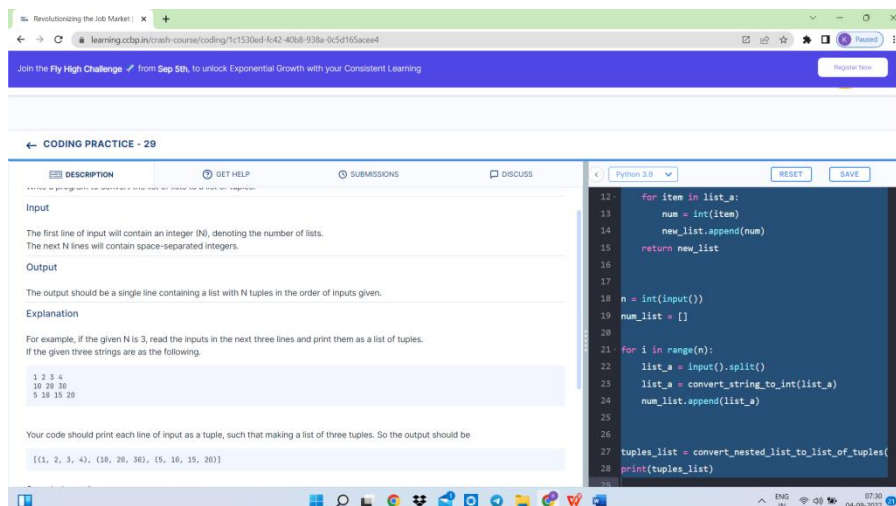


```
def convert_string_to_int(list_a):
    new_list = []
    for item in list_a:
        num = int(item)
        new_list.append(num)
    return new_list

n = int(input())
num_list = []
for i in range(n):
    list_a = input().split()
    list_a = convert_string_to_int(list_a)
    max_list = max(list_a)
    num_list.append(max_list)

print(num_list)
```

2. list of lists to list of tuples



```
def convert_nested_list_to_list_of_tuples(nested_list):
```

```
    tuples_list = []
```

```
    for each_list in nested_list:
```

```
        tuple_a = tuple(each_list)
```

```
        tuples_list.append(tuple_a)
```

```
    return tuples_list
```

```
    # Complete this function
```

```
def convert_string_to_int(list_a):
```

```
    new_list = []
```

```
    for item in list_a:
```

```
        num = int(item)
```

```
        new_list.append(num)
```

```
    return new_list
```

```
n = int(input())
```

```
num_list = []
```

```
for i in range(n):
```

```
    list_a = input().split()
```

```
    list_a = convert_string_to_int(list_a)
```

```
    num_list.append(list_a)
```

```
tuples_list = convert_nested_list_to_list_of_tuples(num_list)
```

```
print(tuples_list)
```

3. Accessing nested lists

Join the Fly High Challenge from Sep 5th, to unlock Exponential Growth with your Consistent Learning

Register Now

CODING PRACTICE - 29

DESCRIPTION GET HELP SUBMIT... DISCUSS

The value at index 2 in the tuple at index 1 in the list is 'hockey', similarly the output list should contain the values at given indexes. So the output should be

```
['hockey', 'grapes', 'banana']
```

Sample Input 1

```
3
1 2
0 3
0 1
```

Sample Output 1

```
['hockey', 'grapes', 'banana']
```

Sample Input 2

```
4
0 0
0 2
1 2
1 1
```

Python 3.9

```
1 list_a = [('apple', 'banana', 'orange', 'grapes'), ('cricket', 'football', 'hockey'), ('car', 'bicycle', 'boat')]
2 # Write your code here
3 n = int(input())
4 new_list = []
5 for i in range(n):
6     index = input().split()
7     tuple_index = int(index[0])
8     value_index = int(index[1])
9     value = list_a[tuple_index][value_index]
10    new_list.append(value)
11 print(new_list)
```

27°C Mostly sunny

ENG IN 07:38 04-09-2022

4. remove N in All Tuples

Join the Fly High Challenge from Sep 5th, to unlock Exponential Growth with your Consistent Learning

Register Now

CODING PRACTICE - 29

DESCRIPTION GET HELP SUBMIT... DISCUSS

The output should be a single line containing the list of tuples without the N.

Explanation

For example, if the given N is 3, as 3 is in tuples at index 0 and 2, remove 3 in those tuples and print the list. So the output should be

```
[(1, 2, 4, 5, 6), (2, 4, 6, 8), (1, 5, 7)]
```

Sample Input 1

```
3
```

Sample Output 1

```
[(1, 2, 4, 5, 6), (2, 4, 6, 8), (1, 5, 7)]
```

Sample Input 2

```
6
```

Sample Output 2

```
[(1, 2, 4, 5, 6), (2, 4, 6, 8), (1, 5, 7)]
```

Python 3.9

```
1 num_list = [(1, 2, 3, 4, 5, 6), (2, 4, 6, 8), (1, 3, 5, 7)]
2 # Write your code here
3 n = int(input())
4 new_list = []
5 for tuple_a in num_list:
6     new_tuple = tuple_a
7     if n in tuple_a:
8         n_index = tuple_a.index(n)
9         new_tuple = tuple_a[:n_index] + tuple_a[n_index+1:]
10    new_list.append(new_tuple)
11 print(new_list)
```

27°C Mostly sunny

ENG IN 07:38 04-09-2022

5. Max and Min value in the list of tuples

Revolutionsizing the Job Market | x +

learning.ccbp.in/crash-course/coding/1c1530ed-fc42-40b8-938a-0c5d165acee4

Join the Fly High Challenge ✓ from Sep 5th, to unlock Exponential Growth with your Consistent Learning

Register Now

← CODING PRACTICE - 29

DESCRIPTION GET HELP SUBMIT... DISCUSS Python 3.9 RESET SAVE

For example, if the given N is 3, read the inputs in the next three lines. If the inputs in the next three lines are the following.

```
1 5
3 2
5 8
```

As 5 is the maximum and 1 is the minimum in the values at index zero and 8 is the maximum and 2 is the minimum in the values at index one. So the output should be

```
(5, 1)
(8, 2)
```

Sample Input 1

```
3
1 5
3 2
5 8
```

Sample Output 1

```
(5, 1)
(8, 2)
```

```
1 n = int(input())
2 zero_index_list = []
3 first_index_list = []
4 for i in range(n):
5     values_list = input().split()
6     first_value = int(values_list[0])
7     zero_index_list.append(first_value)
8     second_value = int(values_list[1])
9     first_index_list.append(second_value)
10 zero_index_min_max_tuple = (max(zero_index_list), min(zero_index_list))
11 first_index_min_max_tuple = (max(first_index_list), min(first_index_list))
12 print(zero_index_min_max_tuple)
13 print(first_index_min_max_tuple)
```

Custom Input DEBUG RUN CODE SUBMIT

27°C Mostly sunny 07:38 04-09-2022

6. List of unique tuples

Revolutionsizing the Job Market | x +

learning.ccbp.in/crash-course/coding/1c1530ed-fc42-40b8-938a-0c5d165acee4

Join the Fly High Challenge ✓ from Sep 5th, to unlock Exponential Growth with your Consistent Learning

Register Now

← CODING PRACTICE - 29

DESCRIPTION GET HELP SUBMIT... DISCUSS Python 3.9 RESET SAVE

Explanation

For example, if the given N is 3, read the inputs in the next three lines. If the inputs in the next three lines are the following.

```
1 2 3
5 10 15
10 20 10 30
```

As the third list contains duplicate elements, the output list should not contain the third list. So the output should be

```
[[1, 2, 3], [5, 10, 15]]
```

Sample Input 1

```
3
1 2 3
5 10 15
10 20 10 30
```

Sample Output 1

```
[[1, 2, 3], [5, 10, 15]]
```

Sample Input 2

```
3
1 2 3
5 10 15
10 20 10 30
```

```
1 def convert_string_to_int(list_a):
2     new_list = []
3     for item in list_a:
4         num = int(item)
5         new_list.append(num)
6     return new_list
7 n = int(input())
8 num_list = []
9 for i in range(n):
10     values_list = input().split()
11     values_list = convert_string_to_int(values_list)
12     values_set = set(values_list)
13     is_equal = len(values_list) == len(values_set)
14     if is_equal:
15         num_list.append(values_list)
16 print(num_list)
```

Custom Input DEBUG RUN CODE SUBMIT

27°C Mostly sunny 07:41 04-09-2022

7. maximum, minimum and sum of matrix

def convert_string_to_int(list_a):

new_list = []

for item in list_a:

num = int(item)

new_list.append(num)

return new_list

def get_sum(nested_list):

total = 0

for each_list in nested_list:

total += sum(each_list)

```
return total
```

```
m, n = input().split()
```

```
m, n = int(m), int(n)
```

```
num_list = []
```

```
for i in range(m):
```

```
    list_a = input().split()
```

```
    list_a = convert_string_to_int(list_a)
```

```
    num_list.append(list_a)
```

```
# Write your code here
```

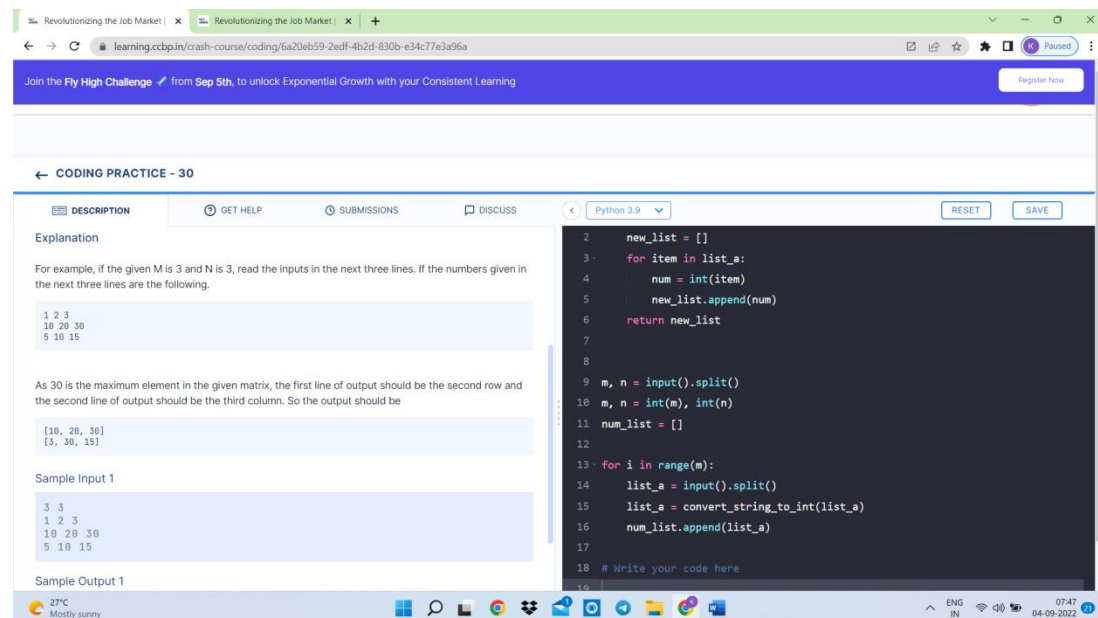
```
print(max(max(num_list)))
```

```
print(min(min(num_list)))
```

```
total = get_sum(num_list)
```

```
print(total)
```

8.Row and column with maximum



```
def convert_string_to_int(list_a):
```

```
    new_list = []
```

```
    for item in list_a:
```

```
        num = int(item)
```

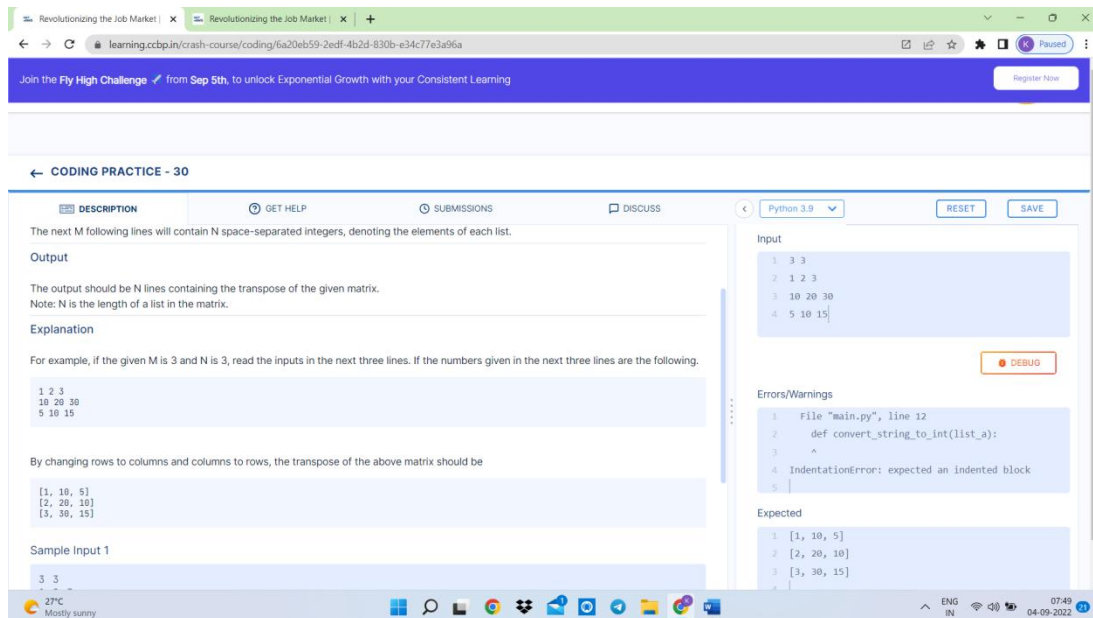
```
        new_list.append(num)
```

```
    return new_list

m, n = input().split()
m, n = int(m), int(n)
num_list = []

for i in range(m):
    list_a = input().split()
    list_a = convert_string_to_int(list_a)
    num_list.append(list_a)
row_wise_max = [] # Maintain row-wise maximum
for row in num_list:
    row_wise_max.append(max(row))
    maximum = max(row_wise_max)
    row_index_containing_max = row_wise_max.index(maximum)
    max_row = num_list[row_index_containing_max]
print(max_row)
column_index_containing_max = max_row.index(maximum)
max_column = []
for row in num_list:
    max_column.append(row[column_index_containing_max])
print(max_column)

10. Transpose matrix
```



```
def get_transpose_of_matrix(matrix, m, n):
```

```
    transpose_matrix = []
```

```
    for i in range(n):
```

```
        row = []
```

```
        for j in range(m):
```

```
            row.append(matrix[j][i])
```

```
        transpose_matrix.append(row)
```

```
    return transpose_matrix
```

```
# Complete this function
```

```
def convert_string_to_int(list_a):
```

```
    new_list = []
```

```
    for item in list_a:
```

```
        num = int(item)
```

```
        new_list.append(num)
```

```
    return new_list
```

```
m, n = input().split()
```

```
m, n = int(m), int(n)
```

```
num_list = []
```

```
for i in range(m):
```

```
    list_a = input().split()
```

```
    list_a = convert_string_to_int(list_a)
```

```
    num_list.append(list_a)
```

```
transpose_matrix = get_transpose_of_matrix(num_list, m, n)
```

```
for row in transpose_matrix:
```

```
    print(row)
```

11.Row-max,min,sum

The screenshot shows a web browser window with a coding practice interface. The page has a blue header with a navigation bar and a main content area. The main content area is divided into two sections: 'DESCRIPTION' and 'CODE'. The 'DESCRIPTION' section contains the problem statement, which asks for the maximum, minimum, and sum of each row in a given matrix. The 'CODE' section shows a Python 3.9 code editor with a solution. The solution defines a function `print_max_min_sum_for_row_wise(num_list)` that iterates over each row in `num_list`, calculates the maximum, minimum, and sum, and appends them to `max_list`, `min_list`, and `sum_list` respectively. The function also defines a helper function `convert_string_to_int(list_a)` to convert a list of strings to a list of integers.

Output

The first line of output should contain the list of the maximum number in each row.
The second line of output should contain the list of minimum number in each row.
The third line of output should contain the list of the sum of each row.

Explanation

For example, if the given M is 3 and N is 3, read the inputs in the next three lines. If the numbers given in the next three lines are the following.

```
1 2 3
10 20 30
5 10 15
```

The first line of output should contain the list of maximum numbers in each row, which are (3, 30, 15).
The second line of output should contain the list of minimum numbers in each row, which are (1, 10, 5).
The third line of output should contain the list of sums of each row, which are (6, 60, 30).
So the output should be

```
[3, 30, 15]
[1, 10, 5]
[6, 60, 30]
```

```
1 def print_max_min_sum_for_row_wise(num_list):
2     max_list = []
3     min_list = []
4     sum_list = []
5     for row in num_list:
6
7         max_list.append(max(row))
8         min_list.append(min(row))
9         sum_list.append(sum(row))
10
11     print(max_list)
12     print(min_list)
13     print(sum_list)
14     # Complete this function
15
16 def convert_string_to_int(list_a):
17     new_list = []
18     for item in list_a:
```

```
def print_max_min_sum_for_row_wise(num_list):
```

```
    max_list = []
```

```
    min_list = []
```

```
    sum_list = []
```

```
    for row in num_list:
```

```
        max_list.append(max(row))
```

```
        min_list.append(min(row))
```

```
        sum_list.append(sum(row))
```

```
    print(max_list)
```

```
    print(min_list)
```

```
    print(sum_list)
```

```
    # Complete this function
```

```
def convert_string_to_int(list_a):
```

```
    new_list = []
```

```
    for item in list_a:
```

```
        num = int(item)
```

```
        new_list.append(num)
```

```
    return new_list
```

```
m, n = input().split()
```

```
m, n = int(m), int(n)
```

```
num_list = []
```

```
for i in range(m):
```

```
    list_a = input().split()
```

```
    list_a = convert_string_to_int(list_a)
```

```
    num_list.append(list_a)
```

```
print_max_min_sum_for_row_wise(num_list)
```

```
12.column-max,min,sum
```

The screenshot shows a web-based coding practice interface. At the top, there's a navigation bar with a logo and text. Below that, a header indicates the current practice session is 'CODING PRACTICE - 30'. The main content area is divided into several sections:

- DESCRIPTION:** Contains the problem explanation. It states: "For example, if the given M is 3 and N is 3, read the inputs in the next three lines. If the numbers given in the next three lines are the following." It then shows a sample input matrix:

```
1 2 3
10 20 30
5 10 15
```

It explains the required output: "The first line of output should contain the list of maximum numbers in each column, which are (5, 20, 30). The second line of output should contain the list of minimum numbers in each column, which are (1, 2, 3). The third line of output should contain the list of sums of each column, which are (16, 32, 48). So the output should be" followed by the expected output:

```
[10, 20, 30]
[1, 2, 3]
[16, 32, 48]
```
- SAMPLE INPUT 1:** Shows the same sample input matrix as above.
- Input:** A text area where the user can input their solution. It shows the sample input matrix being pasted in.
- Your Output:** A text area where the user can input their expected output. It shows the expected output from the description.
- Expected:** A text area showing the expected output for the sample input.

At the bottom of the interface, there's a status bar showing the temperature (27°C), weather (Mostly sunny), and system information (ENG IN, 07:51, 04-09-2022).

```
def get_transpose_of_matrix(matrix, m, n):
```

```
    transpose_matrix = []
```

```
    for i in range(n):
```

```
    row = []
    for j in range(m):
        row.append(matrix[j][i])
    transpose_matrix.append(row)
return transpose_matrix
# Complete this function
```

```
def print_max_min_sum_for_row_wise(num_list):
    max_list = []
    min_list = []
    sum_list = []
    for each_row in num_list:
        max_list.append(max(each_row))
        min_list.append(min(each_row))
        sum_list.append(sum(each_row))
    print(max_list)
    print(min_list)
    print(sum_list)
    # Complete this function

def convert_string_to_int(list_a):
    new_list = []
    for item in list_a:
        num = int(item)
        new_list.append(num)
    return new_list

m, n = input().split()
m, n = int(m), int(n)
num_list = []
for i in range(m):
    list_a = input().split()
    list_a = convert_string_to_int(list_a)
    num_list.append(list_a)
```

```
num_list = get_transpose_of_matrix(num_list, m, n)

print_max_min_sum_for_row_wise(num_list)
```

13.Elements of principle diagonal

The screenshot shows a web-based coding practice environment. The problem is titled "13.Elements of principle diagonal". The description states: "For example, if the given M is 3 and N is 3, read the inputs in the next three lines. If the numbers given in the next three lines are the following." It then shows a 3x3 matrix:


```
1 2 3
10 20 30
5 10 15
```

 The explanation continues: "The principal diagonal elements of the above matrix are (1, 20, 15). So the output should be [1, 20, 15]". It also shows "Sample Input 1" and "Sample Output 1". The code editor on the right shows a Python 3.9 script with a traceback error:


```
1 3 3
2 1 2 3
3 10 20 30
4 5 10 15

Traceback (most recent call last):
  File "main.py", line 26, in <module>
    result = get_principal_diagonal_elements(num_list)
  TypeError: get_principal_diagonal_elements()

Expected
1 [1, 20, 15]
2
```

```
def get_principal_diagonal_elements(matrix, m, n):
    diagonal_result_matrix = []
    for i in range(m):
        if i < n:
            element = matrix[i][i]
            diagonal_result_matrix.append(element)
    return diagonal_result_matrix

# Write your code here
```

```
def convert_string_to_int(list_a):
    new_list = []
    for item in list_a:
        num = int(item)
        new_list.append(num)
    return new_list
```

```
m, n = input().split()
m, n = int(m), int(n)
num_list = []
```

```

for i in range(m):

    list_a = input().split()

    list_a = convert_string_to_int(list_a)

    num_list.append(list_a)

result = get_principal_diagonal_elements(num_list,m,n)

print(result)

```

13.lower triangle

The screenshot shows a web-based coding practice interface. The problem is titled 'lower triangle'. The description states: 'For example, if the given M is 3 and N is 3, read the inputs in the next three lines. If the numbers given in the next three lines are the following.'

Sample Input 1:

```

3 3
1 2 3
10 20 30
5 10 15

```

Sample Output 1:

```

[[
[10, 20],
[5, 10, 15]]

```

The code editor on the right contains the following Python code:

```

1 def print_lower_triangle(matrix):
2     for i in range(len(matrix)):
3         row_list = matrix[i][:i+1]
4         print(row_list)
5     # Complete this function
6
7 def convert_string_to_int(list_a):
8     new_list = []
9     for item in list_a:
10        num = int(item)
11        new_list.append(num)
12    return new_list
13
14
15 m, n = input().split()
16 m, n = int(m), int(n)
17 num_list = []
18

```

```

def print_lower_triangle(matrix):

    for i in range(len(matrix)):

        row_list = matrix[i][:i+1]

        print(row_list)

    # Complete this function

def convert_string_to_int(list_a):

    new_list = []

    for item in list_a:

        num = int(item)

        new_list.append(num)

    return new_list

m, n = input().split()

m, n = int(m), int(n)

num_list = []

for i in range(m):

```

```

list_a = input().split()

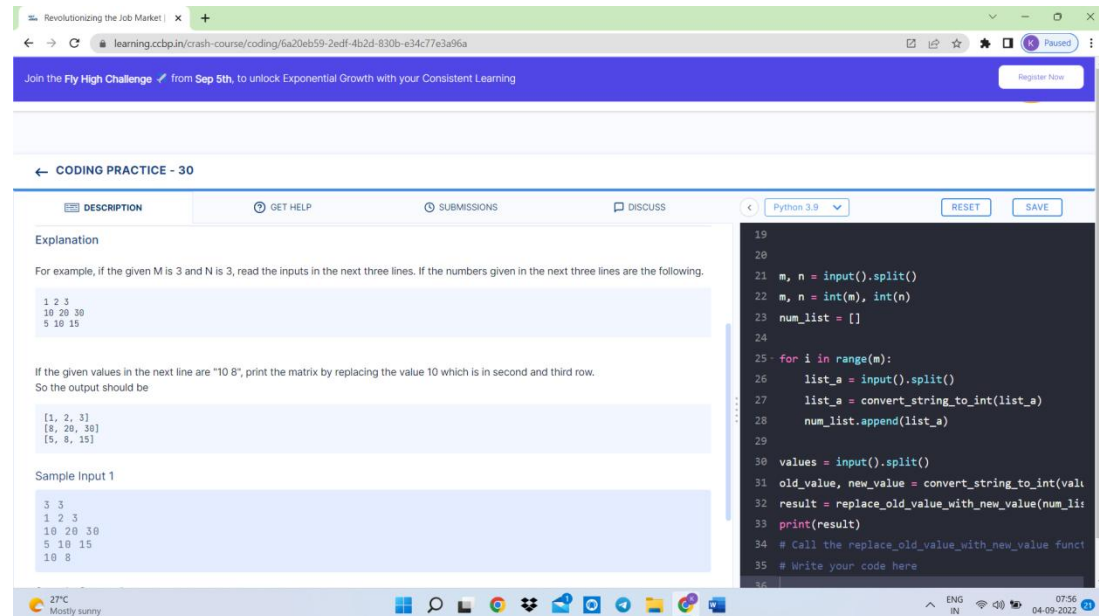
list_a = convert_string_to_int(list_a)

num_list.append(list_a)

print_lower_triangle(num_list)

```

14. Replace a value



```
def replace_old_value_with_new_value(matrix, old_value, new_value):
```

```

    updated_matrix = []

    for row in matrix:

        updated_row = row

        for i in range(len(row)):

            if row[i] == old_value:

                updated_row[i] = new_value

        updated_matrix.append(updated_row)

    return updated_matrix

# Complete this function

```

```
def convert_string_to_int(list_a):
```

```

    new_list = []

    for item in list_a:

        num = int(item)

        new_list.append(num)

    return new_list

```

```

m, n = input().split()
m, n = int(m), int(n)
num_list = []
for i in range(m):
    list_a = input().split()
    list_a = convert_string_to_int(list_a)
    num_list.append(list_a)
values = input().split()
old_value, new_value = convert_string_to_int(values)
result = replace_old_value_with_new_value(num_list, old_value, new_value)
for row in result:
    print(row)

```

15. Add two matrices

The screenshot shows a web-based coding practice environment. The main area displays the problem description for 'Add two matrices'. It explains that for a given M and N, the user must read inputs for two matrices and output their sum. An example is provided with M=3 and N=3. The first matrix is:

```

1 2 3
10 20 30
5 10 15

```

The second matrix is:

```

2 4 6
11 22 33
7 14 21

```

The expected output is:

```

[3, 6, 9]
[21, 42, 63]
[12, 24, 36]

```

On the right side, there is a code editor with a Python 3.9 interpreter. The code includes a comment to call the `add_two_matrices` function. Below the editor, there are buttons for 'Custom Input', 'DEBUG', 'RUN CODE', and 'SUBMIT'. The 'TEST CASE 1' and 'TEST CASE 2' sections show the input and expected output, with a 'Diff' button to compare the user's output with the expected one.

```

def add_two_matrices(first_matrix, second_matrix, m, n):
    result_matrix = []
    for i in range(m):
        row_result = []
        for j in range(n):
            value = first_matrix[i][j] + second_matrix[i][j]
            row_result.append(value)
        result_matrix.append(row_result)
    return result_matrix
# Complete this function

```

```
def convert_string_to_int(list_a):
```

```
    new_list = []
```

```
    for item in list_a:
```

```
        num = int(item)
```

```
        new_list.append(num)
```

```
    return new_list
```

```
def read_matrix_inputs(m):
```

```
    num_list = []
```

```
    for i in range(m):
```

```
        list_a = input().split()
```

```
        list_a = convert_string_to_int(list_a)
```

```
        num_list.append(list_a)
```

```
    return num_list
```

```
m, n = input().split()
```

```
m, n = int(m), int(n)
```

```
first_matrix = read_matrix_inputs(m)
```

```
second_matrix = read_matrix_inputs(m)
```

```
result_matrix = add_two_matrices(first_matrix, second_matrix, m, n)
```

```
for row in result_matrix:
```

```
    print(row)
```

1. PRIFIX AND SUFFIX

```
first_str = input()
second_str = input()
length_1 = len(first_str)
length_2 = len(second_str)
mini = min(length_1, length_2)
last_word=""
first_word=""
for i in range(1,mini+1):
    last_word=first_str[-i]+last_word
    first_word=first_word+second_str[i-1]
    if last_word==first_word:
        break
if last_word==first_word:
    print(last_word)
else:
    print("No overlapping")
```

2. REMOVE WORDS

The screenshot shows a web browser with a URL bar containing a long alphanumeric string. The page has a blue header with a navigation bar. Below the header, there's a section titled "GRAND ASSIGNMENT - 3". The main content area is divided into two columns. The left column contains the problem description, input/output examples, and an explanation. The right column shows a code editor with Python code. The code defines two functions: `the_string(s)` and `remove(sentence, n)`. The `the_string(s)` function concatenates all characters of the input string `s` into a single string `str1`. The `remove(sentence, n)` function splits the input sentence into words, filters out words whose length is not equal to `n`, and then concatenates the remaining words back into a single string using the `the_string` function. The code also includes input handling and a print statement.

Join the Fly High Challenge from Sep 5th, to unlock Exponential Growth with your Consistent Learning

Register Now

← GRAND ASSIGNMENT - 3

DESCRIPTION GET HELP SUBMISSIONS DISCUSS Python 3.9 RESET SAVE

Given a string, write a program to remove all the words with K length.

Input

The first line of the input will contain a string A.
The second line of the input will contain an Integer K.

Output

The output should contain a string after removing all the words whose length is equal to K.

Explanation

For example, string A is "Tea is good for you", k is 3 then output should be "is good."
Here words "Tea", "for", "you" length is equal to 3, so these words are removed from string.

Sample Input 1

```
Tea is good for you
3
```

Sample Output 1

```
is good
```

```
1. def the_string(s):
2.     str1 = ""
3.     for char in s:
4.         str1 += char
5.     return str1
6. def remove(sentence,n):
7.     n_words = []
8.     words = sentence.split(" ")
9.     for word in words:
10.        if not(len(word)) == n:
11.            n_words.append(word)
12.            n_words.append(" ")
13.     return the_string(n_words)
14. a = input()
15. n = int(input())
16. print(remove(a,n))
17.
```

```
def the_string(s):
```

```
    str1 = ""
```

```
    for char in s:
```

```
        str1 += char
```

```
    return str1
```

```
def remove(sentence,n):
```

```
    n_words = []
```

```
    words = sentence.split(" ")
```

```
    for word in words:
```

```
        if not(len(word)) == n:
```

```
            n_words.append(word)
```

```
            n_words.append(" ")
```

```
    return the_string(n_words)
```

```
a = input()
```

```
n = int(input())
```

```
print(remove(a,n))
```

3. ANTI DIAGONALS

Post Attendee - Zoom x Revolutionizing the Job Market x

learning.ccbp.in/crash-course/coding/12e70660-5f6d-473d-9d62-b0bf4dc026e2

Join the Fly High Challenge from Sep 5th, to unlock Exponential Growth with your Consistent Learning

Register Now

← GRAND ASSIGNMENT - 3

DESCRIPTION GET HELP SUBMISSIONS DISCUSS Python 3.9 RESET SAVE

9 10 11 12
13 14 15 16

4 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
So the output should be

```
1
2 5
3 6 9
4 7 10 13
8 11 14
12 15
16
```

```
1 m,n = map(int,input().split())
2 matrix = [list(map(int,input().split())) for _ in range(m)]
3 max_sum = m+n-2
4 for sum in range(max_sum+1):
5     for i in range(sum+1):
6         if i < m and sum - i < n:
7             print(matrix[i][sum-i],end = " ")
8     print()
```

16:03 04-09-2022

4. SIMPLE CALCULATOR

Post Attendee - Zoom x Revolutionizing the Job Market x Revolutionizing the Job Market x

learning.ccbp.in/crash-course/coding/12e70660-5f6d-473d-9d62-b0bf4dc026e2

Join the Fly High Challenge from Sep 5th, to unlock Exponential Growth with your Consistent Learning

Register Now

← GRAND ASSIGNMENT - 3

DESCRIPTION GET HELP SUBMISSIONS DISCUSS Python 3.9 RESET SAVE

If the given operator is "%", print the result of the modulus operation of the two numbers.

Explanation

For example, if the given operator is "+" and the two numbers are 3 and 5. As it is an addition operator, your code should print the sum of the given two numbers (3 + 5), which is 8.

Similarly, if the given operator is "*" and the two numbers are 2 and 5. As it is a multiplication operator, your code should print the result of the multiplication of the given two numbers (2 * 5), which is 10.

Similarly, if the given operator is "-" and the two numbers are 10 and 9. As it is a subtraction operator, your code should print the result of the subtraction of the given two numbers (10 - 9), which is 1.

Sample Input 1

```
3 + 5
```

Sample Output 1

```
8
```

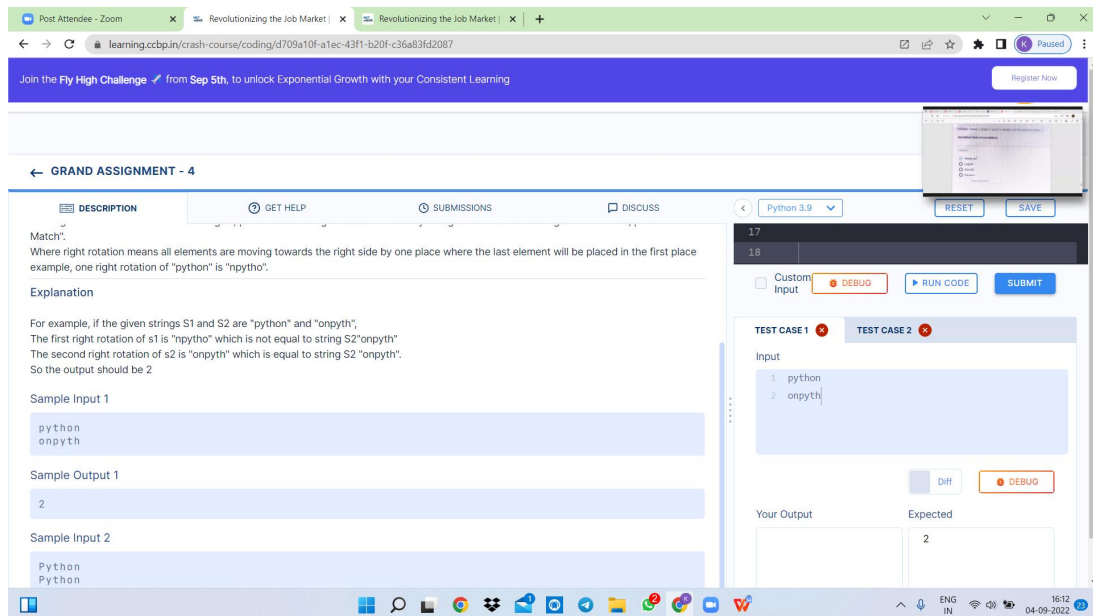
Sample Input 2

```
2 * 5
```

```
1 input1 = input().split()
2 num1 = int(input1[0].strip())
3 num2 = int(input1[2].strip())
4 operator = input1[1].strip()
5
6 if operator == "+":
7     print(num1 + num2)
8 if operator == "-":
9     print(num1 - num2)
10 if operator == "*":
11     print(num1 * num2)
12 if operator == "/":
13     print(num1 / num2)
14 if operator == "%":
15     print(num1 % num2)
```

16:10 04-09-2022

5. STRING ROTATION



```
def main():

    s1 = input()

    s2 = input()

    rotation = getnumberrotation(s1,s2)

    if rotation == -1:

        print("No Match")

    else:

        print(str(rotation))

def getnumberrotation(s1,s2):

    tempstring = s1

    rotation = 0

    while tempstring != s2:

        rotation += 1

        tempstring = s1[len(s1) - rotation:] + s1[0:len(s1) - rotation]

        if rotation >= len(s1):

            return -1

    return rotation

main()
```

6. SECRET MESSAGE-2

Post Attendee - Zoom x Revolutionizing the Job Market x Revolutionizing the Job Market x +

learning.ccbp.in/crash-course/coding/2db521dc-127c-4bd2-8be8-58ea2d687493

Join the Fly High Challenge from Sep 5th, to unlock Exponential Growth with your Consistent Learning [Register Now](#)

← GRAND ASSIGNMENT - 5

DESCRIPTION GET HELP SUBMISSIONS DISCUSS

Python 3.9 RESET SAVE

s	t	u	v	w	x	y	z
19	20	21	22	23	24	25	26

Input

The input will be a string in the single line containing spaces and letters (both uppercase and lowercase).

Output

The output should be a single line containing the secret message. All characters in the output should be in lower case.

Explanation

For example, if the given input is "python", "p" should be replaced with "16", similarly "y" with "25", "t" with "20", "h" with "8", "o" with "15", "n" with "14". So the output should be "16-25-20-8-15-14".

Sample Input 1

```
python
```

Sample Output 1

```
16-25-20-8-15-14
```

```
1
2 import string
3 code={}
4 for i,ch in enumerate(string.ascii_letters):
5     code[ch]=i+1 inpt=input() wasCode=False fo
6
7
8
9
10
11
```

```
import string

code={}

for i,ch in enumerate(string.ascii_letters):
    code[ch]=i+1

inpt=input()

wasCode=False

for ch in inpt:
    ch=ch.lower()

    if ch in code:
        if wasCode:
            print('-',end="")

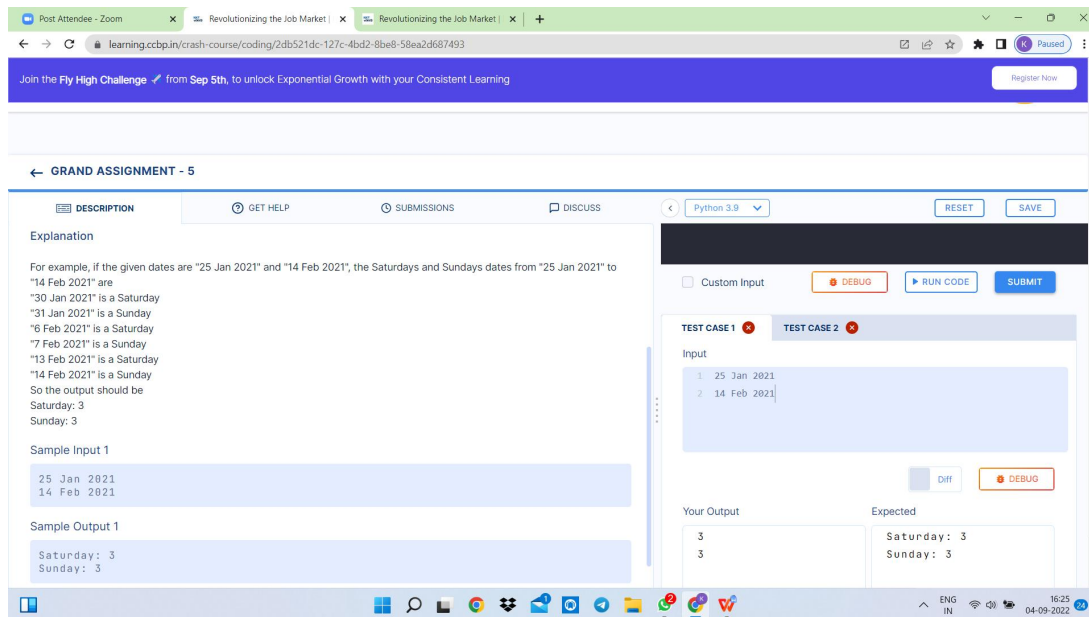
            print(code[ch],end="")

            wasCode=True
        else:
            print(ch,end="")

            wasCode=False

print()
```

7. WEEKENDS



```
import datetime

date_start_str = input()
date_end_str = input()

date_start = datetime.datetime.strptime(date_start_str, '%d %b %Y')
date_end = datetime.datetime.strptime(date_end_str, '%d %b %Y')
day = datetime.timedelta(days=1)

count_saturday = 0
count_sunday = 0

while date_start <= date_end:
    if date_start.isoweekday() == 6:
        count_saturday += 1
    if date_start.isoweekday() == 7:
        count_sunday += 1
    date_start += day

print("Saturday: "+str(count_saturday))
print("Sunday: "+str(count_sunday))
```